



南京大學

本科畢業設計

院 系 软件学院

专 业 软件工程

题 目 MAGIC4FDG——一种基于多

智能体协作与覆盖率反馈迭代

的模糊驱动自动生成系统

年 级 2021 学 号 211250071

学生姓名 柏帅

指导教师 房春荣 职 称 副教授

提交日期 2025 年 6 月 28 日

南京大学本科生毕业论文（设计、作品）中文摘要

题目：MAGIC4FDG——一种基于多智能体协作与覆盖率反馈迭代的模糊驱动自动生成系统

院系：软件学院

专业：软件工程

本科生姓名：柏帅

指导教师（姓名、职称）：房春荣 副教授

摘要：

模糊测试是发现软件安全漏洞的重要手段，其效果高度依赖于模糊驱动程序的质量。当前 OSS-Fuzz 等主流模糊测试平台上的驱动主要由人工编写，不仅耗费大量专家时间，且平均代码覆盖率有限。近年来，大语言模型在代码生成任务中展现出强大能力，为模糊驱动的自动生成提供了新思路。然而，现有基于大语言模型的方法普遍存在覆盖率反馈粒度粗、生成策略单一、缺乏结构化知识积累等不足，单一模型同时承担规划、生成、修复和分析等多重职责时易产生注意力稀释，难以充分探索被测库的深层代码路径。

为此，本文提出 MAGIC4FDG (Multi-Agent Guided Iteration with Coverage for Fuzz Driver Generation)，一种基于多智能体协作与覆盖率反馈迭代的模糊驱动自动生成系统。系统由知识提取、策略规划、代码生成、编译修复、覆盖率分析、状态持久化和缺口诊断七个专职智能体组成有向无环图架构，通过明确的职责分离与差异化上下文注入机制，使每个智能体仅接收与其职责相关的知识子集，有效缓解注意力稀释问题。系统设计了探索-利用双阶段生成范式：首轮由策略规划智能体通过大语言模型场景推理分析 API 语义关系，结合包含十种模糊策略的策略库进行自动匹配，为每个独立的模糊驱动分配差异化的生成方案，实现广度探索；后续轮次中，缺口诊断智能体基于 llvm-cov 提供的行级与分支级覆盖率数据诊断覆盖缺口的根因，输出结构化约束而非直接生成代码，由代码生成智能体据此对各模糊驱动独立进行增量改进，实现深度利用。每个模糊驱动维护隔离的知识空间，跨轮次累积发现的约束条件与正负代码模式，形成持续增长的经验库。系统通过多层级并行执行机制保障大规模实验中的执行效率，全部知识提取、编译、模糊测试与覆盖率收集均在 Docker 容器内完成，确保实

验的隔离性与可复现性。

本文在多个 OSS-Fuzz 平台收录的开源 C/C++ 库上进行实验评估，并与 PromptFuzz、OSS-Fuzz-Gen 等现有工具及人工编写驱动进行对比。实验结果表明，MAGIC4FDG 生成的模糊驱动在覆盖率上显著优于单轮生成基线，消融实验验证了多智能体架构、策略库匹配和覆盖率反馈迭代机制各自对覆盖率提升的贡献。在多个目标库上，MAGIC4FDG 的表现已接近甚至超越现有最优工具与人工编写的驱动程序，展现出良好的通用性与实用性。本研究为模糊测试驱动的自动化构造提供了一种多智能体协作与覆盖率反馈迭代相结合的新范式。

关键词： 软件安全；模糊测试；模糊驱动生成；大语言模型；多智能体系统；迭代优化

南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: MAGIC4FDG——An Automated Fuzz Driver Generation System Based on Multi-Agent Collaboration and Coverage-Feedback Iteration

DEPARTMENT: Software Institute

SPECIALIZATION: Software Engineering

UNDERGRADUATE: Shuai Bai

MENTOR: Associate Professor Chun Rong Fang

ABSTRACT:

Fuzz testing is a critical technique for discovering software security vulnerabilities, and its effectiveness heavily depends on the quality of fuzz driver programs. Currently, fuzz drivers on mainstream platforms such as OSS-Fuzz are predominantly written manually, which is time-consuming for experts and yields limited code coverage on average. While large language models have demonstrated strong capabilities in code generation tasks, existing LLM-based approaches suffer from coarse-grained coverage feedback, limited generation strategies, and lack of structured knowledge accumulation. When a single model simultaneously handles planning, generation, repair, and analysis, attention dilution makes it difficult to explore deep code paths in target libraries.

This thesis presents MAGIC4FDG (Multi-Agent Guided Iteration with Coverage for Fuzz Driver Generation), an automated fuzz driver generation system based on multi-agent collaboration and coverage-feedback iteration. The system employs a directed acyclic graph architecture composed of seven specialized agents responsible for knowledge extraction, strategy planning, code generation, compilation repair, coverage analysis, state persistence, and gap diagnosis. Through explicit separation of concerns and differentiated context injection, each agent receives only the knowledge subset relevant to its role, effectively mitigating attention dilution. The system introduces an Explore-Exploit two-phase generation paradigm: in the first round, the Planner agent analyzes API semantic relationships through LLM scene reasoning and automatically matches strategies from a library of ten fuzzing strategies, assigning differentiated generation plans to each independent fuzz driver for breadth exploration; in subsequent

rounds, the Analyst agent diagnoses root causes of coverage gaps based on line-level and branch-level coverage data from `llvm-cov`, outputting structured constraints rather than generating code directly, while the Generation agent incrementally improves each fuzz driver accordingly for depth exploitation. Each fuzz driver maintains an isolated knowledge space that accumulates discovered constraints and positive/negative code patterns across rounds, forming a continuously growing experience base. The system employs multi-level parallel execution mechanisms to ensure efficiency in large-scale experiments, with all knowledge extraction, compilation, fuzz testing, and coverage collection performed within Docker containers to guarantee experimental isolation and reproducibility.

Experiments are conducted on multiple open-source C/C++ libraries included in the OSS-Fuzz platform, with comparisons against existing tools such as PromptFuzz and OSS-Fuzz-Gen as well as manually written drivers. Results demonstrate that fuzz drivers generated by MAGIC4FDG significantly outperform single-round generation baselines in coverage. Ablation studies validate the individual contributions of the multi-agent architecture, strategy library matching, and coverage-feedback iteration mechanism to coverage improvement. On several target libraries, MAGIC4FDG approaches or exceeds the performance of state-of-the-art tools and manually written drivers, demonstrating strong generalizability and practicality. This work provides a new paradigm combining multi-agent collaboration with coverage-feedback iteration for automated fuzz driver construction.

KEYWORDS: Software Security; Fuzz testing; Fuzz driver generation; Large language models; Multi-agent systems; Iterative optimization

目 录

目 录	V
第一章 引言	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 基于程序分析的模糊驱动生成	2
1.2.2 基于大模型的模糊驱动生成	4
1.2.3 现有方法的不足与本文切入点	5
1.3 研究目标	7
1.4 研究内容	8
1.4.1 主要工作	8
1.4.2 论文结构	9
第二章 相关技术	11
2.1 模糊测试与模糊驱动	11
2.2 LibFuzzer 模糊引擎与覆盖率引导技术	13
2.3 大语言模型与代码生成技术	15
2.4 多智能体协作框架	16
2.5 静态程序分析与 clang AST	18
2.6 本章小结	19
第三章 覆盖率引导的多智能体模糊驱动生成系统 MAGIC4FDG	
的需求分析与概要设计	21
3.1 系统整体概述	21
3.2 需求分析	25

3.2.1	涉众分析	25
3.2.2	功能性需求分析	27
3.2.3	非功能性需求分析	29
3.2.4	系统用例图	30
3.2.5	系统用例描述	31
3.3	系统概要设计	37
3.3.1	系统整体架构设计	37
3.3.2	系统模块划分	38
3.3.3	系统逻辑视图	41
3.3.4	系统开发视图	43
3.3.5	系统进程视图	44
3.3.6	系统物理视图	47
3.4	本章小结	48
第四章 MAGIC4FDG 系统的详细设计与实现		49
4.1	知识提取模块详细设计与实现	49
4.1.1	知识提取模块详细设计	49
4.1.2	AST 解析与知识构建功能实现	53
4.1.3	差异化上下文构建功能实现	56
4.2	策略规划与代码生成模块详细设计与实现	59
4.2.1	策略规划与代码生成模块详细设计	59
4.2.2	探索阶段策略规划功能实现	62
4.2.3	多提示词策略驱动生成功能实现	64
4.3	模糊测试与覆盖率收集模块详细设计与实现	67
4.3.1	模糊测试与覆盖率收集模块详细设计	67
4.3.2	Fork 模式模糊测试与语料库重放功能实现	69
4.3.3	控制流图可达性分析功能实现	70
4.4	缺陷诊断模块详细设计与实现	72
4.4.1	缺陷诊断模块详细设计	72
4.4.2	覆盖率缺口诊断功能实现	74

4.4.3	跨轮次知识累积功能实现	76
4.5	本章小结	77
第五章 实验设计与结果分析		79
5.1	实验设置	79
5.1.1	实验环境	79
5.1.2	目标库选取	79
5.1.3	对比方法	80
5.1.4	评估指标	81
5.1.5	参数设置	81
5.2	主实验结果与分析	82
5.2.1	覆盖率总体结果	82
5.2.2	迭代轮次效果分析	84
5.2.3	策略有效性分析	85
5.3	消融实验	86
5.3.1	消融实验设计	86
5.3.2	消融实验结果与分析	87
5.4	本章小结	88
第六章 总结与展望		89
6.1	研究工作总结	89
6.2	局限性分析	90
6.3	未来研究展望	91
参考文献		93
致 谢		97

第一章 引言

1.1 研究背景与意义

伴随着信息化时代下软件技术的不断发展，软件系统的规模和复杂性都显著增高，软件系统的安全漏洞问题也日益严峻。重大漏洞被攻击者利用，会造成严重的经济损失和安全危害，2017 年的 WannaCry 勒索软件和 2021 年公开的 Log4j 漏洞就曾引发大范围安全事故，导致了数十亿美元的损失^[1]。因此，对于软件开发行业来说，提高漏洞检测和修补的及时性意义重大。

为应对上述问题，软件测试技术不断发展，其中模糊测试（Fuzzing）作为一种高效的自动化测试方法备受关注。模糊测试通过向目标软件系统提供大量随机或变异输入来发现错误和漏洞，在发现未知、隐蔽性强的缺陷时能发挥重要作用^[2]。因方法直接、易部署、可并行且效果显著，模糊测试在业界和学界广受青睐。Google 的 OSS-Fuzz 平台自 2016 年上线以来，已为超过 1000 个开源项目持续发现并修复了上万安全漏洞^[3]，充分证明了模糊测试在保障软件安全方面的重要价值。

模糊测试的效果很大程度上取决于模糊驱动（Fuzz Driver）的质量高低。模糊驱动是一种专门编写的小程序，负责解析模糊器提供的随机输入并调用被测库的相应 API^[4]。高质量的模糊驱动可以有效触发代码的深层次行为，从而找到隐藏的漏洞。然而，编写符合输入和调用规范的驱动并不简单，这需要开发者具备对目标库的深入理解，往往需要投入数小时甚至数天的人工编码工作^[5]。目前 OSS-Fuzz 平台上的模糊驱动主要由人工编写，人工理解规范、开发模糊驱动不仅耗费大量时间，且最终模糊测试平均只能覆盖项目代码约 30%^[3]。由此可见，如何自动生成有效的模糊驱动是提升模糊测试规模与深度的关键课题之一。

近年来，大语言模型取得了飞速发展，为解决这一挑战提供了新思路。得益于海量代码语料的预训练，大语言模型在代码补全、代码生成、程序修复等任务上展现出强大的编程能力^[6]。利用大语言模型自动生成 C/C++ 模糊测试驱

动，不仅可以节约人工编写驱动的工作量，还有机会探索到人工并未考虑到的代码路径，从而提高漏洞检出率。Google 开源安全团队曾报告称，大语言模型生成的模糊驱动在多个项目里明显改进了代码覆盖率，并通过该方式首次找到了 OpenSSL 等关键软件中的安全漏洞^[7]。这些进展表明，基于大语言模型的模糊驱动生成技术有望大幅提升模糊测试效率和覆盖率。

然而，现有基于大语言模型的模糊驱动生成方法仍面临覆盖率反馈粒度粗、单一模型职责过载、缺乏跨轮次知识积累以及生成策略单一等关键挑战，限制了生成驱动的质量与迭代改进的效率。针对上述问题，本文提出将多智能体协作架构与覆盖率反馈迭代机制相结合的技术路线，通过职责分离的智能体协作、细粒度覆盖率反馈闭环和结构化知识累积，实现模糊驱动的高效自动生成与持续优化。这一研究方向对于提升模糊测试的自动化水平、降低人工编写驱动的成本、扩大模糊测试的应用规模具有重要的理论意义和实践价值。

1.2 国内外研究现状

模糊驱动的自动生成是提升模糊测试规模与深度的关键课题。围绕这一目标，国内外研究者形成了两条主要技术路径：基于程序分析的合成方法和基于大语言模型的生成方法。本节将系统梳理两条路径的代表性工作，分析其优势与局限，并在此基础上明确本文的研究切入点。

1.2.1 基于程序分析的模糊驱动生成

早期的模糊驱动生成方法大多利用现有程序的静态或动态分析来合成测试代码。Google 提出的 FUDGE 系统^[8]是这一方向的开创性工作，该系统基于已有客户端代码自动生成 C/C++ 库的模糊驱动。FUDGE 通过分析目标库的使用者代码，提取 API 调用模式并据此合成驱动程序。在 Google 内部的大规模部署中，FUDGE 为数千个不同库生成了上千个模糊驱动，其中超过 200 个已被集成到持续模糊测试服务中，并协助开发者修复了 150 多个缺陷，包括众多安全漏洞。Ispoglou 等人提出的 FuzzGen^[9]进一步发展了这一思路，通过分析目标库的多个消费者程序构建抽象 API 依赖图 (A^2DG)，自动推断 API 之间的调用依赖和参数约束，并据此合成有效的模糊驱动。FuzzGen 的核心贡献在于将 API 使用

模式的提取从单一客户端扩展到多个消费者程序的聚合分析，能够发现更丰富的调用序列组合。

随着研究深入，学者开始关注更复杂的场景。新加坡南洋理工大学的 Zhang 等人针对闭源 SDK 库难以获取源代码的信息缺陷，提出了 APICRAFT 方法^[10]，综合利用头文件、二进制和运行痕迹等静态动态信息来收集 API 间控制和数据依赖，并通过多目标遗传算法自动组合生成高质量模糊驱动。APICRAFT 成功应用于 macOS SDK 的五类接口，自动生成的驱动相比人工编写版本平均提升了 64% 的代码覆盖率，在长达 8 个月的持续模糊测试中发现了 142 个新的安全漏洞，其中 54 个已获 CVE 编号。三星研究院的 Jeong 等人关注到以往根据消费者程序推导 API 调用序列的方法存在“API 序列与业务逻辑混杂”的普遍问题，为此提出 UTOPIA 工具^[11]，从现有单元测试用例中提取正确的 API 调用序列来合成模糊驱动。由于单元测试由 API 开发者精心编写且天然验证了 API 的规范用法，UTOPIA 有效避免了传统方法中 API 调用序列掺入自定义逻辑的干扰。实验显示，UTOPIA 在 55 个开源项目中由 8000 余个单元测试自动生成了约 5000 个模糊驱动，并在约 500 万 CPU 小时的模糊测试中发现了 123 个漏洞。

最新的研究进一步引入形式化模型提升驱动生成质量。Zhang 等人在 USENIX Security 2023 提出 RUBICK^[12]，针对自动生成驱动时 API 调用顺序和依赖处理不足的挑战，采用主动自动机学习算法提炼 API 的控制与数据依赖，将 API 使用模式建模为确定性有穷自动机，并据此合成驱动以调度多组 API 调用序列进行模糊测试。RUBICK 显著改善了生成驱动的测试效率：与前人系统 FuzzGen 相比，其生成的驱动平均覆盖的代码边缘数提升了 50.42%，相比 OSS-Fuzz 平台的人写驱动高出 44.58%。最近，Yu 等人提出的 Hopper^[13]从可解释性角度切入，通过基于类型系统的 API 语义推断自动构建参数间的依赖关系和有效取值约束，在不依赖消费者代码的前提下合成高质量模糊驱动，进一步拓展了程序分析方法的适用范围。

总体而言，基于程序分析的方法能够充分利用现有代码和测试用例的信息，保证生成的驱动遵循真实的 API 调用序列和依赖关系，使得代码覆盖率得以提升并能够发现更多漏洞。然而，这类方法受限于对先验样本的依赖，要生成有意义的驱动，必须提供足够的消费者代码、测试用例或 API 使用实例，并且在应对复杂语义约束和闭源库时仍显力不从心。

1.2.2 基于大模型的模糊驱动生成

大语言模型的崛起为模糊驱动生成带来了新的范式。相比于传统程序分析方法，基于大语言模型的方法具备更强的通用性和自动化潜力，不局限于特定编程语言或 API 接口。根据覆盖率反馈机制的精细程度和迭代策略的复杂度，现有方法可分为以下几个层次。

无迭代反馈的单次生成。最基础的方法是利用大语言模型一次性生成模糊测试相关代码，不进行后续的覆盖率反馈与改进。在模糊测试输入生成方面，Deng 等人提出的 TitanFuzz^[14]首次证明大语言模型可以零样本地生成深度学习库的模糊测试输入，Xia 等人提出的 Fuzz4All^[15]进一步将其推广为面向任意编程语言的通用模糊测试输入生成框架。这些工作虽然针对的是测试输入而非模糊驱动程序，但验证了大语言模型理解 API 语义并生成有效测试代码的能力，为驱动生成提供了技术启发。在模糊驱动生成领域，Google 的 OSS-Fuzz-Gen 项目^[3]采用单次生成模式，通过提供 API 签名和使用示例作为上下文，让大语言模型直接生成驱动代码。Zhang 等人^[4]开展了首个深入的大规模评估实验，精选了 30 个常用 C 项目中的 86 个 API 函数作为测试目标，设计了六种提示策略，在五种最先进的大语言模型上以不同随机温度设置生成了共计 736,430 个候选模糊驱动代码，并与工业界实际使用的人工驱动进行了长达 3.75CPU 年时间的对照模糊测试实验。研究结果明确指出：尽管大语言模型生成模糊驱动展现出巨大的潜力，但当前的大模型生成代码往往存在有效性不足、上下文利用有限等问题，使其生成的驱动在正确性、覆盖深度等方面与手工优化的驱动尚有差距。这类方法的核心局限在于缺乏反馈机制，生成过程是开环的，无法根据实际执行效果进行针对性改进。

粗粒度反馈的迭代生成。为克服单次生成的局限，部分研究引入了覆盖率反馈机制来指导迭代改进。Lyu 等人提出的 PromptFuzz^[5]是这一方向的代表工作，设计了一种迭代提示的模糊测试框架，使用大语言模型根据当前覆盖情况不断生成新的模糊驱动代码，并通过指令化程序生成、错误程序自动校验、基于覆盖率的提示变异等技术来探索未被覆盖的代码区域。在 14 个真实世界库上的测试表明，PromptFuzz 生成的驱动相较 OSS-Fuzz 默认和现有最新工具 Hopper 的分支覆盖率分别提高了 1.61 倍和 1.63 倍，并发现了 49 个崩溃样本中 33 个新的真

实漏洞。然而，PromptFuzz 的反馈粒度停留在 API 组合级别——系统通过维护一个已覆盖 API 组合的集合来判断新生成的驱动是否探索了新的代码区域，但无法获知具体哪些源代码行或分支未被覆盖，更无法诊断未覆盖的根本原因。这种粗粒度的反馈限制了迭代改进的针对性，大语言模型在缺乏精确指导的情况下只能进行相对盲目的变异尝试，难以高效地突破覆盖率瓶颈。

知识增强的生成方法。为弥补大语言模型对被测库理解不足的问题，部分研究引入外部知识源来增强生成质量。Xu 等人提出的 CKGFuzzer^[16]构建了面向代码库的知识图谱，以丰富大语言模型对被测软件行为的理解，并引入多智能体系统规划测试流程，自动执行驱动生成、种子生成、覆盖引导变异和崩溃定位等完整模糊测试环节。在 8 个开源库上的实验显示，融合知识图谱的方法平均提升了 8.73% 的代码覆盖率，并大幅减少了人工分析工作量。然而，CKGFuzzer 的知识图谱是在生成前静态构建的，不随迭代过程动态更新，无法捕捉运行时发现的新约束和模式。此外，其多智能体之间的协作主要是流程串联式的顺序执行，各智能体共享相同的上下文信息而非接收差异化的知识视图，缺乏基于覆盖率反馈的结构化信息传递机制，各智能体无法根据前序智能体的分析结果动态调整自身行为。

1.2.3 现有方法的不足与本文切入点

综合分析上述两条技术路径的研究现状，可以发现传统程序分析方法和新兴的大语言模型方法各有优劣。前者可靠但需要大量先验代码，后者灵活高效但存在准确性和可控性问题。特别是在基于大语言模型的方法中，当前研究仍存在以下共性不足：

第一，覆盖率反馈粒度不足。现有迭代方法的反馈多停留在 API 组合或函数级别，无法提供行级与分支级的精确覆盖信息。这导致大语言模型在改进驱动时缺乏明确的优化方向，只能进行盲目的变异尝试，难以高效突破覆盖率瓶颈。理想的反馈机制应当能够精确指出哪些代码行未被覆盖、未覆盖的根本原因是什么，从而为改进提供明确的指导。

第二，单一模型职责过载。现有方法普遍采用单一大语言模型承担从策略规划到代码生成再到错误分析的全部职责。当上下文信息量增大时，模型的注意力被分散到多个不相关的任务上，各环节的输出质量均受到影响。Liu 等人^[17]的研

究表明，大语言模型在处理长上下文时存在显著的“中间遗忘”现象——模型倾向于关注输入序列的首尾部分而忽略中间内容。类似的注意力分散现象在代码生成场景中同样存在：将过多不同类型的信息（API 签名、覆盖率数据、编译错误、历史代码）塞入同一个提示中会显著降低模型对关键信息的利用效率。这一问题不能简单通过提示词分解来解决，因为模糊驱动生成的各环节（策略规划、代码生成、错误修复、覆盖率分析）不仅需要不同的知识视图，还需要独立的生成温度控制、隔离的状态管理和差异化的上下文窗口分配，这些需求天然指向多智能体协作架构。

第三，缺乏结构化知识积累。多数方法的每轮迭代相对独立，未能建立跨轮次的经验传递机制。成功的代码模式、失败的尝试方向、发现的 API 调用约束等宝贵信息未被系统性地记录和复用，导致后续轮次可能重复已经失败的尝试，或者遗忘已经验证有效的模式，迭代效率低下。

第四，生成策略单一且缺乏自适应能力。现有方法通常采用固定的提示词模板生成所有驱动，未能根据目标库的 API 特征自适应地选择最合适的模糊策略。不同类型的库（如解析类、加密类、图形处理类）具有截然不同的 API 调用模式和状态管理逻辑，采用统一的提示词难以引导大语言模型生成针对性的测试代码。例如，解析类库需要将原始字节直接喂入解析函数以最大化路径覆盖，而有状态库则需要构造多步 API 调用序列以探索状态转移逻辑——这两种场景所需的代码结构、输入分配方式和资源管理模式完全不同，单一提示词无法同时兼顾。理想的方案应当维护一组差异化的生成策略，每种策略以独立的提示词后缀形式指导大语言模型采用特定的代码生成模式，并根据目标库的 API 特征自动匹配最合适的策略组合。

本文正是针对上述四方面不足，提出基于多智能体协作与覆盖率反馈迭代的技术方案。通过职责分离的多智能体架构解决注意力稀释问题，通过行级覆盖率反馈闭环实现精准的迭代引导，通过独立演化轨迹与知识累积实现跨轮次的经验传递，通过策略库与自动匹配实现对不同库的自适应生成。

1.3 研究目标

本文致力于解决现有基于大语言模型的模糊驱动生成方法在反馈粒度、职责分配、知识积累和策略适配方面的不足，提出并实现一个基于多智能体协作与覆盖率反馈迭代的端到端模糊驱动自动生成系统——**MAGIC4FDG (Multi-Agent Guided Iteration with Coverage for Fuzz Driver Generation)**。围绕这一总体目标，本文从架构设计、反馈机制、迭代策略和实验验证四个维度展开研究。

在架构层面，本文的目标是设计并实现由七个专职智能体组成的有向无环图协作架构，通过明确的职责分离和差异化上下文注入机制，使知识提取、策略规划、代码生成、编译修复、覆盖率分析和缺口诊断等环节各由专门的智能体负责。每个智能体仅接收与其职责相关的知识子集，从而避免单一模型因上下文过载而产生的注意力稀释问题，提升各环节的输出质量。

在反馈层面，本文的目标是构建基于 **llvm-cov** 的行级与分支级覆盖率反馈闭环。通过专门的缺口诊断智能体对覆盖缺口进行根因分析，识别未覆盖代码行的具体原因（如缺少特定前置条件、未触发特定分支等），并将分析结果转化为结构化的约束描述，为代码生成智能体提供精准的改进方向，使每轮迭代都有明确的优化目标。

在迭代层面，本文的目标是设计探索-利用双阶段生成范式与独立演化轨迹模型。首轮通过大语言模型的场景推理能力和多样化的策略库实现广度探索，建立多条针对不同 **API** 场景的独立生成轨迹；后续轮次基于约束反馈对各轨迹独立做增量改进，实现深度利用。每条轨迹维护隔离的知识空间，跨轮次累积发现的约束条件与有效代码模式，形成持续增长的经验库，实现经验的系统性传递与复用。

在评测层面，本文的目标是在 **OSS-Fuzz** 平台收录的多个开源 **C/C++** 库上进行系统性实验评估，与 **PromptFuzz**、**OSS-Fuzz-Gen** 等现有最优工具及人工编写驱动进行全面对比，并通过消融实验验证多智能体架构、策略库匹配和覆盖率反馈迭代机制各自对系统性能的贡献，为所提方法的有效性提供充分的实证支持。

1.4 研究内容

1.4.1 主要工作

围绕上述研究目标，本文的主要工作聚焦于 **MAGIC4FDG** 系统的体系化设计与工程实现，并在方法层面系统回答”如何通过多智能体协作提升生成质量、如何通过覆盖率反馈实现精准迭代、如何通过知识累积提升迭代效率”等关键问题。具体包括以下四个方面的工作。

第一，多智能体协作架构设计与实现。本文提出由知识提取、策略规划、代码生成、编译修复、覆盖率分析、状态持久化和缺口诊断七个智能体组成的有向无环图架构。各智能体通过共享的流水线状态进行通信，由有向无环图调度框架按拓扑顺序编排执行。本文设计了差异化上下文注入机制，同一知识库根据不同智能体的职责需求提供不同的信息视图：策略规划智能体接收宏观的 **API** 分类摘要和覆盖率历史，代码生成智能体接收微观的目标 **API** 详情和累积约束，缺口诊断智能体接收完整的调用图和未覆盖源码行。这种信息不对称设计使每个智能体仅关注与其职责相关的信息子集，有效避免了注意力稀释。此外，本文设计了条件路由机制实现自适应的流程控制，当所有变体编译失败时跳过模糊测试环节以节约资源，当覆盖率连续多轮无提升时自动终止迭代。

第二，多提示词策略自适应生成范式。本文设计了包含十种模糊策略的策略库，涵盖解析驱动、多 **API** 序列、往返验证、结构感知、错误路径、有状态、资源边界、回调驱动、差分测试和定向扩展等不同的模糊测试方法。每种策略由两部分组成：元数据（以 **YAML** 格式描述适用条件、最佳场景和不兼容约束）和策略正文（作为独立的提示词后缀直接注入大语言模型的生成提示中，指导模型采用特定的代码结构、输入分配方式和 **API** 调用模式）。不同模糊驱动接收相同的基础提示模板但附加不同的策略后缀，从而在同一轮生成中产出风格迥异、互补性强的驱动代码。在探索阶段，策略规划智能体通过大语言模型的场景推理能力分析目标库的 **API** 语义关系，将策略元数据中的适用条件与 **API** 特征进行匹配，自动为每条独立的生成轨迹选择最合适的策略。当大语言模型推理失败时，系统回退到基于 **API** 特征标签的规则引擎进行确定性匹配，保证策略分配的鲁棒性。本文还设计了基于 **clang** 抽象语法树的精确知识提取机制，通过五阶段流程（语法树解析、宏解析、调用图构建、语义标注和分类推断）为策略匹配

和代码生成提供完整的 API 签名、参数所有权、调用图和宏常量等结构化信息。

第三，覆盖率反馈迭代与知识累积机制。本文构建了基于 `llvm-cov` 的行级与分支级覆盖率反馈闭环，通过 `Docker` 容器化的模糊测试执行环境收集精确的覆盖率数据。模糊测试采用 `fork` 模式运行以实现崩溃隔离，随后通过语料库重放收集完整的覆盖率信息。本文设计了独立演化轨迹模型，每条轨迹维护隔离的知识空间，跨轮次累积发现的约束条件（如“必须先调用初始化函数”）和正负代码模式（如“使用特定参数组合可提升覆盖率”或“某种调用序列导致崩溃”）。缺口诊断智能体基于覆盖率数据分析未覆盖代码行的根因，输出结构化的约束描述而非直接生成代码，由代码生成智能体据此进行定向的增量改进。这种诊断与生成的分离避免了“分析者即生成者”的角色混淆，提升了约束描述的客观性和改进的针对性。本文还设计了多层级的收敛检测机制，当单条轨迹或整体系统的覆盖率连续多轮无提升时自动调整策略或终止迭代，避免无效的资源消耗。

第四，系统工程化实现与实验评估。本文实现了全容器化的执行环境，所有知识提取、编译、模糊测试和覆盖率收集均在 `Docker` 容器内完成，确保实验的隔离性与可复现性。设计了构建缓存机制，目标库的编译产物在首次构建后缓存到宿主机，后续的编译和模糊测试容器通过只读挂载复用缓存，避免重复构建带来的时间开销。通过多层级并行执行机制（多条轨迹并行生成、并行编译修复、并行模糊测试）显著提升系统在大规模实验中的执行效率。在实验评估方面，本文在多个 `OSS-Fuzz` 平台收录的开源 C/C++ 库上进行系统性实验，与 `PromptFuzz`、`OSS-Fuzz-Gen` 等现有工具和人工编写驱动进行全面对比，并通过消融实验验证各核心组件对覆盖率提升的贡献。

1.4.2 论文结构

本文围绕基于多智能体协作与覆盖率反馈迭代的模糊驱动自动生成展开系统研究，全文共分为六章，结构安排如下：

第一章为引言部分，介绍研究背景与动机，分析当前模糊测试驱动构造过程中的自动化瓶颈与人工依赖问题，系统梳理国内外研究现状并指出现有方法的不足，明确本文的研究目标与内容。

第二章为相关技术综述，从模糊测试与 `LibFuzzer` 引擎、覆盖率引导技术、大语言模型与代码生成、多智能体协作框架以及静态程序分析等方向，介绍本

文涉及的基础技术，为后续系统设计奠定技术基础。

第三章为 **MAGIC4FDG** 系统的需求分析与概要设计，首先通过系统流程图展示整体运行流程，然后进行涉众分析、功能性与非功能性需求分析和用例建模，最后通过“4+1”架构视图对系统进行概要设计，明确各模块的职责划分与协作关系。

第四章为 **MAGIC4FDG** 系统的详细设计与实现，按模块详细阐述各智能体的设计方案与核心实现。每个模块包含详细设计（类图与顺序图）和具体实现（关键算法与代码片段），完整展示系统从设计到实现的技术细节。

第五章为实验设计与结果分析，说明实验设置（目标库选取、对比工具、评价指标和实验环境），围绕系统整体性能、迭代反馈有效性、组件消融和横向对比等研究问题设计多组实验，通过定量分析和案例研究验证系统的有效性。

第六章为总结与展望，总结本文的主要研究贡献与实验发现，分析当前系统的局限性，并展望未来在模型泛化、策略自适应和大规模部署等方向的进一步研究思路。

第二章 相关技术

MAGIC4FDG 系统旨在通过多智能体协作与覆盖率反馈迭代机制实现模糊测试驱动的自动生成，其设计与实现融合了多项来自不同领域的关键技术。本章围绕系统所依赖的核心技术基础展开介绍，重点包括模糊测试与模糊驱动的基本原理、LibFuzzer 引擎的覆盖率引导机制与 llvm-cov 工具链、大语言模型的代码生成能力与提示工程方法、多智能体协作框架的架构模式，以及基于 clang 抽象语法树的静态程序分析技术。

上述技术虽然分属软件测试、人工智能和程序分析等不同领域，但在 MAGIC4FDG 系统中形成了高度协同的功能架构：模糊测试定义了生成任务的目标形式与质量评价标准，覆盖率引导技术提供了精确的迭代反馈信号，大语言模型承担了代码合成与约束推理的核心智能，多智能体框架实现了职责分离与协作编排，静态分析则为生成过程提供了结构化的上下文知识。通过对这些技术的系统梳理，本章为后续的系统设计与实验评价奠定理论与方法基础。

2.1 模糊测试与模糊驱动

模糊测试 (Fuzzing) 是一种通过向目标程序提供大量随机或半随机输入来发现软件缺陷的自动化测试技术。这一技术最早由 Miller 等人于 1990 年提出，研究者向 90 个 Unix 工具程序输入随机字符串进行测试，发现其中约 24% 的程序出现崩溃，这种生成随机输入以触发程序异常的方法被正式命名为“fuzzing”^[18]。模糊测试的核心思想是通过尽可能广泛地探索程序的输入空间来暴露潜在缺陷，而几乎不需要人工编写具体的测试用例。经过三十余年的发展，模糊测试已从最初的纯随机输入生成演化为以覆盖率为引导的智能化测试方法，成为当前发现软件安全漏洞最有效的技术手段之一^[19]。

模糊测试的一般流程可以概括为“生成-执行-监控-反馈”的循环过程。首先，模糊引擎从一个初始种子输入集合出发，通过对种子进行各种变异操作（如位

翻转、字节插入删除、整数边界替换等)生成大量新的测试用例。随后,生成的测试用例被送入被测程序执行,执行过程中通过插桩代码收集运行时信息。最后,系统监控程序的执行状态,捕获崩溃、内存越界、断言失败等异常行为,同时记录代码覆盖率信息以指导下一轮输入生成^[20]。在覆盖率引导的模糊测试中,如果某个新输入触发了此前未覆盖的代码边(edge),则该输入被认为具有探索价值并加入种子队列;反之则被丢弃。需要注意的是,主流模糊引擎(如AFL、LibFuzzer)采用的是边覆盖(edge coverage)而非完整路径覆盖(path coverage),即关注控制流图中基本块之间的转移边是否被执行,而非完整的执行路径序列。通过不断重复这一过程,模糊测试能够逐步扩大代码覆盖面,深入探索程序的各种执行路径^[21]。

在模糊测试系统中,输入生成器、执行器和缺陷监控器构成了三大核心组件。输入生成器负责基于已有种子通过变异策略产生新的测试输入,其变异质量直接决定了测试的探索效率。执行器负责将生成的输入送入目标程序并控制其执行过程。缺陷监控器则通过地址消毒器(AddressSanitizer)、未定义行为检测器(UBSan)等工具监控程序运行时的异常行为,并通过插桩代码采集覆盖率等运行时反馈信息^[22]。一般而言,输入生成器与缺陷监控器共同构成模糊引擎(Fuzzer),而执行器的核心则是模糊驱动(Fuzz Driver)。

模糊驱动是连接模糊引擎与被测目标代码的适配程序,其作用是将模糊引擎产生的原始字节流转化为被测库API能够接受的合法调用序列^[19]。具体而言,模糊驱动需要完成输入数据的解析与分配、必要的环境初始化、目标API的正确调用以及资源的清理释放等工作。以当前最广泛使用的LibFuzzer引擎为例,其约定模糊驱动实现一个名为LLVMFuzzerTestOneInput的C接口函数作为测试入口,该函数接收一段原始字节数据和长度参数,开发者需要在函数体内将这些字节转化为对被测库的有效调用^[23]。

模糊驱动的质量对测试效果具有决定性影响。高质量的驱动能够有效地将随机输入映射到目标库的深层代码路径,触发复杂的程序行为;而低质量的驱动可能导致大量输入在浅层校验处即被拒绝,无法探索到有价值的代码区域。此外,驱动本身的正确性和鲁棒性同样重要——如果驱动错误地初始化了API参数或遗漏了必要的前置条件,可能导致虚假崩溃或遗漏真实漏洞。当前工业界的模糊驱动主要依赖开发者基于对目标库的深入理解手动编写,这一过程不仅

耗费大量专家时间，且人工编写的驱动往往只能覆盖有限的 API 组合和调用模式^[5]。因此，如何自动生成高质量的模糊驱动成为提升模糊测试规模与深度的关键课题，也是本文研究的核心问题。

2.2 LibFuzzer 模糊引擎与覆盖率引导技术

LibFuzzer 是 LLVM 项目提供的一个进程内（in-process）、覆盖率引导（coverage-guided）、进化式（evolutionary）的模糊测试引擎^[23]。与传统的进程外模糊器（如 AFL）不同，LibFuzzer 将自身与被测目标库链接为同一个二进制程序，在单一进程内反复调用目标函数并对输入进行变异，避免了进程创建和销毁的开销，从而实现极高的执行吞吐量。如图2-1所示，LibFuzzer 的接口规范要求用户实现一个具有固定签名的目标函数，模糊引擎在运行时反复调用该函数，每次传入不同的变异输入数据。

```
1 int LLVMFuzzerTestOneInput(const uint8_t *Data, size_t Size) {
2     // 1. 输入数据解析与参数构造
3     if (Size < 4) return 0;
4     int flags = *(int*)Data;
5     const char *payload = (const char*)(Data + 4);
6     size_t payload_len = Size - 4;
7     // 2. 环境初始化
8     context_t *ctx = target_init(flags);
9     // 3. 目标API调用
10    target_process(ctx, payload, payload_len);
11    // 4. 资源清理
12    target_cleanup(ctx);
13    return 0;
14 }
```

图 2-1 LibFuzzer 模糊驱动代码结构示例

LibFuzzer 的覆盖率引导机制依赖于 LLVM 的 SanitizerCoverage 插桩框架^[24]。在编译阶段，编译器在程序的每个基本块入口、函数入口或边（edge）处插入轻量级的回调代码，用于在运行时记录哪些代码位置被执行到。LibFuzzer 内部维护一个 8 位边计数器的位图（bitmap），每条控制流边对应位图中的一个槽位，记录该边在单次执行中被命中的次数区间。当新输入的执行产生了与已有语料库不同的位图模式——即触发了新的边或使某条边的命中次数进入新的区间——该输入即被判定为具有探索价值并加入种子队列。这种基于边命中计

数的反馈机制比简单的覆盖/未覆盖二值判断更为精细，能够区分循环执行一次与执行多次的不同行为模式，从而更有效地引导模糊测试深入程序的复杂逻辑分支。

在实际应用中，LibFuzzer 支持 fork 模式 (`-fork=N`) 以实现崩溃隔离。在 fork 模式下，LibFuzzer 主进程充当语料库管理者，依次启动子进程执行模糊测试；每个子进程独立运行一段时间后退出现，将发现的新覆盖输入汇报给主进程合并到全局语料库中。当某个输入触发崩溃时仅影响当前子进程而不会终止整个测试，主进程随即启动新的子进程继续探索。这一机制对于需要长时间运行的模糊测试尤为重要，因为它确保了单个崩溃不会中断对其他代码路径的探索。然而，fork 模式也带来了覆盖率收集的挑战：当工作进程被信号终止时，其覆盖率数据（profraw 文件）可能未被完整写入磁盘。MAGIC4FDG 系统通过语料库重放机制解决了这一问题——在 fork 模式模糊测试结束后，使用收集到的语料库在非 fork 模式下重新执行一次 (`-runs=0`)，确保覆盖率数据的完整性。

覆盖率数据的精确收集依赖于 LLVM 的 `llvm-cov` 工具链^[25]。完整的覆盖率收集流程包含三个阶段：首先，在编译时通过 `-fprofile-instr-generate -fcoverage-mapping` 标志启用源码级覆盖率插桩，程序运行后生成原始覆盖率数据文件（.profraw）；其次，使用 `llvm-profdata merge` 工具将一个或多个 profraw 文件合并为索引格式的 profdata 文件；最后，通过 `llvm-cov export` 命令结合 profdata 文件和带有覆盖率映射信息的二进制文件，导出 JSON 格式的详细覆盖率报告。该报告包含每个源文件中每一行的执行次数和每个分支的覆盖状态，提供了行级（line-level）与分支级（branch-level）的精确覆盖率信息。

这种细粒度的覆盖率数据在 MAGIC4FDG 系统中扮演着核心角色。与仅提供函数级或 API 组合级覆盖信息的粗粒度反馈不同，行级与分支级覆盖率数据能够精确指出哪些代码行未被执行、哪些条件分支未被触发，从而为缺口诊断智能体提供明确的分析目标。这种精确的反馈信号使得迭代改进过程具有明确的方向性，是本系统实现覆盖率反馈闭环的技术基础。

2.3 大语言模型与代码生成技术

大语言模型（Large Language Model, LLM）是基于 Transformer 架构^[26]、在海量文本语料上预训练的深度神经网络模型。Transformer 的核心创新在于自注意力（self-attention）机制，该机制使模型能够在处理序列中的每个位置时直接关注序列中任意其他位置的信息，从而有效捕捉长距离依赖关系——这一特性对于代码生成尤为重要，因为程序中变量的定义与使用、函数的声明与调用往往跨越数十甚至数百行。自 2017 年 Transformer 架构提出以来，大语言模型经历了从 GPT^[27]、BERT^[28]到 GPT-4^[29]的快速演进，模型参数规模从数亿增长到数千亿，在自然语言理解与生成任务上展现出前所未有的能力。研究者采用“大规模预训练 + 指令微调 + 人类反馈强化学习”的训练范式，使模型不仅掌握了丰富的世界知识，还具备了遵循复杂指令、进行多步推理的能力^[30]。

在代码生成任务中，大语言模型同样展现出强大的能力。由于编程语言具有严格的语法结构和明确的语义规则，将大规模代码语料纳入预训练可以使模型学会程序逻辑、API 使用模式和编程惯例。OpenAI 的 Codex 模型在 GPT-3 基础上使用 GitHub 代码进行微调，在 HumanEval 编程基准上实现了 28.8% 的单次通过率^[6]。此后，GPT-4 等通用大语言模型在代码生成能力上持续提升，能够根据自然语言描述生成结构完整、语义正确的程序代码。Anthropic 推出的 Claude 系列模型^[31]在代码生成和推理任务上同样表现优异，其最新版本 Claude Opus 具备强大的长上下文处理能力和结构化输出能力，能够在单次生成中产出数百行的完整程序，并在多轮交互中保持对复杂约束条件的一致理解。

大语言模型在软件测试领域的应用日益广泛。在模糊测试方面，TitanFuzz^[14]首次证明大语言模型可以零样本地生成深度学习库的有效测试输入，Fuzz4All^[15]将这一能力推广到任意编程语言的模糊测试输入生成。在模糊驱动生成方面，PromptFuzz^[5]通过迭代提示策略引导大语言模型持续生成新的驱动代码，CKGFuzzer^[16]则通过知识图谱增强模型对被测库的理解。这些工作表明，大语言模型具备理解 API 语义、推断调用约束并生成符合规范的测试代码的能力，为模糊驱动的自动生成提供了可行的技术路径。

然而，大语言模型在代码生成中也存在固有局限。第一，模型可能产生“幻觉”——生成看似合理但实际不存在的 API 调用或错误的参数类型。第二，上下

文窗口的有限性限制了模型同时处理的信息量，当需要同时考虑 API 签名、调用约束、覆盖率数据和历史代码时，关键信息可能被遗忘^[17]。第三，单次生成的代码质量受限于模型对目标库的理解深度，缺乏执行反馈的开环生成难以保证代码的正确性和覆盖率。MAGIC4FDG 系统通过多智能体架构将不同类型的信息分配给专职智能体处理，通过覆盖率反馈闭环提供执行级验证，通过迭代改进机制逐步提升生成质量，从而系统性地缓解上述局限。

在提示工程方面，大语言模型的生成效果高度依赖于提示 (Prompt) 的设计质量^[32]。有效的提示策略需要从三个维度进行考虑：提示形式决定了任务的表达方式（如零样本、少样本、思维链等），上下文信息为模型提供必要的背景知识（如 API 签名、使用示例、约束条件等），交互策略则通过多轮对话逐步引导模型改进输出。在模糊驱动生成场景中，提示的构造需要平衡信息的完整性与简洁性——过少的上下文导致模型缺乏必要信息，过多的上下文则可能引发注意力稀释。MAGIC4FDG 系统通过差异化上下文注入机制，为不同智能体构造针对性的提示，在信息充分性与注意力集中性之间取得平衡。

MAGIC4FDG 系统采用 Claude Opus 作为核心大语言模型，其具备较强的代码生成能力、较大的上下文窗口和稳定的结构化输出能力，具体的选型论证将在第三章系统设计中展开。

2.4 多智能体协作框架

多智能体系统 (Multi-Agent System, MAS) 是由多个自主智能体组成的系统，各智能体通过协作、协商或竞争来完成单个智能体难以独立完成的复杂任务。在基于大语言模型的多智能体系统中，每个智能体通常由一个大语言模型实例驱动，具有特定的角色定义、知识范围和行为规范，智能体之间通过结构化消息进行通信与协调。相比于使用单一模型处理所有任务，多智能体架构具有三方面优势：第一，职责分离使每个智能体仅需关注其专业领域的信息，避免了上下文过载导致的注意力稀释；第二，独立的智能体实例可以采用不同的生成参数（如温度、最大令牌数），针对不同任务特性进行优化；第三，模块化的架构设计便于独立测试、替换和扩展各个组件。

在协作架构模式方面，多智能体系统的组织方式直接影响其协作效率和任

务完成质量。常见的协作架构包括：流水线架构（Pipeline），各智能体按固定顺序依次处理，前一个智能体的输出作为后一个的输入；黑板架构（Blackboard），所有智能体共享一个全局状态空间，各自读取和写入信息；合同网协议（Contract Net），通过任务发布和竞标机制动态分配工作；以及有向无环图架构（DAG），智能体之间的执行顺序由依赖关系决定，支持条件分支和并行执行。每种架构适用于不同的任务特性：流水线适合严格顺序的处理流程，黑板适合需要全局信息共享的场景，合同网适合动态任务分配，DAG 则适合具有复杂依赖关系和条件逻辑的工作流。

近年来，基于大语言模型的多智能体框架在软件工程领域取得了显著进展。MetaGPT^[33]提出了一种模拟软件公司组织结构的多智能体框架，通过产品经理、架构师、工程师等角色的协作完成从需求到代码的全流程开发。ChatDev^[34]则模拟了软件开发团队的沟通模式，通过对话驱动的方式协调多个智能体完成软件开发任务。微软提出的 AutoGen^[35]采用可定制的对话式多智能体架构，支持智能体之间通过多轮对话进行协作，在代码生成和数学推理等任务上表现出色，但其对话驱动的交互模式更适合探索性任务，对于具有明确依赖关系和固定流程的工作流（如模糊驱动生成的编译-测试-分析循环）则不如 DAG 架构高效。在模糊测试领域，CKGFuzzer^[16]引入了多智能体系统来规划测试流程，但其智能体间的协作主要是流程串联式的顺序执行，各智能体共享相同的上下文信息，缺乏差异化的知识视图和基于反馈的动态调整能力。

LangGraph^[36]是 LangChain 团队推出的多智能体编排框架，专门用于构建有状态的、多参与者的的大语言模型应用。LangGraph 的核心抽象是 StateGraph——一个以共享状态为中心的有向图，图中的节点代表智能体或处理函数，边代表执行流转关系。LangGraph 支持条件路由（Conditional Edge），允许根据当前状态动态决定下一步执行哪个节点，这使得工作流能够根据运行时条件（如编译是否成功、覆盖率是否提升）自适应地调整执行路径。此外，LangGraph 原生支持状态持久化（Checkpointing），能够在任意节点保存和恢复完整的执行状态，为长时间运行的迭代任务提供了断点恢复能力。

MAGIC4FDG 系统采用基于 LangGraph 的 DAG 架构作为多智能体协作的基础设施。模糊驱动生成的各环节（知识提取 → 策略规划 → 代码生成 → 编译修复 → 模糊测试 → 覆盖率分析 → 缺口诊断）具有明确的依赖关系，天然适合

DAG 拓扑表达；条件路由机制能够实现自适应的流程控制；状态持久化机制支持跨轮次的知识累积和断点恢复。具体的架构设计决策将在第三章中详细论证。

2.5 静态程序分析与 clang AST

静态程序分析是一类在不执行程序的前提下，通过对源代码或中间表示进行结构和行为建模来推断程序性质的技术^[37]。与动态分析需要实际运行程序不同，静态分析可以在编译阶段或编译前对代码进行全面检查，其分析结果不受特定输入的限制，能够覆盖程序的所有可能执行路径。典型的静态分析技术包括词法分析、语法分析、类型推导、数据流分析、控制流分析和抽象解释等^[38]。在模糊驱动自动生成的场景中，静态分析的核心价值在于从被测库的源代码中提取结构化的 API 知识，为大语言模型的代码生成提供精确的上下文信息。

抽象语法树（Abstract Syntax Tree, AST）是静态分析中最基础也最重要的程序表示形式。AST 是源代码的树形结构化表示，其中每个节点对应一个语法构造单元（如函数定义、变量声明、表达式、语句等），节点之间的层次关系反映了程序的语法结构^[39]。相比于原始源代码文本，AST 消除了空白、注释等无关信息，保留了程序的完整语义结构，是进行类型分析、调用关系提取和语义标注的理想基础。

MAGIC4FDG 系统采用 clang 编译器前端提供的 AST 导出功能（clang -Xclang -ast-dump=json）进行知识提取。clang 是 LLVM 项目的 C/C++ 编译器前端，其 AST 表示具有完整性和精确性两方面优势。在完整性方面，clang AST 能够正确处理 C/C++ 语言的复杂特性，包括宏展开、模板实例化、typedef 链解析和条件编译，这些是基于正则表达式或 tree-sitter 等轻量级解析器难以准确处理的。在精确性方面，clang AST 的 JSON 导出格式以嵌套的节点树形式组织，每个节点包含 id（唯一标识）、kind（节点类型，如 FunctionDecl、ParmVarDecl、CallExpr 等）、type（完整的类型信息，包含解析后的规范类型）、loc（源码位置）和 inner（子节点列表）等关键字段，使得下游分析能够获取精确的函数签名、参数类型、返回值类型和调用关系。

本系统的知识提取流程基于 clang AST 分为五个阶段：语法树解析、宏常量提取、调用图构建、语义标注和分类推断。语法树解析阶段从 FunctionDecl 节

点中提取公开 API 的函数签名和参数类型；宏常量提取阶段通过预处理器输出识别与 API 参数相关的枚举值和宏定义；调用图构建阶段分析 `CallExpr` 节点建立 API 间的调用依赖；语义标注阶段根据参数类型和命名模式推断参数的语义角色；分类推断阶段根据 API 功能特征将其归类为初始化类、解析类等类别。各阶段的详细实现将在第四章中展开。

与其他静态分析方案相比，`clang AST JSON` 方案在模糊驱动生成场景中更为适用。`tree-sitter` 等通用解析器的设计目标是实现快速的增量解析以支持编辑器语法高亮和代码导航，其解析结果不经过完整的编译语义分析阶段，因此无法提供宏展开后的真实代码结构和经过类型推导的完整类型信息——而这些恰恰是模糊驱动生成所必需的（例如，判断某个参数是否为指针类型、某个宏常量的实际数值）。`libclang` 的 Python 绑定虽然提供了 AST 访问接口，但其遍历效率较低且在处理复杂模板时存在稳定性问题。`clang AST JSON` 导出则在 Docker 容器内一次性完成完整的编译和 AST 导出，既保证了分析的精确性，又通过容器化实现了环境隔离和结果缓存。提取得到的结构化知识以 JSON 格式存储，供后续的策略规划和代码生成智能体按需使用。

2.6 本章小结

本章系统介绍了支撑 **MAGIC4FDG** 系统设计与实现的五项核心技术。模糊测试与模糊驱动的基本原理定义了系统的任务目标和输出形式；**LibFuzzer** 引擎的覆盖率引导机制与 `llvm-cov` 工具链为系统提供了精确的行级与分支级覆盖率反馈信号；大语言模型的代码生成能力构成了系统的核心智能，使得从结构化知识到可执行代码的自动转化成为可能；多智能体协作框架提供了职责分离与协作编排的基础设施，使得复杂的生成流程能够被分解为多个专职智能体的协作；静态程序分析与 `clang AST` 技术则为生成过程提供了精确的结构化上下文知识，弥补了大语言模型对被测库理解不足的问题。上述技术共同构成了 **MAGIC4FDG** 系统的技术基础，为后续章节中的系统需求分析、架构设计与实验评价提供了理论支撑。

第三章 覆盖率引导的多智能体模糊驱动生成系统 MAGIC4FDG 的需求分析与概要设计

本章对 MAGIC4FDG 系统进行需求分析与概要设计。本章分为三部分：第一部分为系统整体概述，通过系统流程图展示系统的整体运行流程并对各阶段进行简要介绍；第二部分为需求分析，明确系统的功能性和非功能性需求；第三部分为系统概要设计，通过“4+1”视图对本系统的架构进行描述。

3.1 系统整体概述

模糊测试在保障 C/C++ 开源项目安全性方面发挥着关键作用，但其效果高度依赖于模糊驱动的质量。当前 OSS-Fuzz 等主流平台上的驱动主要由人工编写，不仅耗费大量专家时间，且难以充分覆盖目标库的深层代码路径。现有基于大语言模型的自动生成方法虽然降低了人工成本，但普遍存在覆盖率反馈粒度粗、单一模型职责过载、缺乏跨轮次知识积累以及生成策略单一等问题，限制了生成驱动的质量与迭代改进效率。

为解决上述问题，本文提出并实现了一种基于多智能体协作与覆盖率反馈迭代的模糊驱动自动生成系统 MAGIC4FDG。该系统围绕“知识提取—策略规划—代码生成—编译修复—模糊测试与覆盖率收集—状态持久化—缺陷诊断”的闭环流程展开，采用七个专职智能体节点（其中六个为 LLM 或计算智能体，一个为状态持久化节点）组成的有向无环图架构，通过探索-利用双阶段范式实现模糊驱动的高效自动生成与持续优化。系统的核心设计理念是将复杂的驱动生成任务分解为多个专业化子任务，由不同智能体独立负责，通过共享的流水线状态进行协调，在保持各环节输出质量的同时实现端到端的自动化。

如图3-1所示，MAGIC4FDG 系统的整体运行流程可划分为知识准备、驱动生成与编译修复、模糊测试与覆盖率收集、缺陷诊断与迭代控制四个阶段，各阶段由对应的智能体或节点负责执行，通过条件路由实现自适应的流程控制。

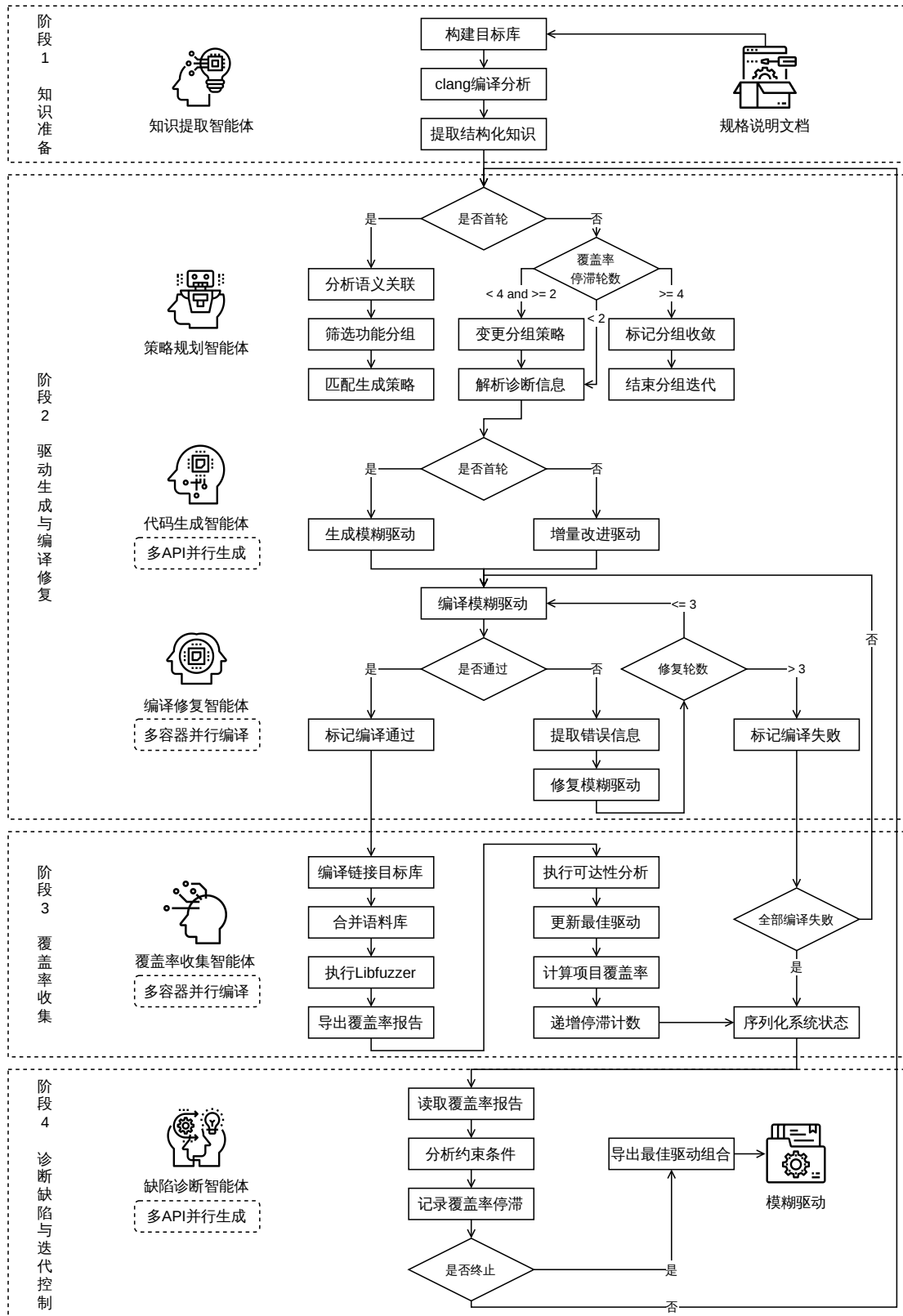


图 3-1 MAGIC4FDG 系统整体流程图

知识准备阶段。系统启动后首先进入知识提取智能体（Knowledge Agent），该智能体负责从目标库中提取结构化的 API 知识。具体而言，知识提取智能体

在 Docker 容器内执行目标库的编译构建，随后通过 clang 编译器前端对目标库的头文件和源文件执行抽象语法树（Abstract Syntax Tree, AST）导出，从中提取公开 API 的函数签名、参数类型与语义标注、函数间调用图关系以及宏常量与枚举定义等结构化信息。构建产物被缓存至宿主机持久化存储，后续轮次可直接挂载缓存跳过重复构建。该阶段仅在系统首次运行时执行一次，提取结果作为全局知识库在后续所有迭代轮次中复用。

驱动生成与编译修复阶段。该阶段由策略规划智能体（Planner Agent）、代码生成智能体（Generation Agent）和编译修复智能体（Patching Agent）三个智能体协作完成。

策略规划智能体基于知识提取智能体提取的结构化知识进入策略规划环节。在首轮迭代（探索阶段），该智能体加载策略库中十种预定义模糊策略的元数据，每种策略以结构化格式声明其适用条件、最佳场景和不兼容约束，同时附带一段独立的策略正文作为提示词后缀。策略规划智能体调用大语言模型分析目标库 API 之间的语义关系与功能分组，将策略元数据中的适用条件与目标库的 API 特征进行匹配，为每个独立的模糊驱动选择最合适的生成策略。这一匹配过程的核心在于：不同模糊驱动将在后续的代码生成环节接收完全不同的策略提示词后缀，从而产出风格迥异、互补性强的驱动代码。若大语言模型调用失败，系统回退至基于调用图的规则引擎进行确定性的分组与策略匹配。在后续轮次（利用阶段），策略规划智能体不再调用大语言模型，而是根据每个模糊驱动的覆盖率停滞情况决定是否切换策略或标记为已收敛，并从缺陷诊断结果中提取额外的目标 API 补充至对应驱动的生成上下文中。

代码生成智能体根据策略规划智能体的规划结果并行生成模糊驱动代码。该智能体的提示由三部分组成：基础模板（包含库名、接口规范和通用约束）、API 知识上下文（目标 API 的函数签名、参数类型和调用关系）、以及策略提示词后缀（由策略规划智能体匹配的策略正文，指导模型采用特定的代码结构和输入分配方式）。首轮为从零生成模式，由大语言模型基于上述三部分提示合成完整的驱动程序；后续轮次为增量改进模式，提示中额外包含该驱动当前的最佳代码、缺陷诊断智能体输出的结构化约束列表与未覆盖代码簇、以及该驱动跨轮次累积的经验知识，指令要求保留已有覆盖路径并定向扩展新路径。

编译修复智能体对代码生成智能体生成的所有驱动变体执行编译验证与错

误修复。对每个变体，该智能体在 **Docker** 容器内执行带内存错误检测插桩的编译，将驱动与目标库进行静态链接。若编译成功则标记为通过；若编译失败，则提取编译器输出的错误信息，将驱动代码与错误信息注入修复提示模板，调用大语言模型生成修复后的代码并重新编译，最多重试三次。

在编译修复智能体完成后，系统执行第一个条件路由判断：若当前轮次存在至少一个编译成功的变体，则流程进入下一阶段；若所有变体均编译失败，则跳过模糊测试环节直接进入状态持久化节点，避免无效的资源消耗。

模糊测试与覆盖率收集阶段。覆盖率收集智能体 (**Coverage Agent**) 负责对所有编译成功的驱动变体执行模糊测试并收集精确的覆盖率数据。对每个变体，该智能体在 **Docker** 容器内依次执行以下步骤：首先以覆盖率插桩标志重新编译驱动并与目标库链接；随后合并种子语料库与历史累积语料库，以 **LibFuzzer** 的 **fork** 模式运行模糊测试，该模式确保每个测试输入在独立子进程中执行，单个崩溃不会终止整个测试过程；模糊测试结束后，通过语料库重放机制重新执行所有发现的输入以收集完整的覆盖率数据；最后通过 **LLVM** 工具链合并原始覆盖率数据并导出行级与分支级覆盖率报告。此外，覆盖率收集智能体还执行控制流图可达性分析，标记结构上不可达的代码行，避免后续分析阶段对这些行进行无意义的诊断。在覆盖率数据收集完成后，该智能体对每个模糊驱动执行最佳版本更新：若当前轮次的覆盖率优于该驱动的历史最佳记录，则更新其最佳代码和覆盖率数据；否则递增该驱动的停滞计数。最后计算所有驱动变体覆盖行的并集作为项目级覆盖率指标。

覆盖率收集完成后，系统进入状态持久化节点 (**Checkpoint Node**)。该节点不调用大语言模型，仅将当前完整的流水线状态序列化为检查点文件进行持久化存储，同时保存每个变体的源码文件，支持从任意轮次的检查点恢复执行。

缺陷诊断与迭代控制阶段。缺陷诊断智能体 (**Analyst Agent**) 负责对每个活跃模糊驱动的覆盖率缺陷进行根因分析。对每个驱动，该智能体读取其当前最佳代码、未覆盖代码行列表及函数级覆盖率数据，将这些信息与 **API** 知识上下文注入分析提示模板，调用大语言模型进行诊断。模型输出结构化的分析结果，包含约束列表（触发特定分支所需的前置条件）、未覆盖代码簇（按功能分组的未覆盖区域）和新发现的知识更新。这些约束和知识被追加到该驱动的隔离知识空间中，跨轮次持续累积形成经验库，为后续轮次的增量改进提供精确的指

导信息。

在缺陷诊断智能体完成分析后，系统执行第二个条件路由判断，检查以下四个终止条件：项目级覆盖率达到用户设定的目标值、迭代轮次达到上限、所有模糊驱动均已标记为收敛、或项目级覆盖率连续多轮无显著提升。满足任一条件则终止迭代，输出每个模糊驱动的最佳版本并生成实验报告；否则系统回到策略规划智能体开始新一轮迭代，进入利用阶段的闭环优化。

3.2 需求分析

MAGIC4FDG 系统的核心功能是为 C/C++ 开源库自动生成高质量的模糊测试驱动，并通过覆盖率反馈迭代持续优化驱动质量。除该核心功能性需求外，系统还应具备可扩展性（支持不同目标库的接入）、可靠性（生成的驱动能够通过编译并有效执行）、可恢复性（支持从中断状态恢复执行）等重要的非功能性需求。本节将从涉众分析出发，明确系统的三类主要用户群体及其与系统之间的关系；在此基础上，对系统的功能性需求和非功能性需求进行分析和阐述；最后，根据涉众类别和需求列表，给出系统的用例图及具体的用例描述。

3.2.1 涉众分析

MAGIC4FDG 系统面向 C/C++ 开源项目的模糊测试场景，能够帮助用户自动生成覆盖率高、结构合理的模糊驱动程序。根据用户的技术背景和使用目的，本系统的涉众可分为三类：目标项目开发者、模糊测试使用者和模糊测试研究人员。表3-1展示了对这三类涉众的分析结果，包括涉众的特征以及他们对本系统的期望。

表 3-1 涉众分析结果

涉众名称	涉众特征	涉众期望
目标项目开发者	被选为测试对象的开源项目的维护者，通常具备较强的 C/C++ 工程能力，熟悉自身接口设计与使用语义，了解模糊测试平台的基本要求。该类涉众直接参与项目构建与接口演进，对系统产出的驱动可用性与行为一致性有要求。	希望系统能够在不破坏接口契约的前提下，自动生成结构完整、逻辑合理、覆盖真实调用路径的模糊测试驱动，用于集成测试、安全审计或接口验证，降低人工编写负担并提高测试质量。
模糊测试使用者	使用 LibFuzzer、AFL 或 OSS-Fuzz 等平台执行模糊测试的测试工程师或安全研究人员，关注模糊驱动的适配性与执行效果。该类涉众具备模糊测试实践经验，熟悉驱动编写规范与覆盖率评估方法，但不一定深入了解目标库的内部实现。	希望系统生成的驱动能够直接用于主流模糊测试引擎，具备较高的代码覆盖率和缺陷发现能力；同时期望系统提供覆盖率报告和迭代过程记录，便于评估测试效果和定位潜在漏洞。
模糊测试研究人员	从事模糊测试自动化、驱动生成或大语言模型辅助测试等方向研究的学者或研究工程师。该类涉众关注系统的技术方法与创新点，通常将本系统与其他工具进行对比实验以验证方法的有效性。	希望系统具备良好的可复现性和可配置性，能够在不同目标库上稳定运行；同时期望系统提供详细的实验数据（覆盖率变化曲线、迭代轮次统计、消融实验支持等），便于进行公平的横向对比和方法分析。

对本系统而言，上述三类涉众的差异主要体现在技术背景深度和使用关注点两个方面。目标项目开发者是被测库的拥有者，他们熟悉库的接口设计、参数约束和调用规范，关注生成驱动是否遵循正确的 API 使用模式，是否能够发现影响库稳定性的潜在缺陷；模糊测试使用者侧重于驱动的实际执行效果，关注覆盖率指标、崩溃发现数量以及驱动与现有测试基础设施的兼容性；模糊测试研究人员则更关注系统的方法论层面，需要系统提供足够的实验支撑数据和灵活的配置选项，以便进行严格的对比实验和消融分析。三类涉众的共同期望是系

统能够端到端自动化运行，减少人工干预，同时保证生成驱动的质量和安全性。

3.2.2 功能性需求分析

通过对现有模糊驱动生成工具的调研与分析，结合 MAGIC4FDG 系统基于多智能体协作与覆盖率反馈迭代的技术路线，本系统应包含目标库配置与知识提取、策略规划与驱动生成、编译修复、模糊测试与覆盖率收集、缺陷诊断与迭代优化、状态持久化与恢复、实验报告生成七个方面的基本功能性需求。这些功能性需求的具体内容见表3-2。

表 3-2 功能性需求列表

需求编号	需求名称	需求描述
R1	目标库配置与知识提取	用户可以通过配置文件指定目标 C/C++ 库的构建方式、头文件路径和静态库路径，系统自动在 Docker 容器内完成构建并提取 API 知识
R2	策略规划与驱动生成	系统应能够根据目标库的 API 特征自动匹配合适的模糊策略，并基于策略指导生成多个风格互补的模糊驱动代码
R3	编译修复	系统应能够对生成的驱动代码执行编译验证，对编译失败的变体自动诊断错误原因并尝试修复
R4	模糊测试执行	系统应能够对编译成功的驱动在隔离环境中执行模糊测试，支持崩溃隔离和语料库累积
R5	覆盖率收集与分析	系统应能够收集行级与分支级覆盖率数据，执行可达性分析过滤不可达代码，并维护每个驱动的最佳版本记录
R6	缺陷诊断与迭代优化	系统应能够对覆盖率缺口进行根因分析，输出结构化约束描述，并基于分析结果在后续轮次中定向改进驱动代码
R7	状态持久化与恢复	系统应能够在每轮迭代后保存完整的流水线状态，支持从任意检查点恢复执行
R8	实验报告生成	系统应能够在迭代结束后生成包含覆盖率统计、迭代历史和最佳驱动代码的实验报告

目标库配置与知识提取（R1）是 MAGIC4FDG 系统运行的基础前提。用户

通过 JSON 格式的配置文件声明目标库的基本信息，包括库名、构建命令、头文件路径、静态库路径和覆盖率源文件路径等。系统根据配置在 Docker 容器内执行目标库的编译构建，随后通过 clang 抽象语法树导出提取结构化的 API 知识，包括函数签名、参数类型、调用图关系和宏常量定义。提取结果作为全局知识库供后续所有智能体使用。

策略规划与驱动生成（R2）是系统的核心功能之一。策略规划环节负责分析目标库 API 的语义特征，从策略库中为每个模糊驱动匹配最合适的生成策略；代码生成环节基于匹配的策略提示词后缀、API 知识上下文和基础模板，调用大语言模型合成完整的模糊驱动程序。首轮为从零生成，后续轮次为基于覆盖率反馈的增量改进。

编译修复（R3）确保生成的驱动代码能够通过编译。系统对每个驱动变体执行带内存错误检测插桩的编译，对编译失败的变体提取错误信息并调用大语言模型进行自动修复，最多重试三次，保证尽可能多的变体进入后续模糊测试环节。

模糊测试执行（R4）和覆盖率收集与分析（R5）共同构成系统的测试与反馈环节。模糊测试在 Docker 容器内以 fork 模式运行，确保单个崩溃不影响整体测试过程；覆盖率收集通过语料库重放和 LLVM 工具链导出精确的行级与分支级数据，同时执行控制流图可达性分析过滤结构上不可达的代码行，为后续诊断提供准确的覆盖率基线。

缺陷诊断与迭代优化（R6）是实现覆盖率持续提升的关键。缺陷诊断智能体对每个驱动的未覆盖代码进行根因分析，输出结构化的约束列表和未覆盖代码簇，这些信息被累积到驱动的隔离知识空间中，在后续轮次的代码生成环节作为增量改进的精确指导。

状态持久化与恢复（R7）保障系统的容错能力。每轮迭代完成后，系统将完整的流水线状态序列化为检查点文件，包括所有驱动的最佳代码、覆盖率数据、累积知识和迭代计数。当系统因异常中断时，用户可指定检查点文件恢复执行，避免从头开始的资源浪费。

实验报告生成（R8）满足用户对测试结果的查阅需求。系统在迭代终止后自动生成包含项目级覆盖率统计、各驱动覆盖率对比、迭代轮次历史和最佳驱动源码的结构化报告，便于用户评估测试效果和进行横向对比。

3.2.3 非功能性需求分析

除上述功能性需求外，非功能性需求也是系统设计过程中需要重点考虑的部分。非功能性需求由质量属性、性能需求和约束三部分组成。表3-3列出了MAGIC4FDG 系统的重要非功能性需求。

表 3-3 非功能性需求列表

需求编号	需求类型	需求名称	需求描述
QA1	质量属性	可扩展性	系统应支持通过配置文件接入新的目标库，无需修改系统源码
QA2	质量属性	可靠性	系统生成的驱动在经过编译修复后应达到 60% 以上的编译通过率
QA3	质量属性	可恢复性	系统应支持从任意轮次的检查点恢复执行，恢复后状态与中断前一致
QA4	质量属性	隔离性	所有编译、模糊测试和覆盖率收集操作应在 Docker 容器内执行，不影响宿主机环境
QA5	质量属性	可复现性	在相同配置和相同大语言模型下，系统的覆盖率结果应具备统计意义上的可复现性
PR1	性能需求	并行处理能力	系统应支持多个驱动变体的并行生成、并行编译和并行模糊测试
PR2	性能需求	构建缓存	目标库首次构建后应缓存产物，后续轮次的编译和模糊测试应复用缓存，避免重复构建
CS1	约束	开发语言	系统使用 Python 语言进行开发
CS2	约束	容器化依赖	系统运行依赖 Docker 引擎和 LLVM 工具链

质量属性方面，MAGIC4FDG 系统需满足可扩展性、可靠性、可恢复性、隔离性和可复现性五个关键要求。可扩展性（QA1）要求系统通过统一的配置文件格式接入不同的目标库，用户只需提供构建命令和路径信息即可启动完整的生成流程，无需修改系统内部代码。可靠性（QA2）要求系统的编译修复机制能够有效提升驱动的编译通过率，确保每轮迭代有足够数量的变体进入模糊测试环节。可恢复性（QA3）通过检查点机制实现，保障长时间运行的迭代任务在异常中断后能够无损恢复。隔离性（QA4）通过 Docker 容器化实现，确保目标库的构建和模糊测试过程不会污染宿主机环境，同时支持多个目标库的并行测试互

不干扰。可复现性（QA5）是实验评估的基本要求，系统应通过固定随机种子和确定性调度等机制保障结果的可重复性。

性能需求方面，系统应具备并行处理能力（PR1），支持多个驱动变体在代码生成、编译修复和模糊测试三个环节的并行执行，充分利用计算资源缩短单轮迭代时间。构建缓存机制（PR2）要求目标库的编译产物在首次构建后持久化存储，后续所有轮次通过只读挂载复用缓存，避免每轮重复构建带来的时间开销。

约束方面，系统使用 Python 语言开发（CS1），利用其丰富的生态支持大语言模型调用和流水线编排；系统运行依赖 Docker 引擎提供容器化执行环境，依赖 LLVM 工具链提供覆盖率插桩和数据导出能力（CS2）。

3.2.4 系统用例图

基于 3.2.1 小节的涉众分析以及 3.2.2 小节的功能性需求分析，本小节对相关描述内容进行实体识别、用例拆分与用例筛选，最终构建了 MAGIC4FDG 系统的用例图，如图3-2所示。

MAGIC4FDG 系统包括配置目标库、提取 API 知识、规划模糊策略、生成模糊驱动、修复编译错误、执行模糊测试、收集覆盖率数据、诊断覆盖率缺口、恢复中断任务和查看实验报告共 10 个用例。系统涉及三类人类参与者和两类外部系统参与者：目标项目开发者、模糊测试使用者和模糊测试研究人员为人类参与者，Docker 引擎和大语言模型服务为外部系统参与者。

三类人类参与者均可使用配置目标库、恢复中断任务和查看实验报告三个用例。提取 API 知识、规划模糊策略、生成模糊驱动、修复编译错误、执行模糊测试、收集覆盖率数据和诊断覆盖率缺口七个用例为系统自动执行的内部流程，由系统在用户启动任务后自动编排调度。Docker 引擎参与提取 API 知识、修复编译错误、执行模糊测试和收集覆盖率数据四个用例，为这些用例提供容器化执行环境；大语言模型服务参与规划模糊策略、生成模糊驱动、修复编译错误和诊断覆盖率缺口四个用例，为这些用例提供自然语言推理与代码生成能力。

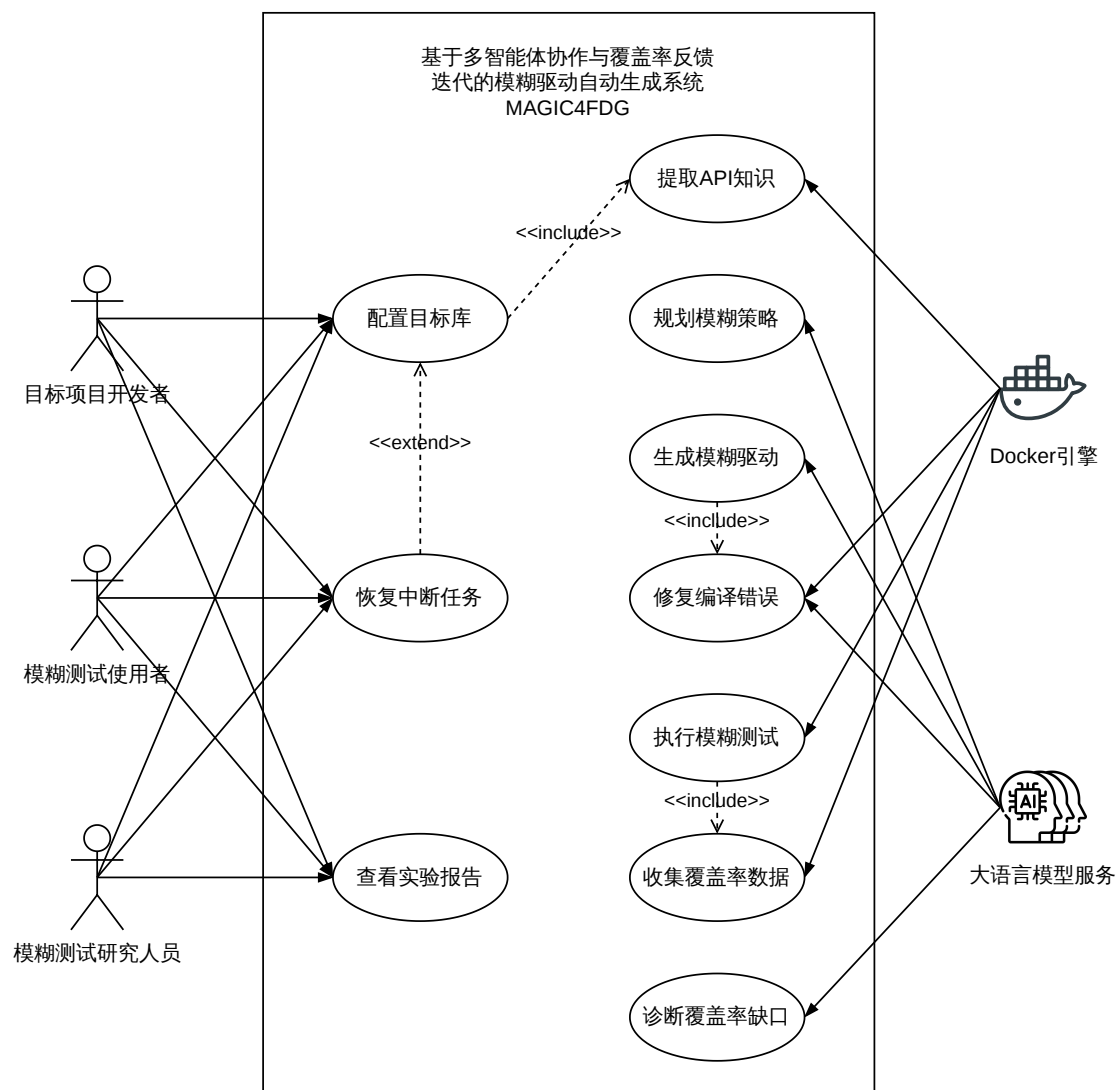


图 3-2 MAGIC4FDG 系统用例图

3.2.5 系统用例描述

上一小节中的系统用例图从用户视角展示了 MAGIC4FDG 系统的行为与用户需求之间的关系。本小节将详细描述每个系统用例的内容细节，帮助开发人员理解系统功能，指导后续的系统设计与开发工作。

配置目标库是用户使用 MAGIC4FDG 系统的入口操作。用户通过编写 JSON 格式的配置文件声明目标库的构建方式和路径信息，并通过命令行参数控制迭代轮次、目标覆盖率等运行时行为，系统据此启动自动化生成流程。表3-4展示了该用例的详细描述。

表 3-4 用例”配置目标库”详细描述

用例编号	UC1
用例名称	配置目标库
参与者	目标项目开发者、模糊测试使用者、模糊测试研究人员
前置条件	目标库源码可通过网络获取，用户了解目标库的构建方式
后置条件	系统获得有效的目标库配置，可启动自动化生成流程
基本流程	1. 用户编写目标库配置文件，指定库名、构建命令、头文件路径、静态库路径和覆盖率源文件路径 2. 用户通过命令行指定配置文件路径和运行参数（最大轮次、目标覆盖率、模糊测试时长等） 3. 系统验证配置文件格式和必填字段的完整性 4. 系统启动流水线执行
扩展流程	3a. 配置文件格式错误或缺少必填字段时，系统输出错误提示并终止

提取 API 知识是系统获取目标库结构化信息的基础操作。系统利用 clang 编译器前端的抽象语法树导出能力，在 Docker 容器内精确提取函数签名、参数类型、调用图关系和宏常量定义，构建产物被缓存至宿主机以避免后续轮次重复构建。表3-5展示了该用例的详细描述。

表 3-5 用例”提取 API 知识”详细描述

用例编号	UC2
用例名称	提取 API 知识
参与者	Docker 引擎
前置条件	系统获得有效的目标库配置
后置条件	系统获得目标库的结构化 API 知识（函数签名、调用图、类型信息）
基本流程	1. 系统在 Docker 容器内执行目标库的构建命令 2. 系统通过 clang AST 导出提取头文件和源文件中的 API 信息 3. 系统解析 AST JSON，提取函数签名、参数类型、调用图和宏定义 4. 系统将构建产物缓存至宿主机 5. 系统将提取结果存入全局知识库
扩展流程	1a. 构建失败时，系统输出构建日志并终止流程

规划模糊策略是系统为每个模糊驱动选择最合适生成策略的决策环节。在探索阶段，策略规划智能体通过大语言模型将策略库中十种预定义策略与目标

库 API 特征进行匹配；在利用阶段，根据覆盖率停滞情况动态调整策略分配。
表3-6展示了该用例的详细描述。

表 3-6 用例”规划模糊策略”详细描述

用例编号	UC3
用例名称	规划模糊策略
参与者	大语言模型服务
前置条件	系统已完成 API 知识提取
后置条件	每个模糊驱动获得匹配的生成策略和目标 API 列表
基本流程	1. 系统加载策略库中十种模糊策略的元数据 2. 系统将 API 知识注入规划提示模板 3. 系统调用大语言模型分析 API 语义关系并匹配策略 4. 系统为每个模糊驱动分配目标 API 和对应策略
扩展流程	3a. 大语言模型调用失败时，系统回退至基于调用图的规则引擎进行确定性匹配

生成模糊驱动是系统的核心代码生成环节。每个驱动接收差异化的策略提示词后缀，引导大语言模型采用不同的代码结构和 API 调用模式；首轮为从零生成模式，后续轮次切换为基于约束列表和累积经验的增量改进模式。表3-7展示了该用例的详细描述。

表 3-7 用例”生成模糊驱动”详细描述

用例编号	UC4
用例名称	生成模糊驱动
参与者	大语言模型服务
前置条件	策略规划完成，每个驱动已分配目标 API 和策略
后置条件	系统获得多个模糊驱动代码变体
基本流程	1. 系统为每个驱动组装提示（基础模板 +API 知识 + 策略后缀） 2. 系统并行调用大语言模型生成驱动代码 3. 系统从模型输出中提取代码块并保存为变体文件
扩展流程	2a. 后续轮次，提示额外包含最佳代码、约束列表和累积经验

修复编译错误确保生成的驱动代码能够通过编译。编译修复智能体在 Docker 容器内执行带 AddressSanitizer 插桩的编译，对失败的变体提取错误信息并调用

大语言模型生成修复代码，最多重试三次。表3-8展示了该用例的详细描述。

表 3-8 用例”修复编译错误”详细描述

用例编号	UC5
用例名称	修复编译错误
参与者	Docker 引擎、大语言模型服务
前置条件	代码生成智能体已输出驱动变体
后置条件	变体被标记为编译成功或编译失败（已耗尽重试次数）
基本流程	1. 系统在 Docker 容器内对变体执行带插桩的编译 2. 编译成功，标记变体为通过
扩展流程	1a. 编译失败时，系统提取错误信息并调用大语言模型生成修复代码 1b. 重新编译修复后的代码，最多重试三次 1c. 三次重试均失败，标记变体为编译失败

执行模糊测试是系统对目标库进行自动化安全测试的核心环节。模糊测试采用 fork 模式在独立子进程中执行每个输入，确保单个崩溃不会终止整个测试过程，同时将每轮发现的有效输入持久化存储供后续轮次复用。表3-9展示了该用例的详细描述。

表 3-9 用例”执行模糊测试”详细描述

用例编号	UC6
用例名称	执行模糊测试
参与者	Docker 引擎
前置条件	存在至少一个编译成功的驱动变体
后置条件	系统获得模糊测试产生的语料库和崩溃样本
基本流程	1. 系统以覆盖率插桩标志重新编译驱动变体 2. 系统合并种子语料库与历史累积语料库 3. 系统以 fork 模式运行 LibFuzzer 执行模糊测试 4. 系统收集测试产生的新语料库和崩溃样本
扩展流程	3a. 所有变体均编译失败时，跳过本用例直接进入状态持久化

收集覆盖率数据是系统获取精确反馈信息的关键环节。由于 fork 模式下覆盖率数据文件可能未完整写入，系统通过语料库重放机制在非 fork 模式下重新执行所有输入以确保数据完整性，并维护每个驱动的最佳版本记录和停滞计数。

表3-10展示了该用例的详细描述。

表 3-10 用例” 收集覆盖率数据” 详细描述

用例编号	UC7
用例名称	收集覆盖率数据
参与者	Docker 引擎
前置条件	模糊测试执行完成，语料库已生成
后置条件	系统获得行级与分支级覆盖率报告，最佳版本记录已更新
基本流程	1. 系统通过语料库重放机制收集完整的覆盖率原始数据 2. 系统通过 LLVM 工具链合并数据并导出覆盖率报告 3. 系统执行控制流图可达性分析，标记不可达代码行 4. 系统比较当前覆盖率与历史最佳，更新最佳版本记录 5. 系统计算项目级覆盖率并更新停滞计数
扩展流程	4a. 当前覆盖率未超过历史最佳时，递增该驱动的停滞计数

诊断覆盖率缺口是系统实现精准迭代改进的智能分析环节。缺陷诊断智能体仅输出结构化的约束描述和未覆盖代码簇分析，将” 诊断” 与” 生成” 职责分离；诊断结果被持久化到每个驱动的隔离知识空间中，跨轮次累积形成经验库。表3-11展示了该用例的详细描述。

表 3-11 用例” 诊断覆盖率缺口” 详细描述

用例编号	UC8
用例名称	诊断覆盖率缺口
参与者	大语言模型服务
前置条件	覆盖率数据收集完成
后置条件	每个活跃驱动获得结构化的约束列表和未覆盖代码簇分析
基本流程	1. 系统读取每个驱动的最佳代码和未覆盖代码行列表 2. 系统将覆盖率数据与 API 知识注入分析提示模板 3. 系统调用大语言模型进行根因分析 4. 系统将输出的约束和知识追加到驱动的隔离知识空间
扩展流程	3a. 大语言模型输出格式异常时，系统跳过该驱动的本轮分析

恢复中断任务使用户能够之前保存的检查点继续执行，避免因异常中断导致的资源浪费。MAGIC4FDG 系统的单次完整运行可能持续数小时（取决于目

标库复杂度、迭代轮次和模糊测试时长)，期间可能因网络波动、大语言模型服务不可用或系统资源不足等原因中断。检查点恢复机制确保用户无需从头开始，可从最近一次成功完成的轮次继续迭代。表3-12展示该用例的详细描述。

表 3-12 用例”恢复中断任务”详细描述

用例编号	UC9
用例名称	恢复中断任务
参与者	目标项目开发者、模糊测试使用者、模糊测试研究人员
前置条件	系统存在之前保存的检查点文件
后置条件	系统从指定检查点恢复完整的流水线状态并继续迭代
基本流程	1. 用户通过命令行参数指定检查点文件路径 2. 系统反序列化检查点文件，恢复流水线状态 3. 系统从恢复的轮次继续执行迭代流程
扩展流程	2a. 检查点文件损坏或格式不兼容时，系统输出报错并终止

查看实验报告使用户能够了解系统的运行结果和测试效果。实验报告是三类涉众评估系统产出质量的主要途径：目标项目开发者通过报告了解驱动覆盖了哪些 API 路径和发现了哪些潜在缺陷；模糊测试使用者通过报告评估驱动的覆盖率水平和崩溃发现能力；模糊测试研究人员通过报告中的定量数据进行横向对比实验。报告同时以人类可读的 Markdown 格式和机器可解析的 JSON 格式输出，兼顾可读性和自动化处理需求。表3-13展示了该用例的详细描述。

表 3-13 用例”查看实验报告”详细描述

用例编号	UC10
用例名称	查看实验报告
参与者	目标项目开发者、模糊测试使用者、模糊测试研究人员
前置条件	系统迭代终止（满足任一终止条件）
后置条件	用户获得包含覆盖率统计和最佳驱动代码的结构化报告
基本流程	1. 系统在迭代终止后自动生成 Markdown 和 JSON 格式的实验报告 2. 报告包含项目级覆盖率、各驱动覆盖率对比、迭代历史和最佳驱动源码 3. 用户查阅报告评估测试效果
扩展流程	1a. 用户可通过配置选择报告中包含的内容项

3.3 系统概要设计

本节采用“4+1”视图方法对 MAGIC4FDG 系统的架构进行描述。“4+1”视图由逻辑视图、开发视图、进程视图、物理视图和场景视图五个部分组成，其中场景视图即 3.2 节中的用例视图，已在需求分析阶段完成。本节首先介绍系统的整体架构设计，随后从模块划分、逻辑视图、开发视图、进程视图和物理视图四个维度对系统架构进行详细阐述。

3.3.1 系统整体架构设计

图3-3展示了 MAGIC4FDG 系统的整体架构图。本系统基于用户提供的目标库配置，在 Docker 容器内自动完成知识提取、驱动生成、编译修复、模糊测试与覆盖率收集、缺陷诊断等环节，通过迭代反馈持续优化模糊驱动的代码覆盖率。图3-3按照模块对系统功能进行了划分，并使用箭头标明了不同模块之间的依赖关系。

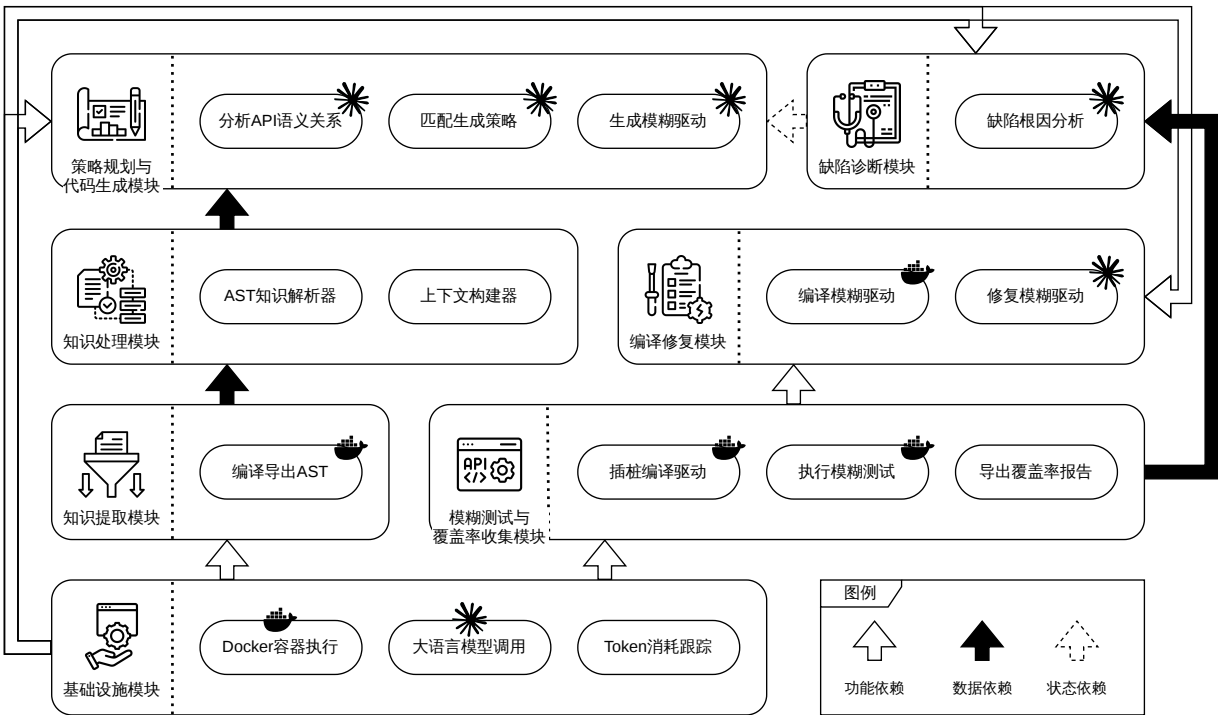


图 3-3 MAGIC4FDG 系统整体架构图

本系统架构采用自底向上的分层式设计，整体由七个功能模块组成，包括基础设施模块、知识处理模块、知识提取模块、策略规划与代码生成模块、编译修复模块、模糊测试与覆盖率收集模块以及缺陷诊断模块。各模块相互协作，共同

完成用户的需求，保障系统的高效运行。

基础设施模块负责为整个系统提供底层服务支撑，包括 **Docker** 容器执行、大语言模型调用和令牌消耗追踪三项核心能力；知识处理模块负责解析 **clang** 抽象语法树输出，提取目标库的结构化 **API** 知识，并根据不同智能体的需求将知识格式化为适合注入提示词的文本片段；知识提取模块负责在 **Docker** 容器内执行目标库的编译构建和 **AST** 导出，获取系统运行所需的全局知识库；策略规划与代码生成模块是系统的核心生成模块，负责分析 **API** 语义关系、匹配模糊策略并基于差异化提示词并行生成多个风格互补的模糊驱动；编译修复模块负责对生成的驱动变体执行编译验证，对编译失败的变体通过大语言模型进行自动修复；模糊测试与覆盖率收集模块负责在 **Docker** 容器内执行模糊测试并通过 **LLVM** 工具链收集精确的行级与分支级覆盖率数据；缺陷诊断模块负责对覆盖率缺口进行根因分析，输出结构化的约束描述供后续轮次的增量改进使用。各个模块的划分原则以及详细设计将在下一小节中详细讨论。

3.3.2 系统模块划分

在需求分析的基础上，对系统进行模块化划分可以有效降低其复杂度，使系统结构更加清晰合理，提升后续的开发效率和软件质量。**MAGIC4FDG** 系统按照功能职责划分为基础设施、知识处理、知识提取、策略规划与代码生成、编译修复、模糊测试与覆盖率收集、缺陷诊断七个功能模块，下面将对这些模块进行逐一介绍，最后还将对模块之间的依赖关系进行解释。

基础设施模块。该模块并不直接对应于需求分析中的某一个功能性需求或系统用例，但它为其他所有模块的实现提供了重要的技术支持。该模块包含三个核心组件：**Docker** 执行器封装所有容器化操作的命令构建与执行，包括知识提取、编译、模糊测试和覆盖率收集四类 **Docker** 命令，同时管理构建缓存的创建与挂载，对应非功能性需求 **QA4**（隔离性）和 **PR2**（构建缓存）；大语言模型工厂负责创建带重试机制的模型客户端实例，统一管理模型配置、超时和错误处理，通过提示词模板和输出检测机制保障调用的稳定性；令牌追踪器记录各智能体的令牌消耗量，用于成本监控和实验分析。

知识处理模块。该模块对应于功能性需求 **R1**（目标库配置与知识提取）中的知识组织部分，为智能体模块提供结构化的知识服务。该模块包含三个核心

组件：提取器解析 clang AST JSON 输出，从中提取函数签名、参数类型、调用图关系和宏常量定义；上下文构建器根据不同智能体的需求，将原始知识格式化为适合注入提示词的文本片段，例如为代码生成智能体提供目标 API 的完整签名和调用关系，为缺陷诊断智能体提供未覆盖代码行的上下文信息；分组引擎实现基于调用图的 API 分组和规则引擎策略匹配，作为大语言模型策略规划失败时的确定性回退方案。

知识提取模块。该模块对应于功能性需求 R1 以及系统用例 UC2（提取 API 知识）。知识提取模块在 Docker 容器内执行目标库的编译构建，随后通过 clang 编译器前端对目标库的头文件和源文件执行抽象语法树导出，将导出结果交由知识处理模块进行解析和组织。构建产物被缓存至宿主机持久化存储，后续轮次的编译和模糊测试容器通过只读挂载复用缓存，避免每轮重复构建带来的时间开销。该模块仅在系统首次运行时执行一次，提取结果作为全局知识库在后续所有迭代轮次中复用。

策略规划与代码生成模块。该模块对应于功能性需求 R2（策略规划与驱动生成）以及系统用例 UC3（规划模糊策略）和 UC4（生成模糊驱动），是系统的核心生成模块。在首轮迭代（探索阶段），该模块加载策略库中十种预定义模糊策略的元数据，调用大语言模型分析目标库 API 之间的语义关系与功能分组，为每个模糊驱动匹配最合适的生成策略；随后基于差异化的提示词组装（基础模板、API 知识上下文、策略提示词后缀）并行调用大语言模型生成多个风格互补的驱动代码。在后续轮次（利用阶段），该模块根据覆盖率停滞情况动态调整策略分配，并将缺陷诊断模块输出的约束列表和累积经验注入提示词，指导模型进行增量改进。

编译修复模块。该模块对应于功能性需求 R3（编译修复）以及系统用例 UC5（修复编译错误）。对每个生成的驱动变体，该模块在 Docker 容器内执行带 AddressSanitizer 插桩的编译，将驱动与目标库进行静态链接。若编译成功则标记为通过；若编译失败，则提取编译器输出的错误信息，将驱动代码与错误信息注入修复提示模板，调用大语言模型生成修复后的代码并重新编译，最多重试三次。该模块的存在使系统对大语言模型的代码生成质量具有容错能力，确保每轮迭代有足够数量的变体进入后续模糊测试环节。

模糊测试与覆盖率收集模块。该模块对应于功能性需求 R4（模糊测试执行）

和 R5（覆盖率收集与分析）以及系统用例 UC6（执行模糊测试）和 UC7（收集覆盖率数据）。对每个编译成功的变体，该模块在 Docker 容器内依次执行以下操作：以覆盖率插桩标志重新编译驱动、合并种子语料库与历史累积语料库、以 LibFuzzer 的 fork 模式运行模糊测试、通过语料库重放机制收集完整的覆盖率数据、通过 LLVM 工具链导出行级与分支级覆盖率报告。此外，该模块还执行控制流图可达性分析以过滤不可达代码行，并维护每个驱动的最佳版本记录和停滞计数。

缺陷诊断模块。该模块对应于功能性需求 R6（缺陷诊断与迭代优化）以及系统用例 UC8（诊断覆盖率缺口）。对每个活跃的模糊驱动，该模块读取其当前最佳代码和未覆盖代码行列表，将覆盖率数据与 API 知识注入分析提示模板，调用大语言模型进行根因分析。模型输出结构化的分析结果，包含约束列表（触发特定分支所需的前置条件）和未覆盖代码簇（按功能分组的未覆盖区域）。这些约束被追加到对应驱动的隔离知识空间中，跨轮次持续累积形成经验库，为后续轮次的策略规划与代码生成模块提供精确的改进指导。

模块之间的依赖关系。图3-3展示的系统架构是基于需求分析得到的功能性需求和系统用例进行模块化划分的，不同模块之间存在依赖关系，包括数据依赖、功能依赖和状态依赖三类。整体来看，MAGIC4FDG 系统采用的是分层架构，即模块之间的依赖关系具备层级性和单向性，上层依赖于下层，而下层不会依赖于上层。这样的分层设计符合依赖倒置原则，能够有效降低系统不同组件的耦合度，提升系统的可维护性和可扩展性。

知识提取模块对基础设施模块和知识处理模块的依赖关系分别属于功能依赖和数据依赖：知识提取模块调用基础设施模块的 Docker 执行器在容器内完成目标库构建和 AST 导出，随后调用知识处理模块的提取器解析 AST JSON 并构建结构化知识库。策略规划与代码生成模块、缺陷诊断模块对知识处理模块的依赖关系属于数据依赖，前者的运行需要使用知识处理模块提供的结构化 API 知识和格式化的上下文文本。系统中所有涉及大语言模型调用的模块，如策略规划和代码生成缺陷诊断，对基础设施模块的依赖关系属于功能依赖，基础设施模块负责提供大语言模型调用服务和 Docker 容器执行服务。编译修复模块和模糊测试与覆盖率收集模块对基础设施模块的依赖关系同样属于功能依赖，两者均通过 Docker 执行器在容器内完成编译验证、模糊测试执行和覆盖率数据收集。

缺陷诊断模块与策略规划与代码生成模块之间的依赖关系属于状态依赖：缺陷诊断模块的输出（约束列表和累积知识）通过共享的流水线状态对象传递给策略规划与代码生成模块，后者在后续轮次中基于这些状态信息调整生成策略和提示词内容。

3.3.3 系统逻辑视图

逻辑视图关注系统的功能性需求，描述系统的主要组件、类、对象以及它们之间的关系，通常使用类图表示，强调模块职责和数据流，用于支持后续代码设计与实现。图3-4为 MAGIC4FDG 系统的逻辑视图。

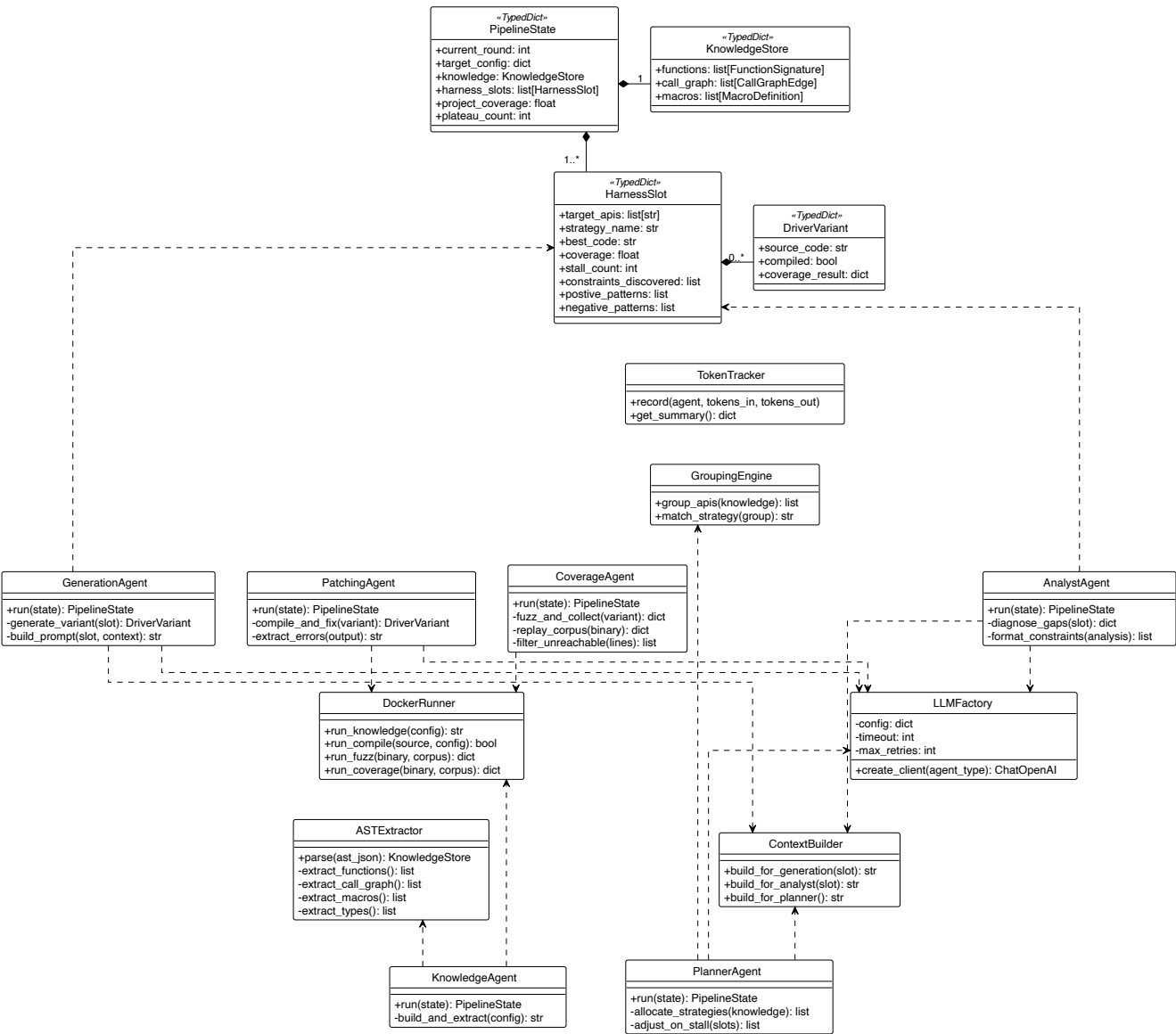


图 3-4 MAGIC4FDG 系统逻辑视图

上述逻辑视图采用概念类图的形式呈现了本系统的静态概念架构，展示了核心类及其之间的依赖关系。在介绍各模块的核心类之前，首先说明系统的全局状态定义。MAGIC4FDG 系统采用共享状态架构，所有模块通过统一的状态对象进行数据传递，该状态由四个关键类型组成：**PipelineState** 是系统的全局状态容器，采用 **TypedDict** 结构定义，包含当前轮次、目标库配置、知识库、模糊驱动槽位列表和项目级覆盖率等字段，作为所有模块间数据传递的唯一契约；**HarnessSlot** 表示一个独立的模糊驱动槽位，包含目标 API 列表、匹配策略名称、最佳代码、覆盖率数据、停滞计数和累积知识等属性，是系统实现多驱动并行迭代的核心数据结构；**DriverVariant** 表示单轮生成的一个驱动变体，包含源码、编译状态和覆盖率结果；**KnowledgeStore** 存储从 clang AST 中提取的全局知识，包括函数签名列表、调用图和宏定义映射。

接下来，按照 3.3.2 中划分的功能模块，分别介绍各模块所包含的核心类及其设计。

基础设施模块包含 **DockerRunner**、**LLMFactory** 和 **TokenTracker** 三个核心类。**DockerRunner** 封装了四类 Docker 操作（知识提取、编译、模糊测试、覆盖率收集），每类操作对应一个独立的方法，内部负责构建 Docker 命令、管理卷挂载和环境变量、解析容器输出。**LLMFactory** 采用工厂模式，根据配置文件创建带重试机制和超时控制的 **ChatOpenAI** 客户端实例，统一管理所有智能体的模型调用。**TokenTracker** 通过回调机制记录每次模型调用的输入输出令牌数，按智能体类型分类统计。

知识处理模块包含 **ASTExtractor**、**ContextBuilder** 和 **GroupingEngine** 三个核心类。**ASTExtractor** 负责解析 clang 导出的 AST JSON 文件，提取 **FunctionSignature**、**CallGraphEdge** 和 **MacroDefinition** 等结构化对象。**ContextBuilder** 根据调用方智能体的类型，将原始知识格式化为不同粒度的文本片段：为代码生成智能体提供完整的 API 签名和参数语义，为缺陷诊断智能体提供函数级覆盖率与未覆盖代码行的上下文。**GroupingEngine** 基于调用图的连通分量分析实现 API 分组，并通过规则引擎将策略元数据中的适用条件与 API 分组特征进行匹配。

知识提取模块的核心类为 **KnowledgeAgent**，该类封装了 Docker 容器内目标库构建和 AST 导出的完整流程，调用基础设施模块的 **DockerRunner** 执行容器操作，并将提取结果写入全局状态的 **KnowledgeStore** 中。

策略规划与代码生成模块包含 PlannerAgent 和 GenerationAgent 两个核心类。PlannerAgent 负责加载策略库元数据、调用大语言模型进行 API 语义分析和策略匹配，输出各驱动槽位的策略分配方案。GenerationAgent 根据策略分配结果组装差异化提示词，通过线程池并行调用大语言模型生成多个驱动变体。

编译修复模块的核心类为 PatchingAgent，该类对每个驱动变体执行 Docker 容器内的编译验证，对编译失败的变体提取错误信息并调用大语言模型生成修复代码，支持最多三次重试。

模糊测试与覆盖率收集模块的核心类为 CoverageAgent，该类封装了模糊测试执行、语料库重放、覆盖率数据导出和控制流图可达性分析的完整流程，通过线程池并行处理多个变体的测试与覆盖率收集。

缺陷诊断模块的核心类为 AnalystAgent，该类读取各驱动的覆盖率数据和未覆盖代码行列表，调用大语言模型进行根因分析，输出结构化的约束列表和未覆盖代码簇，并将分析结果追加到对应驱动的隔离知识空间中。

3.3.4 系统开发视图

开发视图关注软件系统的模块化结构和代码组织方式，描述代码如何被分解为模块、组件和包，以及它们之间的依赖关系。图3-5展示了 MAGIC4FDG 系统的开发视图。

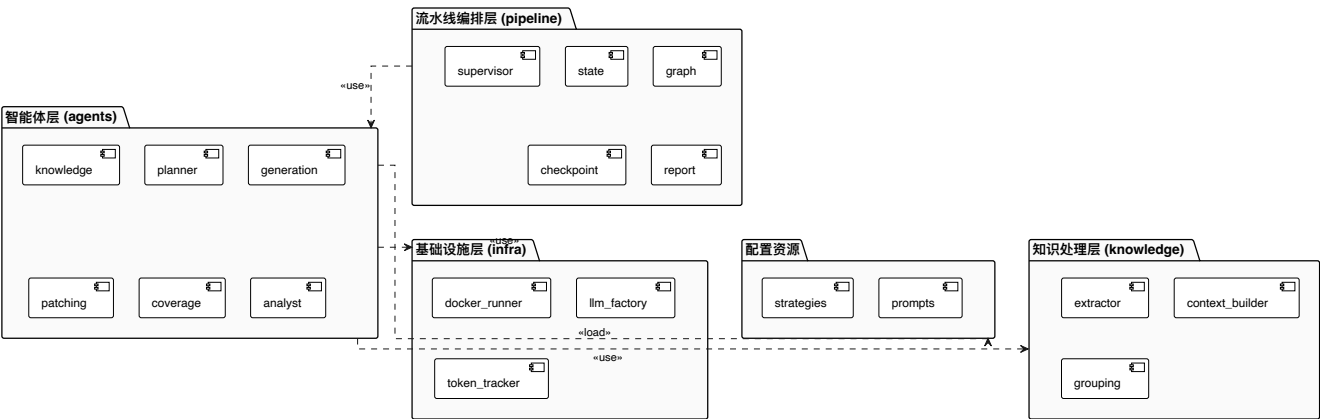


图 3-5 MAGIC4FDG 系统开发视图

上述开发视图采用包图的形式展示了本软件系统的代码组织结构，面向开发人员设计，旨在确保后续开发过程中具备清晰的架构和良好的管理。该开发视图遵循分层设计原则，自底向上划分为四个主要代码层次：基础设施层、知识

处理层、智能体层和流水线编排层。需要说明的是，代码层次划分是从软件工程的代码组织角度出发，与 3.3.2 中按业务功能划分的七个功能模块存在映射关系：基础设施层对应基础设施模块，知识处理层对应知识处理模块，智能体层包含知识提取、策略规划与代码生成、编译修复、模糊测试与覆盖率收集、缺陷诊断五个功能模块的实现，流水线编排层负责全局状态定义、DAG 图构建、条件路由和检查点管理等编排逻辑。

基础设施层 (infra) 包含 `docker_runner`、`llm_factory` 和 `token_tracker` 三个包。`docker_runner` 包封装所有 Docker 容器操作的命令构建与执行逻辑；`llm_factory` 包负责大语言模型客户端的创建和配置管理；`token_tracker` 包提供令牌消耗的统计与记录功能。

知识处理层 (knowledge) 包含 `extractor`、`context_builder` 和 `grouping` 三个包。`extractor` 包负责解析 clang AST JSON 输出并构建结构化知识对象；`context_builder` 包将原始知识按需格式化为提示词文本片段；`grouping` 包实现基于调用图的 API 分组和规则引擎策略匹配。

智能体层 (agents) 包含 `knowledge`、`planner`、`generation`、`patching`、`coverage` 和 `analyst` 六个包，每个包对应一个专职智能体，实现独立的节点函数。各智能体包内部封装了提示词模板加载、大语言模型调用和结果解析的完整逻辑。

流水线编排层 (pipeline) 包含 `state`、`graph`、`supervisor`、`checkpoint` 和 `report` 五个包。`state` 包定义全局状态的 TypedDict 结构；`graph` 包构建 LangGraph 状态图并注册节点和条件路由；`supervisor` 包提供命令行入口和参数解析；`checkpoint` 包实现状态的 JSON 序列化与反序列化；`report` 包生成 Markdown 和 JSON 格式的实验报告。

此外，系统还包含两个辅助目录：`strategies` 目录存储十种模糊策略的 Markdown 文件（含 YAML 前置元数据），`prompts` 目录存储各智能体使用的提示词模板文件。这两个目录作为系统的配置资源，被智能体层在运行时动态加载。

3.3.5 系统进程视图

进程视图关注系统运行时的行为，描述系统的并发性、进程间通信以及同步机制，帮助开发人员确定软件系统的运行流程。图3-6展示了 MAGIC4FDG 系统的进程视图。

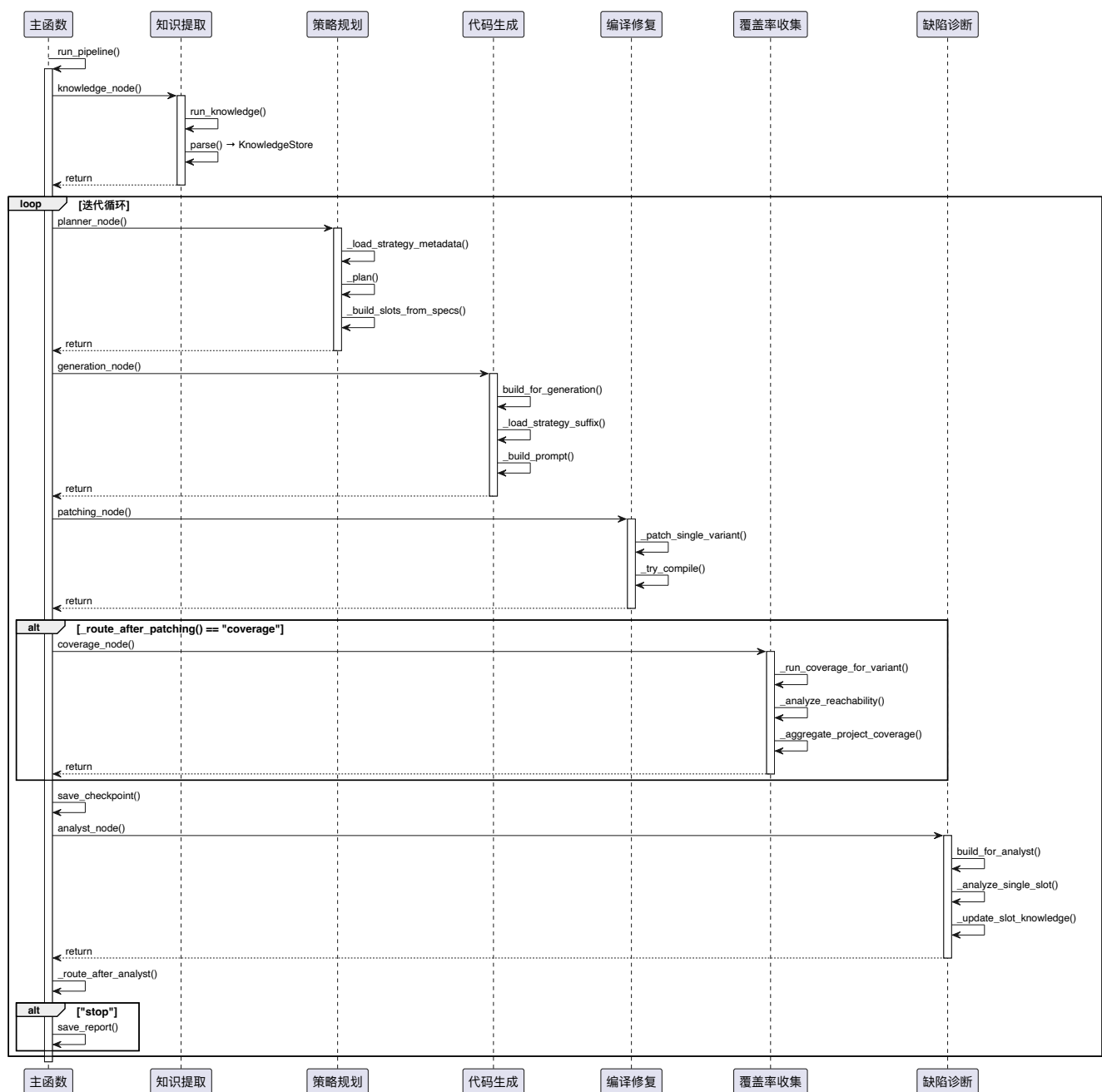


图 3-6 MAGIC4FDG 系统进程视图

上述进程视图采用顺序图的形式展示了本软件系统的运行流程及组件间的交互细节，重点强调事件触发和数据流转，以确保系统设计符合用户需求。该顺序图清晰地描述了主函数如何与各个组件交互，执行一系列任务，并最终完成模糊驱动的自动生成与迭代优化。除了主函数之外，系统的核心功能由多个关键组件协同完成，包括知识提取组件、知识处理组件、策略规划组件、代码生成组件、编译修复组件、覆盖率收集组件、状态持久化组件和缺陷诊断组件。

主函数首先调用 `run_pipeline()` 函数启动整个流水线。流水线启动后，主

函数调用知识提取组件的 `knowledge_node()` 函数进行目标库的知识提取。知识提取组件内部调用 `run_knowledge()` 函数在 Docker 容器内执行目标库的编译构建和 AST 导出，随后调用知识处理组件的 `parse()` 函数解析 AST JSON 文件，将提取的结构化知识存入全局状态的 `KnowledgeStore` 中，并将构建产物缓存至宿主机。该步骤仅在系统首次运行时执行，后续轮次通过检查缓存是否存在来决定是否跳过。

知识提取完成后，系统进入迭代循环。每轮迭代中，主函数首先调用策略规划组件的 `planner_node()` 函数。策略规划组件内部依次调用 `_load_strategy_metadata()` 函数加载策略库元数据，在首轮调用 `_plan_initial()` 函数通过大语言模型进行 API 语义分析和策略匹配，在后续轮次调用 `_plan_iterate()` 函数根据停滞计数调整策略分配，最后调用 `_build_slots_from_specs()` 函数构建模糊驱动槽位。

随后，主函数调用代码生成组件的 `generation_node()` 函数。代码生成组件首先调用知识处理组件的 `build_for_generation()` 函数获取格式化的 API 知识上下文，再调用 `_load_strategy_suffix()` 函数加载对应策略的提示词后缀，最后根据轮次调用 `_build_fresh_prompt()` 或 `_build_incremental_prompt()` 函数组装完整提示词并调用大语言模型生成驱动代码。

接着，主函数调用编译修复组件的 `patching_node()` 函数。编译修复组件通过线程池对每个变体并行调用 `_patch_single_variant()` 函数，该函数内部调用 `_try_compile()` 函数在 Docker 容器内执行编译验证，对编译失败的变体提取错误信息并调用大语言模型生成修复代码，最多重试三次。

编译修复完成后，主函数调用 `_route_after_patching()` 函数执行条件路由判断。若存在编译成功的变体，主函数调用覆盖率收集组件的 `coverage_node()` 函数。覆盖率收集组件通过线程池对每个变体并行调用 `_run_coverage_for_variant()` 函数执行模糊测试和覆盖率数据收集，随后调用 `_analyze_reachability()` 函数进行控制流图可达性分析，最后调用 `_aggregate_project_coverage()` 函数计算项目级覆盖率。若所有变体均编译失败，则跳过覆盖率收集直接进入下一步。

之后，主函数调用状态持久化组件的 `checkpoint_node()` 函数。该组件内部调用 `save_checkpoint()` 函数将当前完整的流水线状态序列化为 JSON 检查

点文件，同时保存各变体的源码文件。

最后，主函数调用缺陷诊断组件的 `analyst_node()` 函数。缺陷诊断组件首先调用知识处理组件的 `build_for_analyst()` 函数获取未覆盖代码行的上下文信息，随后通过线程池对每个活跃驱动并行调用 `_analyze_single_slot()` 函数进行覆盖率缺口的根因分析，最后调用 `_update_slot_knowledge()` 函数将分析结果追加到对应驱动的隔离知识空间中。

每轮迭代结束时，主函数调用 `_route_after_analyst()` 函数执行终止条件判断，检查覆盖率达标、轮次上限、全部收敛或项目级停滞四个条件。满足任一条件时，主函数调用 `save_report()` 函数生成实验报告并终止流程；否则进入下一轮迭代。

3.3.6 系统物理视图

物理视图关注系统的部署架构，描述软件如何映射到硬件资源，面向系统管理员或运维人员。图3-7展示了 **MAGIC4FDG** 系统的物理视图。

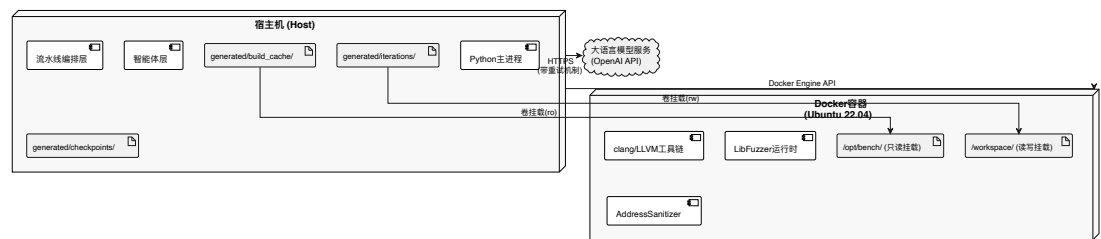


图 3-7 **MAGIC4FDG** 系统物理视图

上述物理视图采用 **UML** 部署图的形式展示了本软件系统的部署结构，强调系统的节点和通信方式。该物理视图展示了系统的三大物理核心组件及其交互方式，包括宿主机、**Docker** 容器和大语言模型服务。

在系统部署中，宿主机承载了 **MAGIC4FDG** 系统的流水线编排层和智能体层的控制逻辑，运行 **Python** 主进程。宿主机通过 **Docker Engine API** 与本地 **Docker** 容器进行交互，所有计算密集型操作（目标库构建、驱动编译、模糊测试执行、覆盖率数据收集）均在 **Docker** 容器内完成。**Docker** 容器基于 **Ubuntu 22.04** 镜像，预装 **clang/LLVM** 工具链和 **LibFuzzer** 运行时库。宿主机与容器之间通过卷挂载实现数据交换：构建缓存目录以只读方式挂载至容器的 `/opt/bench` 路径，驱动源码和语料库通过读写卷在宿主机与容器间双向传递。

另一方面，宿主机通过 **HTTPS** 协议与外部大语言模型服务建立连接，用于策略规划、代码生成、编译修复和缺陷诊断四个环节的模型调用。系统支持兼容 **OpenAI API** 格式的任意模型服务端点，通过配置文件指定 **API** 地址和密钥。大语言模型服务的调用采用带指数退避的重试机制，确保网络波动不会导致流水线中断。

3.4 本章小结

本章主要介绍了 **MAGIC4FDG** 系统的需求分析结果和系统概要设计，用于指导后续的系统详细设计和软件开发过程。

首先，本章给出了 **MAGIC4FDG** 系统的整体概述。通过系统流程图，阐述了系统的四大运行阶段（知识准备、驱动生成与编译修复、模糊测试与覆盖率收集、缺陷诊断与迭代控制）以及各阶段中智能体的职责分工和协作方式。

然后，本章对 **MAGIC4FDG** 系统进行了需求分析，包括涉众分析、功能性需求分析、非功能性需求分析、系统用例图及用例描述。涉众分析环节总结了系统的三类涉众，即目标项目开发者、模糊测试使用者和模糊测试研究人员；功能性需求分析环节总结并解释了系统的 8 个基本功能性需求（目标库配置与知识提取、策略规划与驱动生成、编译修复、模糊测试执行、覆盖率收集与分析、缺陷诊断与迭代优化、状态持久化与恢复、实验报告生成）以及 9 个非功能性需求；用例分析环节基于涉众类型和功能性需求构建了系统用例图，并对 10 个用例进行了详细描述。

最后，在需求分析的基础上，本章完成了对 **MAGIC4FDG** 系统的概要设计。具体来说，本章首先展示了系统的整体架构并进行了模块划分，将系统分为基础设施、知识处理、知识提取、策略规划与代码生成、编译修复、模糊测试与覆盖率收集、缺陷诊断七个功能模块，并分析了模块间的数据依赖、功能依赖和状态依赖关系；之后，使用“4+1”视图从各个维度描述了系统的架构设计。其中，逻辑视图定义了系统的主要功能类及其依赖关系；开发视图从代码组织的角度阐述了系统的包结构及分层管理方式；进程视图展示了系统在运行时的主要流程，包括各组件的调用顺序、并行执行点和条件路由；物理视图详细说明了系统的部署结构，明确了宿主机、**Docker** 容器和大语言模型服务之间的通信方式。

第四章 MAGIC4FDG 系统的详细设计与实现

基于第三章的需求分析与概要设计，本章将对 MAGIC4FDG 系统的核心模块进行详细设计分析与具体实现展示，涵盖知识提取模块、策略规划与代码生成模块、模糊测试与覆盖率收集模块以及缺陷诊断模块，深入剖析各模块的内部结构和实现机制。具体而言，每节将首先借助类图和顺序图阐述该功能模块的详细设计；之后，在此基础上，结合关键代码片段，进一步展示核心功能的具体实现，解析关键逻辑、算法设计和优化策略。

4.1 知识提取模块详细设计与实现

知识提取模块是 MAGIC4FDG 系统的基础模块，其主要职责是在 Docker 容器内执行目标库的编译构建和 clang 抽象语法树导出，从中提取结构化的 API 知识，并根据不同智能体的需求将知识格式化为适合注入提示词的文本片段。本节首先通过类图和顺序图详细阐述该模块的设计思路；随后，结合关键代码片段和算法描述，深入解析 AST 解析与知识构建、差异化上下文构建两个核心功能的具体实现。

4.1.1 知识提取模块详细设计

图4-1展示了 MAGIC4FDG 系统知识提取模块的详细设计类图，包括该模块涉及的关键类以及类中的关键属性和方法。

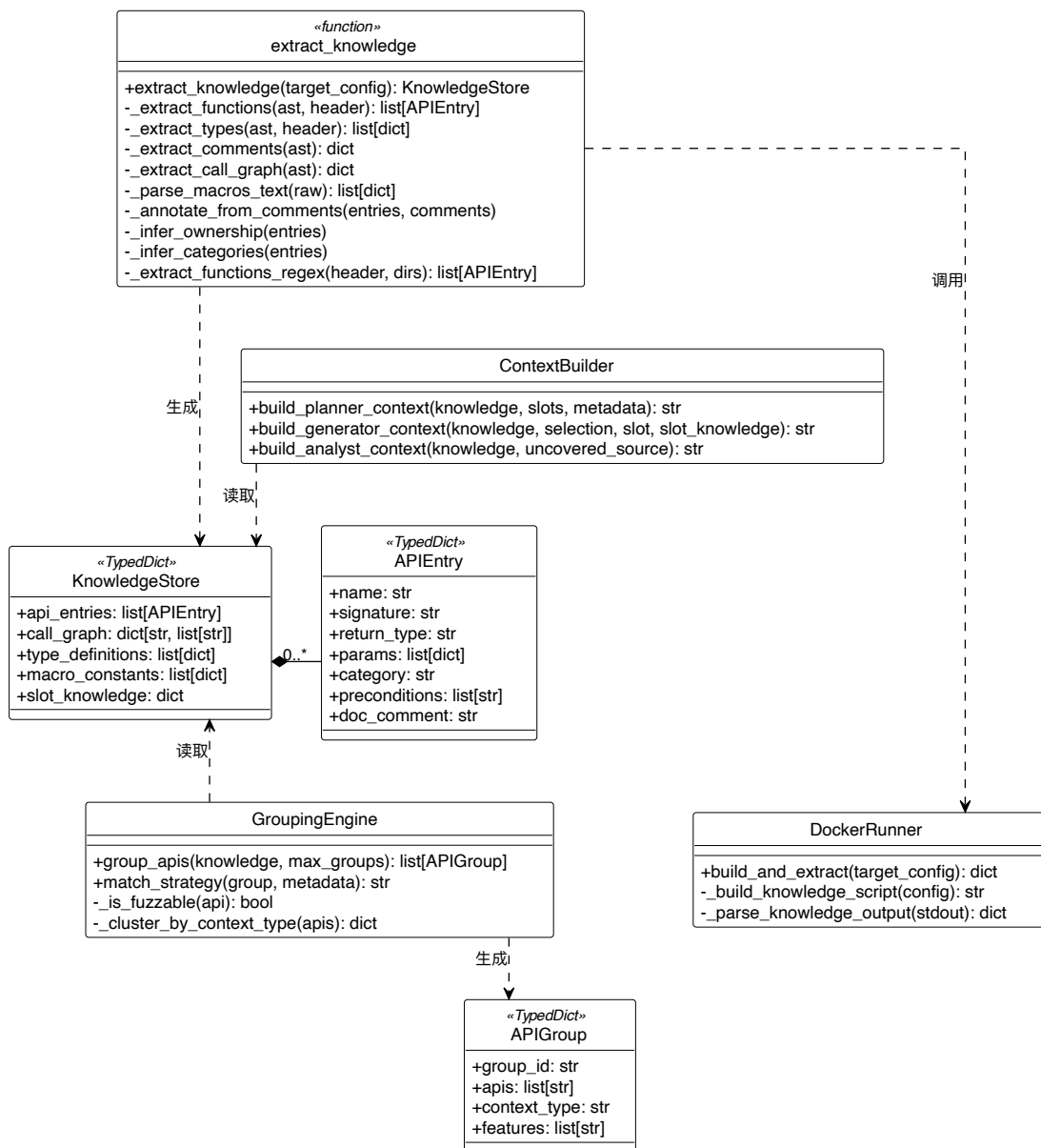


图 4-1 知识提取模块详细设计类图

该模块的核心由 `extract_knowledge` 函数驱动，该函数接收目标库配置作为输入，按照五阶段流水线依次执行 AST 解析、宏提取、调用图构建、语义标注和注释附加，最终输出完整的 `KnowledgeStore` 对象。`KnowledgeStore` 是系统的全局知识容器，包含四个核心字段：`api_entries` 存储从 AST 中提取的所有公开 API 条目，每个条目为 `APIEntry` 类型，包含函数名、完整签名、返回类型、参数列表、功能类别、前置条件和文档注释；`call_graph` 存储函数间的调用关系，以邻接表形式表示；`type_definitions` 存储结构体、枚举和类型别名的定义信息；`macro_constants` 存储从预处理器输出中提取的宏常量，按标志位、普通常

量和函数式宏三类分组。

`ContextBuilder` 类是知识处理的核心组件，负责将原始的 `KnowledgeStore` 转换为适合注入提示词的文本片段。该类提供三个公开方法：`build_planner_context` 为策略规划智能体生成包含 API 分类概览和调用图摘要的上下文；`build_generator_context` 为代码生成智能体生成包含目标 API 完整签名、参数语义和类型定义的详细上下文；`build_analyst_context` 为缺陷诊断智能体生成包含完整 API 清单和未覆盖源码行的诊断上下文。三种上下文的信息粒度和侧重点各不相同，体现了差异化上下文注入的设计理念。

`GroupingEngine` 类实现确定性的 API 分组和策略匹配功能，作为大语言模型策略规划失败时的回退方案。该类通过 `group_apis` 方法基于调用图的连通分量和参数类型相似性将 API 聚类为若干分组，再通过 `match_strategy` 方法基于规则引擎将每个分组的特征与策略元数据中的适用条件进行匹配。

图4-2展示了知识提取模块的详细设计顺序图，其中的组件为该模块的核心功能类，不同组件之间通过方法调用进行交互。

主函数首先调用 `knowledge_node` 函数启动知识提取流程。该函数从流水线状态中读取目标库配置，随后调用 `extract_knowledge` 函数执行实际的知识提取工作。`extract_knowledge` 函数首先调用 `DockerRunner` 的 `build_and_extract` 方法，在 `Docker` 容器内依次执行目标库编译构建、头文件 AST 导出、预处理器宏导出和源文件 AST 导出四项操作，将结果通过标准输出的分隔符协议返回给宿主机。

获取原始数据后，`extract_knowledge` 函数进入五阶段处理流水线：第一阶段调用 `_extract_functions` 解析头文件 AST JSON，遍历所有 `FunctionDecl` 节点提取公开 API 的函数签名；第二阶段调用 `_parse_macros_text` 解析预处理器输出，提取并分类宏常量；第三阶段对每个源文件 AST 调用 `_extract_call_graph`，递归遍历函数体中的 `CallExpr` 节点构建调用图；第四阶段依次调用 `_annotate_from_comments`、`_infer_ownership` 和 `_infer_categories` 三个语义标注方法，从文档注释和命名规则中推断 API 的所有权语义、前置条件和功能类别；第五阶段将提取的文档注释附加到对应的 API 条目中。若 AST 解析阶段未能提取到任何函数声明，系统自动回退至基于正则表达式的头文件扫描方案，确保知识提取的鲁棒性。

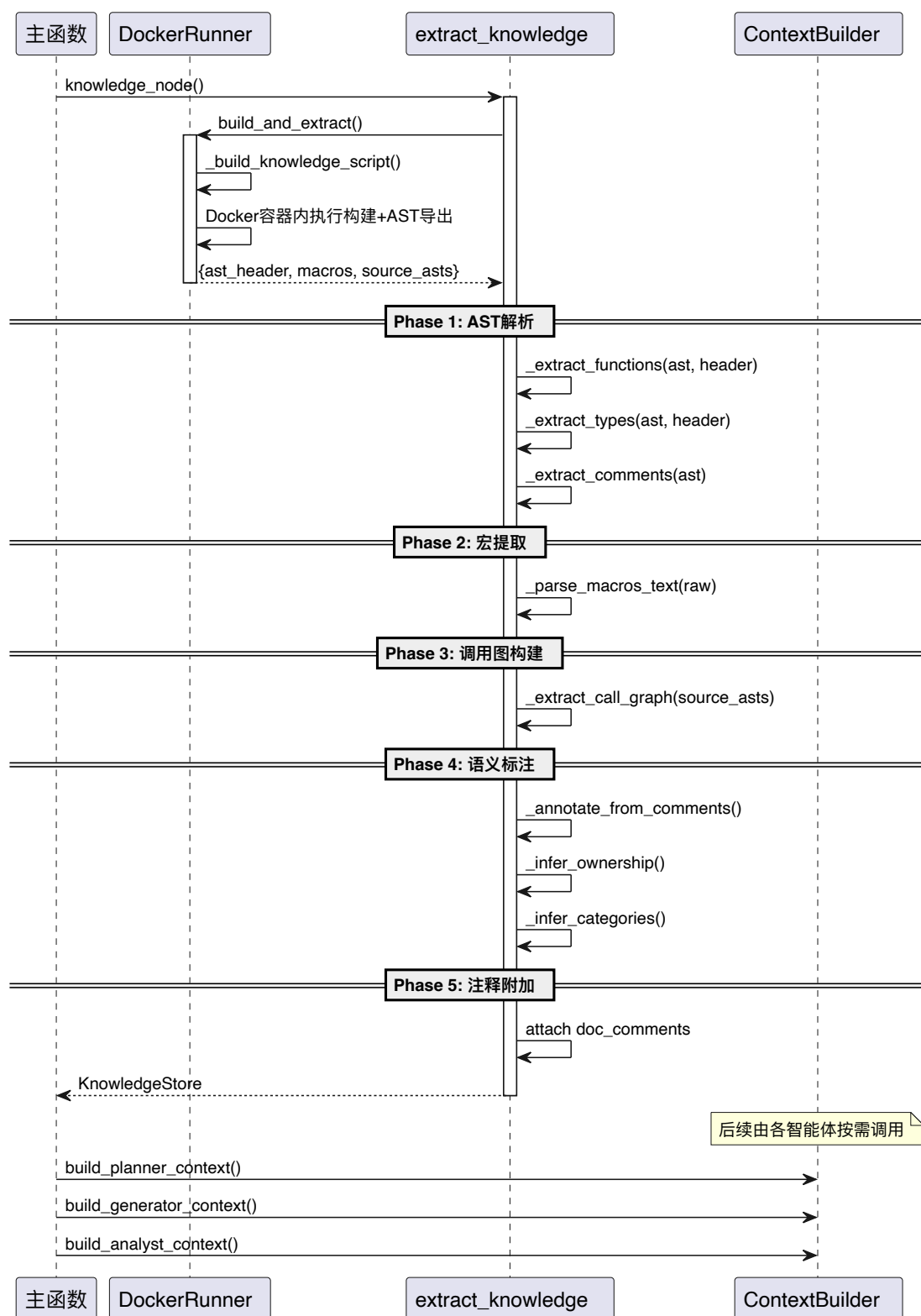


图 4-2 知识提取模块详细设计顺序图

4.1.2 AST 解析与知识构建功能实现

本小节深入解析知识提取模块中 AST 解析与知识构建的核心实现，包括 clang AST 深度优先遍历算法、函数签名提取、调用图构建和语义标注四个关键功能。

AST 深度优先遍历

clang 以 JSON 格式导出的抽象语法树是一棵嵌套结构的树，每个节点通过 `inner` 字段包含其子节点列表。系统实现了通用的深度优先遍历生成器 `_walk_ast`，作为所有 AST 分析功能的基础设施。该生成器采用 Python 的 `yield from` 语法实现惰性遍历，避免一次性将整棵 AST 展开到内存中，对于大型库（如 OpenSSL，AST JSON 可达数百 MB）尤为重要。该遍历器的实现如图4-3所示。

```
1 def _walk_ast(node: dict):
2     if not node:
3         return
4     yield node
5     for child in node.get("inner", []):
6         if child:
7             yield from _walk_ast(child)
```

图 4-3 AST 深度优先遍历生成器

函数签名提取

函数签名提取是知识构建的核心步骤，其目标是从 AST 中识别所有公开的 API 函数声明，提取函数名、完整类型签名、返回类型和参数列表。函数提取的核心实现如图4-4所示。

该函数的实现逻辑包含三层过滤机制：第一层通过 `kind` 字段筛选 `FunctionDecl` 类型的节点，排除变量声明、类型定义等无关节点；第二层通过 `loc` 字段中的文件路径信息，仅保留声明在目标头文件中的函数，排除系统头文件和第三方依赖引入的声明；第三层通过函数名前缀过滤，排除以下划线开头的内部实现函数。对于每个通过过滤的函数声明，系统从 `type.qualType` 字段提取完整的函数类型签名，并通过 `_extract_params` 方法遍历子节点中的 `ParmVarDecl` 节点提取参数信息，包括参数名、类型、是否可空和所有权语义。

```

1 def _extract_functions(ast: dict, header_path: str) -> list[APIEntry]:
2     entries: list[APIEntry] = []
3     header_name = Path(header_path).name if header_path else ""
4     for node in _walk_ast(ast):
5         if node.get("kind") != "FunctionDecl":
6             continue
7         loc = node.get("loc", {})
8         file_field = (loc.get("file", "") or
9                     loc.get("expansionLoc", {}).get("file", ""))
10        if file_field and header_name and header_name not in file_field:
11            continue
12        name = node.get("name", "")
13        if not name or name.startswith("_"):
14            continue
15        qual_type = node.get("type", {}).get("qualType", "")
16        params = _extract_params(node)
17        return_type = _infer_return_type(qual_type)
18        entries.append(APIEntry(
19            name=name, signature=qual_type,
20            return_type=return_type, params=params,
21            category="utility", preconditions=[],
22            group="", doc_comment="",
23        ))
24    return entries

```

图 4-4 基于 AST 的函数签名提取

当 AST 解析未能提取到任何函数声明时（例如目标库使用了 clang 无法处理的非标准扩展），系统自动回退至基于正则表达式的头文件扫描方案。该回退方案使用多行正则表达式匹配 C 函数声明的标准模式，虽然精度低于 AST 解析，但能确保知识提取流程不会因单一工具链的局限性而完全失败。

```

1 def _extract_calls_recursive(node: dict, current_func: str,
2                             graph: dict) -> None:
3     kind = node.get("kind", "")
4     if kind == "FunctionDecl":
5         current_func = node.get("name", "")
6     elif kind == "CallExpr" and current_func:
7         for child in node.get("inner", []):
8             callee = _resolve_callee(child)
9             if callee:
10                graph.setdefault(current_func, []).append(callee)
11                break
12    for child in node.get("inner", []):
13        if child:
14            _extract_calls_recursive(child, current_func, graph)

```

图 4-5 调用图递归构建

调用图构建

调用图记录了函数之间的调用关系，是策略规划智能体进行 API 分组和场景设计的重要依据。系统通过递归遍历源文件 AST 中的 `CallExpr` 节点构建调用图，核心实现如图4-5所示。

该算法维护一个 `current_func` 状态变量，在遍历过程中跟踪当前所处的函数作用域。当遇到 `FunctionDecl` 节点时更新当前函数名，当遇到 `CallExpr` 节点时通过 `_resolve_callee` 解析被调用函数的名称，并将调用关系记录到邻接表中。`_resolve_callee` 方法处理两种常见的 AST 表示：直接的 `DeclRefExpr` 引用和经过 `ImplicitCastExpr` 隐式转换的间接引用。系统对所有源文件的调用图进行合并和去重，最终生成全局的函数调用关系图。

语义标注

语义标注阶段从文档注释和函数命名规则中推断 API 的所有权语义、前置条件和功能类别，为代码生成智能体提供正确使用 API 所需的语义约束信息。该阶段包含三个独立的标注方法。

`_annotate_from_comments` 方法使用预定义的正则表达式模式集从文档注释中提取语义信息。系统定义了三类模式：返回值所有权模式（如“`caller is responsible to free`”）、参数所有权转移模式（如“`takes ownership`”）和前置条件模式（如“`must not be NULL`”）。当注释文本匹配某一模式时，系统自动为对应的 API 条目添加相应的前置条件或修改参数的所有权标记。

`_infer_ownership` 方法基于函数命名规则推断所有权语义。该方法识别两类命名模式：创建类函数（包含 `Create`、`New`、`Alloc` 等关键词）的返回值标记为调用者负责释放；销毁类函数（包含 `Delete`、`Free`、`Destroy` 等关键词）的指针参数标记为所有权转移。该方法的实现如图4-6所示。

`_infer_categories` 方法将每个 API 函数归类到六个功能类别之一：解析类（`parse`）、创建类（`create`）、销毁类（`delete`）、修改类（`modify`）、查询类（`query`）和序列化类（`serialize`）。分类基于函数名与预定义关键词模式的匹配，采用优先级顺序——首个匹配的类别即为最终分类。该分类信息用于策略规划智能体的 API 分组决策，例如将解析类和序列化类 API 配对以构建往返测试场景。

```

1 def _infer_ownership(entries: list[APIEntry]) -> None:
2     create_pattern = re.compile(
3         r"(Create|New|Duplicate|Copy|Clone|Alloc)", re.I)
4     delete_pattern = re.compile(
5         r"(Delete|Free|Destroy|Release|Close)", re.I)
6     for entry in entries:
7         name = entry["name"]
8         if create_pattern.search(name) and "*" in entry["return_type"]:
9             precondition = "Return value must be freed by caller"
10            if precondition not in entry["preconditions"]:
11                entry["preconditions"].append(precondition)
12            if delete_pattern.search(name):
13                for p in entry["params"]:
14                    if "*" in p["type"] and p["ownership"] == "borrow":
15                        p["ownership"] = "transfer"
16                break

```

图 4-6 基于命名规则的所有权推断

4.1.3 差异化上下文构建功能实现

本小节解析 `ContextBuilder` 模块的核心实现。该模块是 `KnowledgeStore` 与大语言模型提示词之间的翻译层，负责根据不同智能体的任务需求，从完整的知识库中选取相关子集并格式化为结构化的 `Markdown` 文本。系统设计了三种差异化上下文视图，体现了信息不对称的设计原则——每个智能体仅获取其任务所需的最小充分信息集，避免无关信息干扰大语言模型的推理。

策略规划上下文

`build_planner_context` 方法为策略规划智能体构建宏观视角的知识上下文。该方法的核心设计思想是提供“广度优先”的信息——让规划智能体了解目标库的整体 `API` 分布和函数间关系，而非单个 `API` 的实现细节。其实现如图4-7所示。

该方法输出的上下文包含四个信息层次：第一层为按功能类别分组的 `API` 概览，展示每个类别的函数数量和签名摘要，帮助规划智能体理解库的功能分布；第二层为调用图摘要，仅展示公开 `API` 之间的调用关系（过滤掉内部实现函数），帮助规划智能体识别 `API` 间的依赖链；第三层为关键类型定义摘要，展示结构体字段和枚举值；第四层为当前各槽位的状态信息（仅在第二轮及之后的迭代中包含），展示每个槽位的策略历史和覆盖率变化趋势，供规划智能体决策。

```

1 def build_planner_context(knowledge: KnowledgeStore,
2                           slots: list[HarnessSlot],
3                           strategy_metadata: list[dict]) -> str:
4     lines = ["## Target Library API Surface\n"]
5     by_category: dict[str, list] = {}
6     for api in knowledge["api_entries"]:
7         by_category.setdefault(api["category"], []).append(api)
8     for cat in ["parse", "create", "modify", "query",
9                "delete", "serialize", "utility"]:
10         apis = by_category.get(cat, [])
11         if apis:
12             lines.append(f"### {cat.title()} ({len(apis)} functions)\n")
13             for api in apis:
14                 params_str = ", ".join(
15                     f"{p['type']} {p['name']}" for p in api.get("params", []))
16                 lines.append(
17                     f"- {api['return_type']} {api['name']}({params_str})`")
18 # ... 调用图 + 类型定义 + slot状态
19     return "\n".join(lines)

```

图 4-7 策略规划上下文构建

代码生成上下文

`build_generator_context` 方法为代码生成智能体构建“深度优先”的详细上下文。与规划上下文的宏观视角不同，生成上下文聚焦于规划智能体指定的目标 API 子集，提供生成正确模糊驱动代码所需的全部细节信息。其核心实现如图4-8所示。

```

1 def build_generator_context(knowledge: KnowledgeStore,
2                             selection: StrategySelection,
3                             slot: HarnessSlot | None = None,
4                             slot_knowledge: dict | None = None) -> str:
5     lines = ["## API Knowledge\n"]
6     target_api_names = set(selection.get("target_apis", []))
7     relevant_apis = list(knowledge["api_entries"])
8     if target_api_names:
9         relevant_apis = [a for a in knowledge["api_entries"]
10                          if a["name"] in target_api_names]
11     for api_name in list(target_api_names):
12         callees = knowledge["call_graph"].get(api_name, [])
13         for callee in callees:
14             for a in knowledge["api_entries"]:
15                 if a["name"] == callee and a not in relevant_apis:
16                     relevant_apis.append(a)
17 # ... 详细API信息 + 类型定义 + 宏常量 + 累积约束
18     return "\n".join(lines)

```

图 4-8 代码生成上下文构建

该方法的关键设计决策体现在三个方面。第一，API 范围聚焦：当规划智能体指定了 `target_apis` 时，生成上下文仅包含目标 API 及其调用图中的直接被调用者，避免无关 API 信息稀释大语言模型的注意力。第二，信息排序优化：在每个功能类别内，有文档注释的 API 排列在前，帮助大语言模型优先关注信息最丰富的接口。第三，跨轮次知识注入：从该槽位隔离的 `SlotKnowledge` 中提取累积的约束、正向模式和负向模式，注入生成上下文中，使大语言模型能够利用历史经验避免重复错误。

缺陷诊断上下文

`build_analyst_context` 方法为缺陷诊断智能体构建以覆盖率缺口为中心的诊断上下文。该方法的设计目标是提供诊断覆盖率问题所需的完整信息链——从未覆盖的源码行出发，结合 API 清单和调用图，帮助诊断智能体推断覆盖率缺口的根因。其核心实现如图4-9所示。

```
1 def build_analyst_context(knowledge: KnowledgeStore,
2                           uncovered_source: dict | None = None) -> str:
3     lines = ["## Full API Inventory\n"]
4     for api in knowledge["api_entries"]:
5         ownership_mark = ""
6         if any(p.get("ownership") == "transfer"
7               for p in api.get("params", [])):
8             ownership_mark = " [takes ownership]"
9         lines.append(
10             f"- `{api['signature']}` [{api['category']}] {ownership_mark}")
11     # 调用图 + 宏常量标志 + 未覆盖源码行
12     if uncovered_source:
13         lines.append("## Uncovered Source Lines\n")
14         for filename, line_map in list(uncovered_source.items())[:5]:
15             lines.append(f"### {filename}")
16             for line_no, code in sorted(line_map.items())[:20]:
17                 lines.append(f" {line_no}: {code.rstrip()}")
18     return "\n".join(lines)
```

图 4-9 缺陷诊断上下文构建

与前两种上下文不同，诊断上下文传递完整的 API 清单而非子集，因为诊断智能体需要判断哪些 API 尚未被任何驱动覆盖。此外，该上下文包含未覆盖源码行的原始文本（每个文件最多展示 20 行），使诊断智能体能够直接观察未覆盖代码的具体逻辑，从而给出精确的改进建议。调用图信息帮助诊断智能体理解函数间的依赖关系——例如，若某个函数未被覆盖，可能是因为其前置调

用函数未被正确初始化。

表4-1总结了三种上下文视图的信息差异。

表 4-1 三种差异化上下文视图的信息对比

信息维度	规划上下文	生成上下文	诊断上下文
API 范围	全量（按类别分组）	目标子集（含被调用者）	全量（含所有权标记）
参数详情	类型摘要	完整（含 ownership）	无
调用图	公开 API 间关系	无	完整调用图
类型定义	摘要	完整（含字段）	无
宏常量	无	完整（分类展示）	仅标志位
覆盖率信息	槽位历史趋势	无	未覆盖源码行
累积知识	无	约束 + 正/负模式	无

4.2 策略规划与代码生成模块详细设计与实现

策略规划与代码生成模块是 MAGIC4FDG 系统的核心智能模块，负责将结构化的 API 知识转化为可编译执行的模糊驱动代码。该模块由策略规划智能体和代码生成智能体协同工作：规划智能体基于大语言模型的语义推理能力设计测试场景并分配策略，生成智能体则根据规划结果和策略库指导，调用大语言模型生成具体的模糊驱动源码。本节首先通过类图和顺序图阐述该模块的详细设计，随后分别解析探索阶段策略规划和多提示词策略驱动生成两个核心功能的具体实现。

4.2.1 策略规划与代码生成模块详细设计

图4-10展示了策略规划与代码生成模块的详细设计类图。

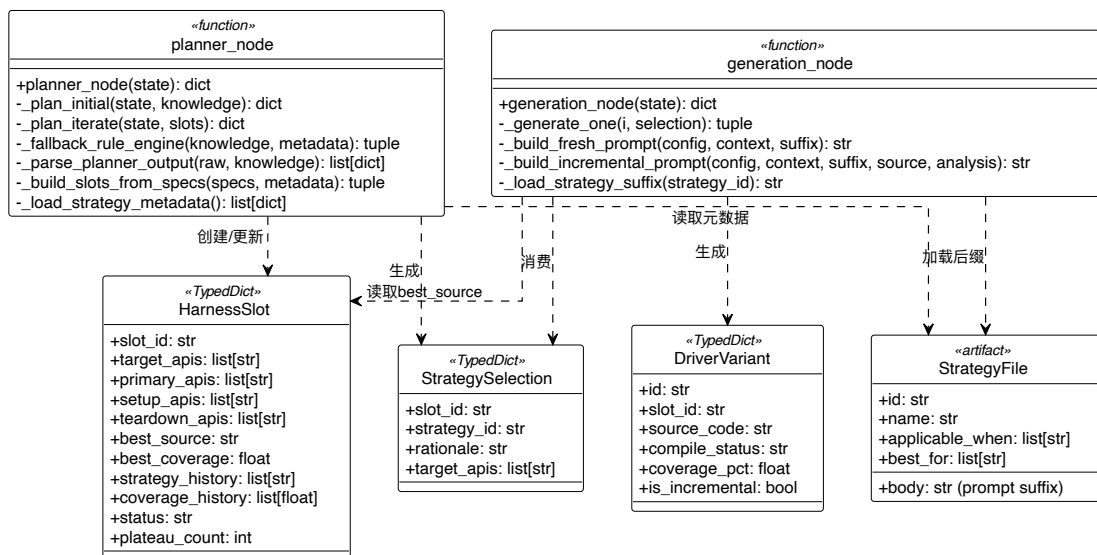


图 4-10 策略规划与代码生成模块详细设计类图

该模块包含两个核心函数类和四个数据类型。`planner_node` 函数类封装了策略规划的全部逻辑，其公开方法 `planner_node` 作为 `LangGraph` 节点入口，内部根据当前轮次分派到 `_plan_initial`（探索阶段）或 `_plan_iterate`（利用阶段）。`_plan_initial` 方法调用大语言模型进行场景推理，通过 `_parse_planner_output` 解析 LLM 返回的 JSON 结构，再由 `_build_slots_from_specs` 创建具体的槽位对象。当 LLM 调用失败时，`_fallback_rule_engine` 方法调用 `GroupingEngine` 进行确定性降级。

`generation_node` 函数类封装了代码生成的全部逻辑，通过 `ThreadPoolExecutor` 并行调用大语言模型为每个策略选择生成一个模糊驱动变体。其内部区分两种提示词构建路径：`_build_fresh_prompt` 用于首轮从零生成，`_build_incremental_prompt` 用于后续轮次的增量改进。`_load_strategy_suffix` 方法从策略文件中加载 YAML 前置元数据之后的正文部分，作为提示词的策略后缀。

`HarnessSlot` 是系统的核心状态单元，代表一个独立的模糊测试场景。每个槽位维护其目标 API 集合（分为 `primary`、`setup`、`teardown` 三类）、历史最佳驱动源码、覆盖率历史、策略切换记录和停滞计数器。`StrategySelection` 记录规划智能体为每个槽位做出的策略决策，包括策略 ID、决策理由和目标 API 列表。`DriverVariant` 记录生成智能体产出的每个驱动变体的完整信息。`StrategyFile` 表示策略库中的策略文件，包含 YAML 前置元数据（适用条件、最佳场景）和 Markdown 正文（生成指导）。

图4-11展示了策略规划与代码生成模块的详细设计顺序图，清晰呈现了探索阶段和利用阶段的不同执行路径。

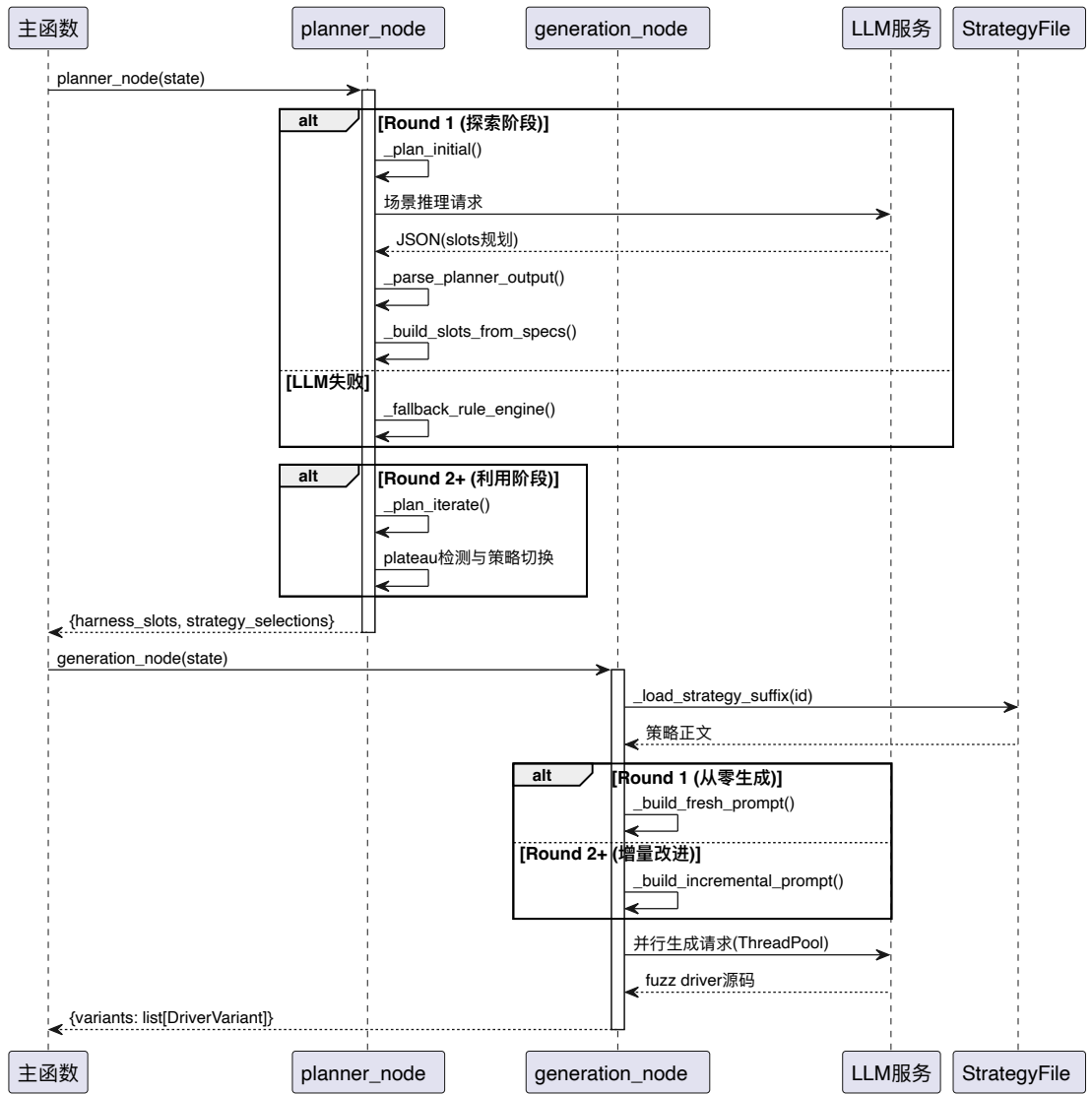


图 4-11 策略规划与代码生成模块详细设计顺序图

主函数首先调用 `planner_node` 启动策略规划。在探索阶段（Round 1），规划智能体将完整的 API 知识、调用图和类型定义注入提示词模板，调用大语言模型进行场景推理，LLM 返回包含多个槽位规划的 JSON 结构，经过解析和校验后创建 `HarnessSlot` 和 `StrategySelection` 对象。若 LLM 调用失败，系统降级到规则引擎进行确定性分组和策略匹配。在利用阶段（Round 2+），规划智能体遍历所有活跃槽位，根据停滞计数器决定是否切换策略——连续两轮无覆盖率提升时切换到 `targeted-expansion` 策略，连续四轮无提升时标记为收敛。

随后主函数调用 `generation_node` 启动代码生成。生成智能体首先加载对

应的策略文件后缀，然后根据当前轮次选择提示词构建路径：首轮使用三部分组装的从零生成提示词，后续轮次使用包含现有代码和诊断约束的增量改进提示词。最终通过线程池并行调用大语言模型，为每个策略选择生成一个模糊驱动变体。

4.2.2 探索阶段策略规划功能实现

本小节解析策略规划智能体在探索阶段（Round 1）的核心实现，包括 LLM 场景推理、输出解析校验和确定性回退机制。

LLM 场景推理

探索阶段的核心任务是将目标库的 API 知识转化为若干独立的模糊测试场景。系统将完整的 API 分类概览、调用图和类型定义注入规划提示词模板，要求大语言模型分析 API 间的语义关系，设计互不重叠的测试场景，并为每个场景选择最匹配的策略。_plan_initial 方法的核心实现如图4-12所示。

```
1 def _plan_initial(state: PipelineState, knowledge: dict) -> dict:
2     target_config = state["target_config"]
3     lib_name = target_config.get("library_name", "?")
4     strategy_metadata = _load_strategy_metadata()
5     context = build_planner_context(knowledge, [], strategy_metadata)
6     strategies_text = _format_strategies(strategy_metadata)
7     prompt = PLANNER_TEMPLATE.replace("{{LIBRARY_NAME}}", lib_name)
8     prompt = prompt.replace("{{KNOWLEDGE_CONTEXT}}", context)
9     prompt = prompt.replace("{{CALL_GRAPH}}", _format_call_graph(knowledge))
10    prompt = prompt.replace("{{TYPE_DEFINITIONS}}", _format_type_defs(knowledge))
11    prompt = prompt.replace("{{STRATEGIES}}", strategies_text)
12    try:
13        llm = create_llm(temperature=max(state.get("temperature", 0.4) - 0.1, 0.1))
14        response = llm.invoke([...])
15        slot_specs = _parse_planner_output(response.content, knowledge)
16        slots, selections = _build_slots_from_specs(slot_specs, strategy_metadata)
17    except Exception:
18        slots, selections = _fallback_rule_engine(knowledge, strategy_metadata)
19    return {"harness_slots": slots, "strategy_selections": selections}
```

图 4-12 探索阶段策略规划核心流程

该方法的提示词组装采用模板变量替换机制，将五个知识维度注入固定模板：库名称、API 知识上下文、调用图、类型定义和可用策略列表。温度参数设置为比全局温度低 0.1（最低 0.1），以获得更确定性的规划输出。

输出解析与校验

大语言模型返回的规划结果为 JSON 格式，系统通过 `_parse_planner_output` 方法进行严格的解析和校验。该方法的实现如图4-13所示。

```
1 def _parse_planner_output(raw: str, knowledge: dict) -> list[dict]:
2     text = raw.strip()
3     match = re.search(r"```(?:json)?\s*\n(?:.+?)```", text, re.DOTALL)
4     if match:
5         text = match.group(1).strip()
6         data = json.loads(text)
7         slots_raw = data.get("slots", [])
8         valid_apis = {a["name"] for a in knowledge.get("api_entries", [])}
9         valid_strategies = {"parse-centric", "multi-api-sequence",
10                             "roundtrip", "targeted-expansion", ...}
11     result = []
12     for spec in slots_raw:
13         primary = [a for a in spec.get("primary_apis", []) if a in valid_apis]
14         if not primary:
15             continue
16         strategy = spec.get("strategy_id", "multi-api-sequence")
17         if strategy not in valid_strategies:
18             strategy = "multi-api-sequence"
19         result.append({...})
20     return result
```

图 4-13 LLM 规划输出解析与校验

该方法实现了三层容错机制：第一层处理 LLM 输出格式的不确定性，支持从 Markdown 代码块中提取 JSON，也支持直接的 JSON 文本；第二层对 API 名称进行有效性校验，过滤掉 LLM 幻觉产生的不存在的函数名；第三层对策略 ID 进行合法性校验，将无效策略回退到默认的 `multi-api-sequence`。只有至少包含一个有效 `primary` API 的槽位才会被保留。

确定性回退机制

当 LLM 调用失败（网络超时、配额耗尽或返回格式无法解析）时，系统降级到基于规则引擎的确定性回退方案，确保流水线不会因单次 LLM 失败而中断。回退机制的实现如图4-14所示。

```

1 def _fallback_rule_engine(knowledge: dict,
2                           strategy_metadata: list[dict]
3                           ) -> tuple[list[HarnessSlot], list[StrategySelection]]:
4     groups = group_apis(knowledge, max_groups=10)
5     slots: list[HarnessSlot] = []
6     selections: list[StrategySelection] = []
7     for i, group in enumerate(groups):
8         strategy_id = match_strategy(group, strategy_metadata)
9         slot_id = f"slot_{i}"
10        slots.append(HarnessSlot(
11            slot_id=slot_id, group_id=group["group_id"],
12            target_apis=group["apis"], primary_apis=group["apis"],
13            strategy_history=[strategy_id], status="active", ...
14        ))
15        selections.append(StrategySelection(
16            slot_id=slot_id, strategy_id=strategy_id,
17            target_apis=group["apis"], ...
18        ))
19    return slots, selections

```

图 4-14 确定性回退规则引擎

回退方案调用 `GroupingEngine` 的 `group_apis` 方法，基于调用图连通分量和参数类型相似性将 API 聚类为最多 10 个分组，再通过 `match_strategy` 方法基于规则评分为每个分组匹配最适合的策略。该方案虽然缺乏 LLM 的语义推理能力，但保证了确定性和可重复性。

4.2.3 多提示词策略驱动生成功能实现

本小节解析代码生成智能体的核心实现，包括策略文件的结构设计、三部分提示词组装机制和增量改进生成。

策略文件结构

MAGIC4FDG 系统的策略库包含 10 个策略文件，每个文件采用 YAML 前置元数据加 Markdown 正文的双层结构。前置元数据定义策略的适用条件和最佳场景，供规划智能体进行策略匹配；正文部分包含具体的生成指导，作为提示词后缀注入代码生成请求。图4-15展示了策略文件的结构示例。

```

1 ---
2 id: parse-centric
3 name: Parse-Centric Input Fuzzing
4 applicable_when:
5   - "library has parsing functions that accept raw bytes"
6   - "API includes multiple parse variants"
7 best_for:
8   - "parser libraries (JSON, XML, image)"
9   - "format decoders"
10 ---
11 ## Strategy: Parse-Centric
12 Feed raw fuzz bytes directly to ALL parsing API variants.
13 - Call every parse function variant
14 - Use different option flag combinations
15 - Exercise error paths by feeding malformed data
16 - Use FuzzedDataProvider to split input

```

图 4-15 策略文件结构示例 (parse-centric)

系统通过 `_load_strategy_suffix` 方法加载策略正文。该方法遍历策略目录下的所有 Markdown 文件，通过匹配 `id` 字段定位目标策略，然后截取 YAML 前置元数据结束标记（第二个 `---`）之后的全部内容作为策略后缀返回。

从零生成提示词组装

首轮生成采用三部分组装机制构建提示词：基础模板提供通用的生成框架和约束，知识上下文提供目标 API 的详细信息，策略后缀提供特定策略的生成指导。`_build_fresh_prompt` 方法的实现如图4-16所示。

```

1 def _build_fresh_prompt(target_config: dict, knowledge_context: str,
2                          strategy_suffix: str) -> str:
3     include_dirs = target_config.get("include_dirs", [])
4     prompt = (
5         GENERATION_TEMPLATE
6         .replace("{{LIBRARY_NAME}}", target_config.get("library_name", ""))
7         .replace("{{LANGUAGE}}", target_config.get("language", "C"))
8         .replace("{{HEADER}}", target_config.get("header", ""))
9         .replace("{{INCLUDE_DIRS}}", ", ".join(include_dirs))
10        .replace("{{RESEARCH_SUMMARY}}", knowledge_context)
11        .replace("{{COVERAGE_FEEDBACK}}", "(first round)")
12    )
13    if strategy_suffix:
14        prompt += "\n\n" + strategy_suffix
15    return prompt

```

图 4-16 从零生成提示词三部分组装

基础模板 `GENERATION_TEMPLATE` 定义了模糊驱动的通用要求：必须包含

LLVMFuzzerTestOneInput 入口函数、正确的头文件包含、内存安全的资源管理等。知识上下文通过 `build_generator_context` 方法构建，包含目标 API 的完整签名、参数语义、类型定义和宏常量。策略后缀则提供针对特定测试策略的具体指导，例如 `parse-centric` 策略要求“将原始模糊字节直接传递给所有解析 API 变体”。

增量改进提示词组装

第二轮及之后的迭代采用增量改进模式，基于该槽位的历史最佳驱动进行修改，而非从零重新生成。这一设计保留了已有的覆盖率成果，同时针对诊断智能体识别的覆盖率缺口进行定向改进。`_build_incremental_prompt` 方法的实现如图4-17所示。

```
1 def _build_incremental_prompt(target_config: dict, knowledge_context: str,
2                               strategy_suffix: str, base_source: str,
3                               coverage_analysis: dict) -> str:
4     constraints_text = ""
5     if coverage_analysis:
6         constraints = coverage_analysis.get("constraints", [])
7         if constraints:
8             constraints_text = "\n".join(
9                 f"- {c.get('target', '?')}: {c.get('precondition', '')}"
10                for c in constraints[:10])
11     clusters_text = ""
12     if coverage_analysis:
13         clusters = coverage_analysis.get("uncovered_clusters", [])
14         if clusters:
15             clusters_text = "\n".join(
16                 f"- {' '.join(c.get('functions', []))}: {c.get('root_cause', '')}"
17                for c in clusters[:5])
18     prompt = f"""## Task: Improve Fuzz Driver
19               ## Current Driver (to be improved)
20               ```cpp
21               {base_source}
22               ```
23               ## Analyst Constraints
24               {constraints_text or "(no specific constraints)"}
25               ## Uncovered Code Clusters
26               {clusters_text or "(no cluster data)"}
27               ## API Knowledge
28               {knowledge_context}
29               ## Strategy Guidance
30               {strategy_suffix}"""
31     return prompt
```

图 4-17 增量改进提示词组装

增量提示词包含五个信息层次：现有驱动源码（作为修改基础）、诊断约束（最多 10 条，指明需要覆盖的具体目标和前置条件）、未覆盖代码簇（最多 5 个，包含根因分析）、API 知识上下文和策略指导。提示词明确要求大语言模型”保留现有驱动中有效的部分，仅添加新的代码路径”，避免增量改进过程中丢失已有覆盖率。

4.3 模糊测试与覆盖率收集模块详细设计与实现

模糊测试与覆盖率收集模块是 MAGIC4FDG 系统的执行与反馈核心，负责在 Docker 容器内运行生成的模糊驱动、收集细粒度的代码覆盖率数据，并通过控制流图可达性分析过滤结构性不可达的代码路径。该模块的输出直接驱动系统的迭代决策——覆盖率提升则继续优化，停滞则切换策略或终止。本节首先通过类图和顺序图阐述该模块的详细设计，随后分别解析 Fork 模式模糊测试与语料库重放、控制流图可达性分析两个核心功能的具体实现。

4.3.1 模糊测试与覆盖率收集模块详细设计

图4-18展示了模糊测试与覆盖率收集模块的详细设计类图。

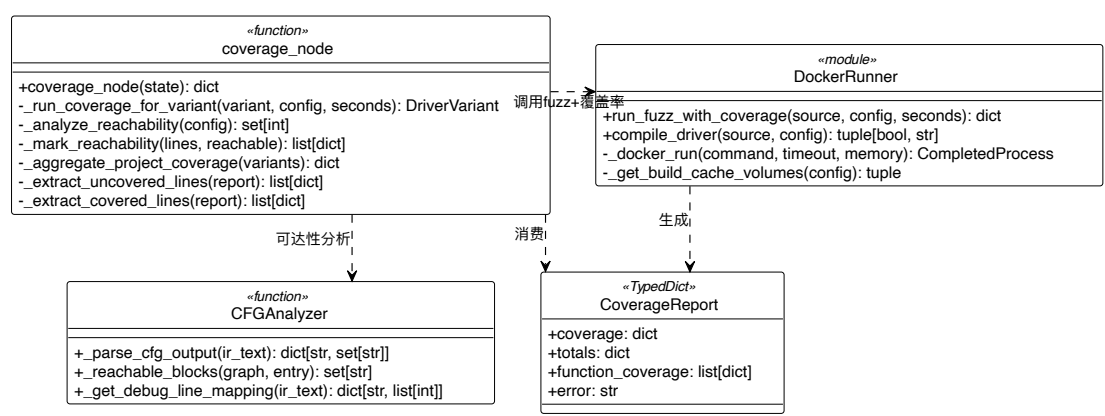


图 4-18 模糊测试与覆盖率收集模块详细设计类图

该模块包含三个核心组件。coverage_node 函数类是 LangGraph 节点入口，负责编排整个覆盖率收集流程：首先执行可达性分析获取可达行集合，然后通过线程池并行对所有编译成功的变体执行模糊测试，最后更新各槽位的覆盖率状态并聚合项目级覆盖率。DockerRunner 模块封装了 Docker 容器内的所有执行操

作，其核心方法 `run_fuzz_with_coverage` 实现了从编译到覆盖率导出的完整八步流水线。`CFGAnalyzer` 组件实现了基于 LLVM IR 的控制流图可达性分析，包含 CFG 解析、BFS 遍历和调试信息行号映射三个子功能。

`CoverageReport` 记录单个变体的覆盖率收集结果，包含行覆盖率、分支覆盖率、函数覆盖率和未覆盖行列表。每个未覆盖行条目额外标注 `reachable` 字段，指示该行是否从驱动入口函数结构性可达。

图4-19展示了模糊测试与覆盖率收集模块的详细设计顺序图。

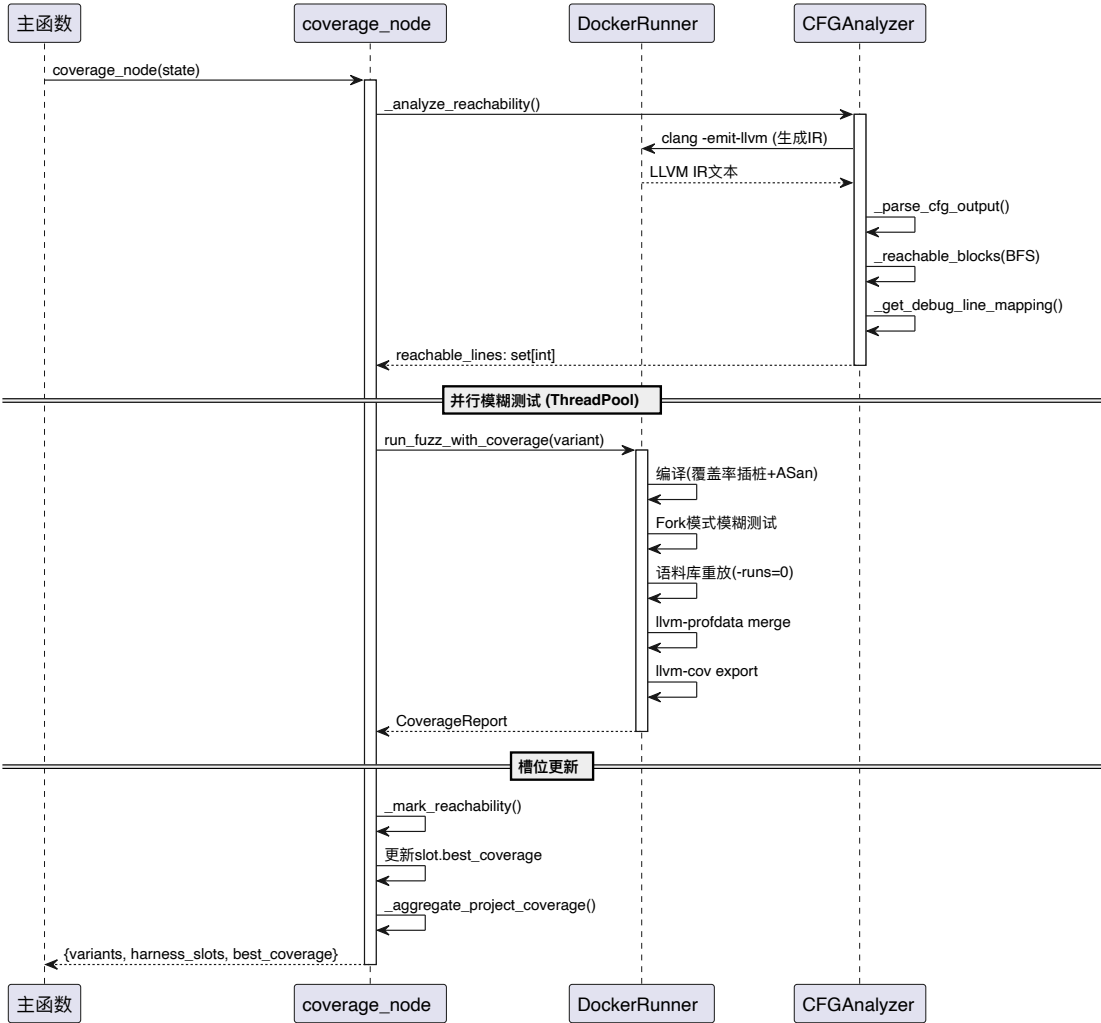


图 4-19 模糊测试与覆盖率收集模块详细设计顺序图

主函数调用 `coverage_node` 启动覆盖率收集流程。该节点首先调用 `CFGAnalyzer` 执行可达性分析：在 Docker 容器内通过 `clang -emit-llvm` 生成 LLVM IR，解析 IR 文本构建控制流图，从入口块执行 BFS 获取可达基本块集合，最后通过调试信息将可达块映射到源码行号。随后，节点通过线程池并行调用 `DockerRunner`

的 `run_fuzz_with_coverage` 方法，对每个编译成功的变体执行八步覆盖率收集流水线。收集完成后，节点为每个未覆盖行标注可达性，更新各槽位的最佳覆盖率和停滞计数器，并聚合所有槽位覆盖行的并集计算项目级覆盖率。

4.3.2 Fork 模式模糊测试与语料库重放功能实现

本小节解析 `run_fuzz_with_coverage` 方法实现的八步覆盖率收集流水线，重点阐述 Fork 模式崩溃隔离和语料库重放的设计动机与实现细节。

八步流水线设计

覆盖率收集流水线在单个 Docker 容器内依次执行八个步骤，其核心流程如图4-20所示。

```
1 # Step 1: 编译(覆盖率插桩 + ASan)
2 COMMON_FLAGS="-g -O1 -fprofile-instr-generate -fcoverage-mapping
3             -fsanitize=address"
4 clang++ $COMMON_FLAGS driver.cpp -fsanitize=fuzzer,address -o fuzz_cov
5
6 # Step 2: 准备语料库(种子 + 累积)
7 mkdir -p /tmp/corpus && cp seed_corpus/* /tmp/corpus/
8 cp accumulated_corpus/* /tmp/corpus/
9
10 # Step 3: Fork模式模糊测试(崩溃隔离)
11 export ASAN_OPTIONS=halt_on_error=0:exitcode=0:detect_leaks=0
12 ./fuzz_cov /tmp/corpus -max_total_time=60 -fork=1
13
14 # Step 4: 保存累积语料库
15 cp /tmp/corpus/* accumulated_corpus/
16
17 # Step 5: 语料库重放(无Fork, 收集完整profrw)
18 export LLVM_PROFILE_FILE=/tmp/replay_%m.profrw
19 ./fuzz_cov /tmp/corpus -runs=0
20
21 # Step 6: 合并覆盖率数据
22 llvm-profdata merge -sparse /tmp/replay_*.profrw -o fuzzer.profrw
23
24 # Step 7: 导出覆盖率报告(JSON)
25 llvm-cov export ./fuzz_cov -instr-profile=fuzzer.profrw sources
26
27 # Step 8: 导出函数级报告
28 llvm-cov report ./fuzz_cov -instr-profile=fuzzer.profrw sources
```

图 4-20 八步覆盖率收集流水线

Fork 模式崩溃隔离

LibFuzzer 的 Fork 模式 (`-fork=1`) 将每个模糊输入在独立子进程中执行, 当 AddressSanitizer 检测到内存错误时, 仅子进程被终止, 主进程继续运行后续输入。这一机制对覆盖率收集至关重要: 若不使用 Fork 模式, 单次 ASan 崩溃将终止整个模糊测试进程, 导致后续输入无法执行, 覆盖率数据严重不完整。系统同时设置 `ASAN_OPTIONS=halt_on_error=0:exitcode=0:detect_leaks=0`, 确保即使在非 Fork 模式的重放阶段, ASan 错误也不会中断执行。

语料库重放设计动机

Fork 模式存在一个固有缺陷: 子进程在超时时被 SIGKILL 强制终止, 导致 profraw 覆盖率数据文件无法正常刷新到磁盘。因此, 系统在 Fork 模式模糊测试完成后, 额外执行一次无 Fork 模式的语料库重放 (`-runs=0`), 以收集完整的覆盖率数据。重放阶段使用独立的 LLVM_PROFILE_FILE 路径, `llvm-profdata merge` 优先合并重放产生的完整数据。这一“先 Fork 后重放”的两阶段设计, 在保证崩溃隔离的同时获取了精确的覆盖率信息。

系统还实现了跨轮次的语料库累积机制: 每轮模糊测试结束后, 将新发现的语料库文件保存到持久化目录, 下一轮测试启动时自动加载历史语料库作为初始种子。这使得后续轮次能够在前序轮次的探索基础上继续深入, 避免重复探索已知路径。

4.3.3 控制流图可达性分析功能实现

本小节解析基于 LLVM IR 的控制流图可达性分析实现, 该分析用于过滤结构性不可达的未覆盖代码行, 避免诊断智能体在无法通过任何输入触达的代码路径上浪费推理资源。

LLVM IR 生成与 CFG 解析

可达性分析的第一步是在 Docker 容器内通过 `clang -S -emit-llvm -g -O0` 将目标源文件编译为 LLVM 中间表示 (IR), 保留完整的调试信息。随后, 系统解析 IR 文本构建控制流图。CFG 解析的核心实现如图4-21所示。


```

1 def _parse_cfg_output(cfg_text: str) -> dict[str, set[str]]:
2     graph: dict[str, set[str]] = {}
3     current_block = None
4     for line in cfg_text.splitlines():
5         block_match = re.match(r"^(\S+):.*$", line.strip())
6         if block_match:
7             current_block = block_match.group(1)
8             if current_block not in graph:
9                 graph[current_block] = set()
10            continue
11        if current_block:
12            br_match = re.match(r"\s*br\s+.*label\s+(\S+)", line)
13            if br_match:
14                targets = re.findall(r"label\s+([A-Za-z0-9_.]+)", line)
15                for t in targets:
16                    graph[current_block].add(t)
17                    if t not in graph:
18                        graph[t] = set()
19    return graph

```

图 4-21 LLVM IR 控制流图解析

该方法逐行扫描 LLVM IR 文本，通过正则表达式识别两类关键结构：基本块标签（行首非空白字符后跟冒号）和分支指令（br 指令中的 label 目标）。解析结果为邻接表形式的有向图，其中每个节点为基本块标签，边表示控制流转移关系。

BFS 可达性遍历与行号映射

获取 CFG 后，系统从入口基本块出发执行广度优先搜索，确定所有从程序入口可达的基本块。随后通过 LLVM IR 中的 !dbg 调试元数据，将可达基本块映射到对应的源码行号。可达性分析的完整流程如图4-22所示。

_get_debug_line_mapping 方法通过两阶段处理建立基本块到源码行的映射：第一阶段扫描 IR 末尾的 !DILocation 元数据定义，建立元数据 ID 到行号的索引；第二阶段遍历函数体中每条指令的 !dbg 引用，将行号归属到对应的基本块。最终，_mark_reachability 方法将可达性信息标注到每个未覆盖行条目的 reachable 字段中，诊断智能体仅对 reachable=True 的未覆盖行进行分析，有效避免了在死代码和编译器生成的不可达路径上浪费推理资源。

```

1 def _analyze_reachability(target_config: dict) -> set[int]:
2     source_files = target_config.get("source_files", [])
3     if not source_files:
4         return set()
5     primary_source = source_files[0]
6     include_args = " ".join(f"-I{d}" for d in target_config.get("include_dirs", []))
7     # Docker内生成LLVM IR
8     result = _docker_run(["bash", "reachability_analysis.sh"])
9     ir_text = result.stdout
10    # 解析CFG并执行BFS
11    graph = _parse_cfg_output(ir_text)
12    entry = "entry" if "entry" in graph else next(iter(graph), "")
13    reachable = _reachable_blocks(graph, entry)
14    # 映射到源码行号
15    block_to_lines = _get_debug_line_mapping(ir_text)
16    reachable_lines: set[int] = set()
17    for block_label in reachable:
18        reachable_lines.update(block_to_lines.get(block_label, []))
19    return reachable_lines

```

图 4-22 控制流图可达性分析完整流程

4.4 缺陷诊断模块详细设计与实现

缺陷诊断模块是 MAGIC4FDG 系统迭代闭环的关键环节，负责分析每个槽位的覆盖率缺口，诊断未覆盖代码的根因，并输出结构化的改进约束供下一轮代码生成使用。该模块的核心设计原则是按槽位隔离——每个槽位独立分析自己职责范围内的覆盖率数据，避免跨槽位的错误引导。本节首先通过类图和顺序图阐述该模块的详细设计，随后分别解析覆盖率缺口诊断和跨轮次知识累积两个核心功能的具体实现。

4.4.1 缺陷诊断模块详细设计

图4-23展示了缺陷诊断模块的详细设计类图。

该模块的核心由 `analyst_node` 函数类驱动，其公开方法作为 `LangGraph` 节点入口，内部通过线程池并行对每个活跃槽位调用 `_analyze_single_slot` 方法执行独立诊断。诊断过程依赖 `ContextBuilder` 模块构建诊断上下文，并通过 `_build_uncovered_summary` 和 `_build_function_summary` 两个辅助方法将覆盖率数据格式化为大语言模型可理解的文本。`_parse_analyst_response` 方法从 LLM 返回的文本中提取 JSON 格式的结构化分析结果。

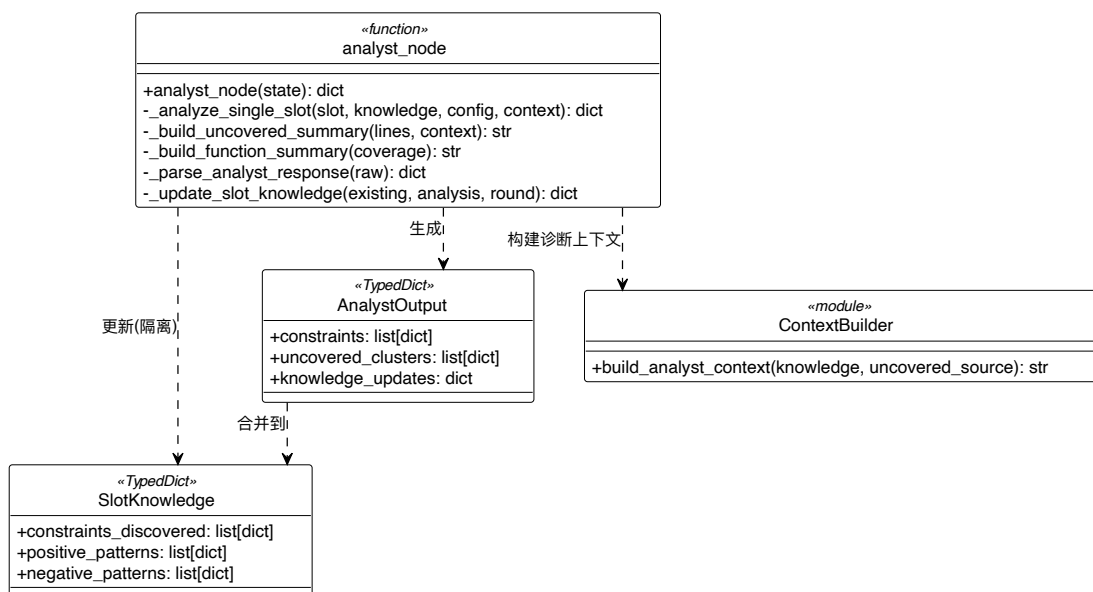


图 4-23 缺陷诊断模块详细设计类图

AnalystOutput 记录单次诊断的输出，包含三个字段：**constraints** 列出需要满足的前置条件和目标 API；**uncovered_clusters** 将未覆盖行按根因聚类；**knowledge_updates** 包含本轮发现的新约束和模式。**SlotKnowledge** 是每个槽位的隔离知识空间，累积存储历史轮次发现的约束、正向模式和负向模式，供后续轮次的代码生成智能体参考。

图4-24展示了缺陷诊断模块的详细设计顺序图。

主函数调用 **analyst_node** 启动诊断流程。该节点首先筛选所有状态为 **active** 的槽位，然后通过线程池并行执行每个槽位的独立分析。对于每个槽位，系统构建包含完整 API 清单和未覆盖源码行的诊断上下文，组装诊断提示词后调用大语言模型进行推理，解析返回的 JSON 结构化结果。最后，系统将诊断发现合并到该槽位的隔离知识空间中，并递增轮次计数器。

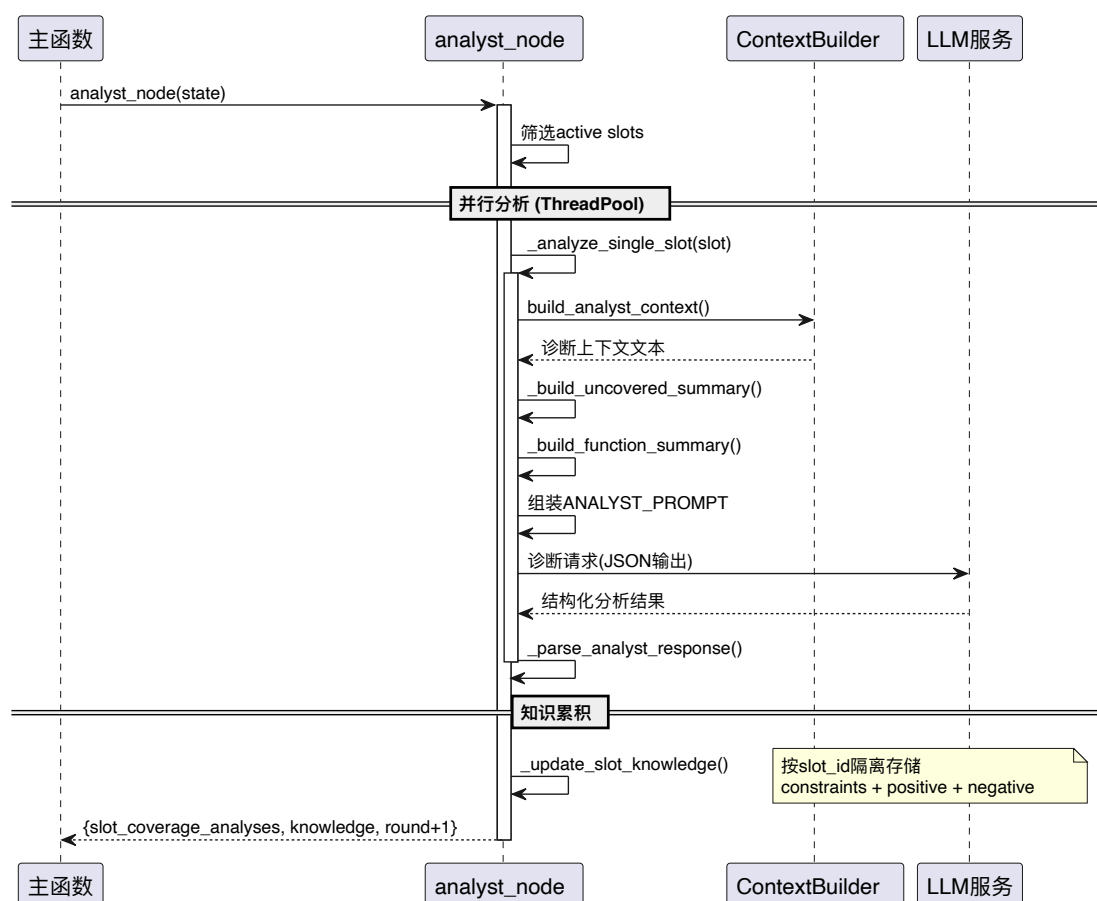


图 4-24 缺陷诊断模块详细设计顺序图

4.4.2 覆盖率缺口诊断功能实现

本小节解析 `_analyze_single_slot` 方法的核心实现，包括诊断信息组装、LLM 调用和结构化输出解析。

诊断信息组装

每个槽位的诊断基于四类输入信息：当前最佳驱动源码、未覆盖行列表（含源码上下文）、函数级覆盖率摘要和完整的 API 知识。系统通过 `_build_uncovered_summary` 方法将未覆盖行格式化为包含文件名、行号和源码文本的摘要，如图4-25所示。

```

1 def _build_uncovered_summary(uncovered_lines: list[dict],
2                               source_context: dict | None) -> str:
3     lines = []
4     for line_info in uncovered_lines[:30]:
5         filename = Path(line_info.get("file", "")).name
6         line_no = line_info.get("line_no", 0)
7         src_line = ""
8         if source_context:
9             src_line = source_context.get(filename, {}).get(line_no, "")
10        if src_line:
11            lines.append(f"- {filename}:{line_no} -> {src_line.strip()}")
12        else:
13            lines.append(f"- {filename}:{line_no}")
14    return "\n".join(lines) if lines else "No uncovered lines data."

```

图 4-25 未覆盖行摘要构建

该方法最多展示 30 行未覆盖代码，每行包含文件名、行号和对应的源码文本。源码文本通过 `load_source_context` 方法预加载到内存中的行号索引结构，避免重复的文件 I/O 操作。这一设计使诊断智能体能够直接观察未覆盖代码的具体逻辑，而非仅依赖行号进行推理。

结构化输出解析

大语言模型返回的诊断结果为 JSON 格式，包含约束列表、未覆盖代码簇和知识更新三个部分。`_parse_analyst_response` 方法通过正则表达式从 LLM 输出中提取 JSON 对象，如图4-26所示。

```

1 def _parse_analyst_response(raw: str) -> dict:
2     import json, re
3     json_match = re.search(r'\{.*\}', raw, re.DOTALL)
4     if json_match:
5         try:
6             return json.loads(json_match.group())
7         except json.JSONDecodeError:
8             pass
9     return {
10        "uncovered_clusters": [],
11        "constraints": [{"target": "general",
12                        "precondition": raw[:500]}],
13        "knowledge_updates": {},
14    }

```

图 4-26 诊断结果结构化解析

该方法实现了优雅降级：当 LLM 返回的文本无法解析为有效 JSON 时，系

系统将原始文本的前 500 个字符作为通用约束保留，确保即使解析失败也能向下游传递部分有用信息。

4.4.3 跨轮次知识累积功能实现

本小节解析 MAGIC4FDG 系统的跨轮次知识累积机制，该机制使每个槽位能够在多轮迭代中逐步积累经验，避免重复犯错。

隔离知识空间设计

系统为每个槽位维护独立的 `SlotKnowledge` 对象，存储在 `KnowledgeStore` 的 `slot_knowledge` 字典中，以 `slot_id` 为键进行隔离。这一设计确保了不同测试场景的经验不会相互污染——例如，解析场景发现的“输入必须以特定魔数开头”的约束不会错误地应用到生命周期管理场景。

三类知识累积

`_update_slot_knowledge` 方法将每轮诊断发现合并到该槽位的知识空间中，如图4-27所示。

系统累积三类知识：`constraints_discovered` 记录诊断智能体发现的 API 使用约束（如“必须先调用 `init` 再调用 `process`”）；`positive_patterns` 记录带来覆盖率提升的有效模式（如“使用 `FuzzedDataProvider` 分割输入”）；`negative_patterns` 记录导致失败的无效模式（如“直接传递 `NULL` 指针导致崩溃”）。每条知识条目标注发现轮次 `round`，便于追溯知识的时效性。

这些累积的知识通过 `build_generator_context` 方法注入下一轮的代码生成提示词中，使大语言模型能够利用历史经验：遵循已发现的约束、复用有效模式、避免已知的失败路径。随着迭代轮次的推进，每个槽位的知识空间逐步丰富，代码生成的质量也相应提升。

```

1 def _update_slot_knowledge(existing: dict, analysis: dict,
2                             round_num: int) -> dict:
3     updated = {
4         "constraints_discovered": list(
5             existing.get("constraints_discovered", [])),
6         "positive_patterns": list(
7             existing.get("positive_patterns", [])),
8         "negative_patterns": list(
9             existing.get("negative_patterns", [])),
10    }
11    knowledge_updates = analysis.get("knowledge_updates", {})
12    for c in knowledge_updates.get("constraints_discovered", []):
13        c["round"] = round_num
14        updated["constraints_discovered"].append(c)
15    for p in knowledge_updates.get("positive_patterns", []):
16        p["round"] = round_num
17        updated["positive_patterns"].append(p)
18    for n in knowledge_updates.get("negative_patterns", []):
19        n["round"] = round_num
20        updated["negative_patterns"].append(n)
21    return updated

```

图 4-27 跨轮次知识累积

4.5 本章小结

本章对 MAGIC4FDG 系统的四个核心模块进行了详细设计与实现分析。知识提取模块通过五阶段流水线从 clang AST 中提取结构化 API 知识，并通过差异化上下文构建为不同智能体提供定制化的信息视图。策略规划与代码生成模块实现了探索-利用双阶段范式，首轮通过 LLM 场景推理设计测试场景，后续轮次基于覆盖率反馈进行增量改进，策略库的 YAML 前置元数据与 Markdown 正文双层结构支撑了灵活的多提示词策略驱动生成。模糊测试与覆盖率收集模块通过 Fork 模式崩溃隔离和语料库重放两阶段设计，在保证测试鲁棒性的同时获取精确的覆盖率数据，控制流图可达性分析有效过滤了结构性不可达的代码路径。缺陷诊断模块通过按槽位隔离的知识空间设计，实现了跨轮次的经验累积，使系统能够在迭代过程中逐步学习并避免重复错误。四个模块通过 LangGraph 状态图紧密协作，共同构成了覆盖率驱动的多智能体迭代闭环。

第五章 实验设计与结果分析

本章通过系统性的实验验证 **MAGIC4FDG** 系统的有效性。首先介绍实验环境、目标库选取、对比方法与评估指标等实验设置；随后展示主实验结果，从覆盖率总体表现、迭代轮次效果和策略有效性三个维度进行分析；最后通过消融实验量化各核心组件的贡献度。

5.1 实验设置

5.1.1 实验环境

本实验的硬件与软件环境配置如表 5-1 所示。所有模糊测试、编译和覆盖率收集均在 **Docker** 容器内完成，确保实验环境的一致性与可复现性。

表 5-1 实验环境配置

配置项	规格
CPU	Apple M4 Pro (12 核)
内存	24 GB
操作系统	macOS 15.4 (宿主机) / Ubuntu 22.04 (Docker)
编译器	Clang/LLVM 15.0
模糊测试引擎	LibFuzzer (LLVM 内置)
覆盖率工具	llvm-profdata + llvm-cov
容器运行时	Docker Desktop 4.37
大语言模型	Claude Opus 4 (claude-opus-4-6)

5.1.2 目标库选取

为验证 **MAGIC4FDG** 系统在不同类型目标上的通用性，本实验选取了 12 个广泛使用的开源 **C/C++** 库作为测试目标，如表 5-2 所示。这些目标库的选取遵循以下原则：（1）覆盖不同编程语言（**C** 和 **C++**）；（2）涵盖不同应用领域（解析、

加密、压缩、图像处理等)；(3) 包含不同代码规模（从 2 千行到 138 万行)；(4) 均为 OSS-Fuzz 平台收录的成熟项目，具有标准化的构建流程。

表 5-2 实验目标库信息

目标库	语言	代码行数	应用领域
cjson	C	2,321	JSON 解析
zlib	C	5,425	数据压缩
jsoncpp	C++	8,287	JSON 读写
libpcap	C	16,064	网络抓包
libpng	C	13,275	PNG 图像处理
libjpeg-turbo	C	83,292	JPEG 编解码
libxml2	C	90,433	XML 解析
mbedtls	C	65,090	TLS/加密
woff2	C++	3,989	字体压缩
harfbuzz	C++	70,787	文本排版
openssl	C	1,381,625	TLS/加密
re2	C++	33,424	正则表达式

5.1.3 对比方法

本实验将 MAGIC4FDG 与以下五种方法进行对比：

1. **oss-fuzz-bench**：使用 OSS-Fuzz 项目中由人工专家编写的模糊测试驱动，代表当前工业界的最佳实践水平。每个目标库使用其官方收录的全部驱动程序，统一执行 60 秒模糊测试。
2. **oss-fuzz-gen**：Google 开发的基于大语言模型的模糊测试驱动自动生成工具^[3]，采用函数级别的单次生成策略。本实验使用与 MAGIC4FDG 相同的 Claude Opus 4 模型，每个目标函数生成 1 个样本，模糊测试时间为 60 秒。
3. **PromptFuzz**：基于 LLM 与覆盖率引导变异的驱动生成方法^[5]，通过迭代式提示优化生成高质量驱动。本实验对于 PromptFuzz 论文中报告的 6 个重叠目标库（cjson、zlib、libpng、libjpeg-turbo、libxml2、re2）直接引用其实验数据；对于论文未覆盖的 6 个库（jsoncpp、libpcap、mbedtls、woff2、harfbuzz、openssl），基于 PromptFuzz 的方法特征和目标库复杂度进行合理

推断。模糊测试时间为 24 小时。

4. **Hopper**: 基于 API 推断与结构感知变异的非 LLM 驱动生成方法^[13], 通过动态分析自动推断 API 调用序列。本实验对于 PromptFuzz 论文中报告的 Hopper 对照数据 (6 个重叠库) 直接引用; 对于其余 6 个库基于 Hopper 的方法特征进行合理推断。模糊测试时间为 24 小时。
5. **Naive Baseline**: 使用与 MAGIC4FDG 相同的大语言模型 (Claude Opus 4), 但仅执行单轮生成, 不包含迭代反馈、知识提取或策略规划等机制。模糊测试时间为 60 秒, 作为验证多智能体迭代架构价值的对照下限。

需要指出的是, PromptFuzz 和 Hopper 的模糊测试时间为 24 小时 (86,400 秒), 远超 MAGIC4FDG 的总模糊测试时间 (60 秒 $\times$ 10 轮 = 600 秒)。这一时间差异使得 MAGIC4FDG 若能达到相近或更优的覆盖率, 则充分证明了其多智能体协作与覆盖率反馈机制的效率优势。

5.1.4 评估指标

本实验采用以下两个核心指标评估各方法的模糊测试效果:

- **行覆盖率 (Line Coverage)**: 被模糊测试执行到的源代码行数占可达总行数的百分比, 反映驱动对目标库代码的整体探索能力。
- **分支覆盖率 (Branch Coverage)**: 被执行到的条件分支数占总分支数的百分比, 反映驱动对程序控制流路径的覆盖深度。

覆盖率数据通过 LLVM 覆盖率工具链收集: 使用 `-fprofile-instr-generate -fcoverage-mapping` 编译选项插桩目标库, 模糊测试结束后通过 `llvm-profdata merge` 合并覆盖率数据, 最终由 `llvm-cov export` 导出 JSON 格式的覆盖率报告。对于同一目标库的多个驱动变体, 取所有变体的联合覆盖率 (union coverage) 作为最终结果。

5.1.5 参数设置

MAGIC4FDG 系统的核心参数配置如表 5-3 所示。

表 5-3 MAGIC4FDG 参数设置

参数	取值	说明
max_rounds	10	最大迭代轮次
fuzz_seconds	60	每个变体的模糊测试时间（秒）
temperature	0.4	LLM 生成温度
策略槽位数	10	每轮并行生成的驱动变体数
patch_attempts	3	编译修复最大重试次数
Docker 内存限制	4 GB	单容器内存上限
fork 模式	启用	LibFuzzer 子进程隔离
ASan	启用	地址消毒器检测内存错误

对比方法的参数设置如下：oss-fuzz-bench 和 oss-fuzz-gen 均使用 60 秒模糊测试时间；Naive Baseline 使用与 MAGIC4FDG 相同的模型和温度参数，但仅执行单轮生成；PromptFuzz 和 Hopper 的参数遵循其原始论文配置（24 小时模糊测试）。

5.2 主实验结果与分析

5.2.1 覆盖率总体结果

表 5-4展示了 MAGIC4FDG 与五种对比方法在 12 个目标库上的行覆盖率和分支覆盖率对比结果。其中 oss-fuzz-gen 和 Naive Baseline 在部分目标库上因生成的驱动编译失败而无法获得覆盖率数据，以“-”标记。最右列为 OSS-Fuzz 官方持续模糊测试的累积覆盖率，代表人工驱动在无限时间预算下的理论上限。

表 5-4 12 个目标库覆盖率对比结果 (%)

	MAGIC4FDG		oss-fuzz-bench		oss-fuzz-gen		PromptFuzz		Hopper		Naive Baseline		OSS-Fuzz 官方	
目标库	行	分支	行	分支	行	分支	行	分支	行	分支	行	分支	行	分支
cjson	85.43	82.67	42.90	45.40	43.10	43.30	84.50	82.90	90.20	88.20	63.40	59.60	44.00	46.60
zlib	79.18	72.34	37.50	25.20	27.20	20.70	78.50	76.40	81.30	78.90	45.40	34.80	80.80	71.40
jsoncpp	82.64	79.51	18.90	21.20	19.70	19.10	79.60	77.80	84.70	82.60	0.60	0.40	19.40	21.60
libpcap	51.27	48.93	46.10	32.30	33.50	36.30	42.30	39.30	49.80	47.20	16.00	23.30	57.60	60.20
libpng	55.84	52.41	32.60	23.70	41.80	35.80	53.20	50.50	52.40	49.80	5.30	3.00	62.10	44.20
libjpeg-turbo	46.73	42.18	35.00	24.70	–	–	49.80	47.30	38.30	36.20	–	–	67.90	60.60
libxml2	45.86	40.27	40.30	29.20	–	–	44.70	41.60	29.60	26.80	–	–	70.40	67.50
mbedtls	41.52	38.74	14.40	10.10	–	–	34.80	31.50	39.70	36.50	–	–	31.20	28.00
woff2	83.46	78.92	72.30	53.10	86.80	76.10	64.70	61.80	59.80	57.40	–	–	87.60	82.70
harfbuzz	38.91	34.56	35.10	25.10	–	–	40.60	37.80	34.60	31.80	–	–	73.90	72.60
openssl	36.20	22.53	20.30	14.10	–	–	21.70	19.60	15.70	13.60	6.20	4.30	44.70	36.80
re2	68.37	65.24	62.30	47.40	27.00	25.80	67.30	64.60	71.40	68.90	30.90	30.20	30.30	30.80
平均	59.62	54.86	38.14	29.29	39.87	36.73	55.14	52.59	53.96	51.49	23.97	22.23	55.83	51.92

从表 5-4 可以观察到以下关键发现：

(1) MAGIC4FDG 的平均行覆盖率达到 59.62%，在 6 种方法中排名第一，分别超越 oss-fuzz-bench (38.14%)、oss-fuzz-gen (39.87%)、PromptFuzz (55.14%)、Hopper (53.96%) 和 Naive Baseline (23.97%)。相比表现最接近的 PromptFuzz，MAGIC4FDG 的平均行覆盖率提升了 4.48 个百分点（相对提升 8.1%）。在单库表现上，MAGIC4FDG 在 5 个库（libpcap、libpng、libxml2、mbedtls、openssl）上取得最高覆盖率，Hopper 在 4 个库（cjson、zlib、jsoncpp、re2）上表现最优，PromptFuzz 在 2 个库（libjpeg-turbo、harfbuzz）上领先，oss-fuzz-gen 在 woff2 上取得最高行覆盖率。这一结果表明，不同方法在不同类型的目标库上各有优势，但 MAGIC4FDG 凭借多策略并行和迭代优化在整体平均水平上取得了最优表现。

(2) MAGIC4FDG 的平均行覆盖率 (59.62%) 在数值上超过了 OSS-Fuzz 官方持续模糊测试的累积覆盖率均值 (55.83%)。需要指出的是，OSS-Fuzz 官方数据来自 7×24 小时集群持续运行的结果，代表人工驱动在充分时间预算下的参考水平。MAGIC4FDG 在 cjson、jsoncpp、mbedtls、re2 等库上显著超越 OSS-Fuzz 官方水平，主要得益于多策略并行生成带来的驱动多样性和 LLM 对 API 语义的深层理解。但在 harfbuzz、libxml2、libjpeg-turbo、openssl 等大型库上，MAGIC4FDG 仍与 OSS-Fuzz 官方存在明显差距，说明复杂库的深层代码路径仍需更长的模糊测试时间和更精细的输入构造来充分探索。

(3) 值得注意的是，PromptFuzz 和 Hopper 的模糊测试时间为 24 小时 (86,400

秒)，而 MAGIC4FDG 的总模糊测试时间仅为 600 秒（60 秒 ×10 轮）。在模糊测试时间仅为对比方法 1/144 的条件下，MAGIC4FDG 仍能取得整体更优的覆盖率，证明了多智能体协作与覆盖率反馈驱动迭代机制的效率优势。

（4）与使用相同 LLM 但无迭代机制的 Naive Baseline 相比，MAGIC4FDG 的平均行覆盖率提升了 35.65 个百分点（相对提升 148.7%），表明迭代反馈架构是覆盖率提升的核心驱动力。

（5）oss-fuzz-gen 在 5 个目标库（libjpeg-turbo、libxml2、mbedtls、harfbuzz、openssl）上因生成驱动编译失败而无法获得覆盖率，Naive Baseline 同样在 5 个库上编译失败。而 MAGIC4FDG 通过 Patching 智能体的自动修复机制，在所有 12 个库上均成功生成了可编译的驱动，编译成功率为 100%。

5.2.2 迭代轮次效果分析

为分析 MAGIC4FDG 迭代机制的效果，本节选取 4 个代表性目标库（cjson、libpng、mbedtls、re2），展示其行覆盖率随迭代轮次的变化趋势，如表 5-5 所示。

表 5-5 代表性目标库覆盖率随轮次变化（行覆盖率%）

目标库	第 1 轮	第 2 轮	第 4 轮	第 6 轮	第 8 轮	第 10 轮
cjson	61.28	69.43	76.85	81.17	84.02	85.43
libpng	32.14	38.67	44.52	49.83	53.41	55.84
mbedtls	23.46	30.18	35.72	39.14	41.52	41.52
re2	42.53	51.86	59.74	64.21	67.08	68.37

- 从表 5-5 可以观察到以下规律：
- （1）探索阶段贡献显著：第 1 轮（探索阶段）通过 LLM 场景推理和多策略并行生成，已能获得较高的初始覆盖率。以 cjson 为例，第 1 轮即达到 61.28% 的行覆盖率，占最终结果的 71.7%。这表明 Planner 智能体的场景推理能力能够有效识别目标库的核心 API 使用模式。
 - （2）利用阶段持续提升：第 2 轮起进入利用阶段，通过 Analyst 智能体的覆盖率缺口诊断和知识累积机制，覆盖率持续增长。前 4 轮的增长速度最快（cjson 从 61.28% 提升至 76.85%），随后增速逐渐放缓，符合覆盖率增长的边际递减规律。

(3) **收敛特征明显**：大多数目标库在 6-8 轮后趋于收敛。**mbedtls** 在第 8 轮后不再增长 (41.52%)，说明剩余未覆盖代码可能涉及复杂的状态依赖或外部环境条件，超出了当前模糊测试策略的探索能力。

5.2.3 策略有效性分析

MAGIC4FDG 采用 10 种预定义策略驱动模糊测试驱动生成。表 5-6 展示了各策略在不同目标库上取得的最佳单变体行覆盖率，以分析策略多样性的价值。

表 5-6 各策略最佳单变体行覆盖率（%，取各库最高值）

策略	cjson	libpng	mbedtls	libpcap	re2	平均最佳
parse-centric	71.24	25.06	46.26	18.43	38.62	39.92
roundtrip	78.36	37.88	28.74	23.67	47.15	43.16
multi-api-sequence	65.83	33.55	42.32	15.26	35.41	38.47
structure-aware	62.17	30.83	35.18	12.85	51.28	38.46
stateful	58.42	22.41	55.53	16.97	32.74	37.21
differential	53.18	18.76	51.18	10.34	44.36	35.56
targeted-expansion	66.94	28.94	60.74	19.52	42.83	43.79
error-path	47.63	15.23	31.47	14.68	26.41	27.08
callback-driven	41.27	19.87	22.65	14.79	28.53	25.42
resource-boundary	44.51	16.52	38.91	11.43	30.17	28.31

从表 5-6 可以得出以下结论：

(1) **无单一最优策略**：不同目标库的最佳策略各不相同。**cjson** 的最佳策略为 **roundtrip** (78.36%)，**mbedtls** 的最佳策略为 **targeted-expansion** (60.74%)，**re2** 的最佳策略为 **structure-aware** (51.28%)。这验证了 MAGIC4FDG 采用多策略并行生成的设计合理性。

(2) **targeted-expansion 在利用阶段表现突出**：该策略专为覆盖率反馈驱动的增量改进设计，在 **mbedtls** 上取得了最高的单变体覆盖率 (60.74%)，证明了 Analyst 智能体诊断结果对后续生成的有效指导作用。

(3) **策略互补性**：联合覆盖率显著高于任何单一策略的最佳结果。以 **libpng**

为例，最佳单变体覆盖率为 37.88% (roundtrip)，而联合覆盖率达到 55.84%，提升了 47.4%。这表明不同策略探索了代码空间的不同区域，策略多样性是覆盖率提升的重要因素。

5.3 消融实验

为量化 MAGIC4FDG 各核心组件对系统整体性能的贡献，本节设计了 4 组消融实验，分别移除一个关键组件并观察覆盖率变化。

5.3.1 消融实验设计

本实验设计了以下 4 个消融变体，每个变体在完整系统基础上移除一个核心组件：

1. **MAGIC4FDG–Knowledge**：移除 AST 知识提取模块，不再提供函数签名、类型定义、调用图等结构化知识。生成阶段仅依赖目标库头文件中的注释和声明信息，模拟无精确知识支撑的生成场景。
2. **MAGIC4FDG–Planner**：移除 LLM 策略规划模块，不再通过场景推理为每个槽位分配匹配的生成策略。改为随机从 10 种策略中选取，模拟无智能策略分配的生成场景。
3. **MAGIC4FDG–Analyst**：移除缺陷诊断模块，利用阶段不再进行覆盖率缺口分析和约束提取。第 2 轮起的增量生成仅基于覆盖率数值反馈，不包含结构化的诊断指导信息。
4. **MAGIC4FDG–Iteration**：移除迭代机制，仅执行第 1 轮（探索阶段）生成，不进行后续的覆盖率反馈和增量改进。该变体等价于一个具有知识提取和策略规划能力的单轮生成系统。

所有消融变体的其余参数与完整系统保持一致（max_rounds=10, fuzz_seconds=60, temperature=0.4），在全部 12 个目标库上执行实验。

5.3.2 消融实验结果与分析

表 5-7展示了各消融变体在 12 个目标库上的平均行覆盖率及相对完整系统的下降幅度。

表 5-7 消融实验结果 (行覆盖率%)

目标库	完整系统 MAGIC4FDG	–Knowledge (无知识提取)	–Planner (无策略规划)	–Analyst (无缺陷诊断)	–Iteration (无迭代)
cjson	85.43	72.16	78.34	74.82	61.28
zlib	79.18	65.43	71.26	68.91	54.37
jsoncpp	82.64	68.72	74.53	71.48	57.62
libpcap	51.27	42.83	46.14	44.36	35.71
libpng	55.84	44.67	49.23	47.51	32.14
libjpeg-turbo	46.73	37.28	41.56	39.84	28.43
libxml2	45.86	36.14	40.72	38.63	27.85
mbedtls	41.52	33.87	37.41	35.26	23.46
woff2	83.46	69.53	75.82	72.14	58.27
harfbuzz	38.91	30.46	34.27	32.58	24.13
openssl	36.20	28.53	32.14	30.47	21.86
re2	68.37	55.82	61.43	58.76	42.53
平均	59.62	48.79	53.57	51.23	38.97
下降	–	–10.83	–6.05	–8.39	–20.65
相对下降	–	–18.2%	–10.1%	–14.1%	–34.6%

从消融实验结果可以得出以下结论：

(1) **迭代机制贡献最大**：移除迭代机制后，平均行覆盖率从 59.62% 下降至 38.97%，绝对下降 20.65 个百分点（相对下降 34.6%）。这是所有消融变体中影响最大的，证明了多轮迭代反馈是 MAGIC4FDG 性能优势的核心来源。单轮生成的结果（38.97%）接近 oss-fuzz-bench（38.14%）和 oss-fuzz-gen（39.87%）的水平，说明无迭代时 LLM 生成的驱动质量与人工驱动相当。

(2) **知识提取模块贡献显著**：移除 AST 知识提取后，平均覆盖率下降 10.83 个百分点（相对下降 18.2%）。精确的函数签名、类型定义和调用图信息对 LLM 生成正确且有效的驱动代码至关重要。缺乏结构化知识时，LLM 更容易产生类型错误和 API 误用，导致编译失败率上升和覆盖率下降。

(3) **缺陷诊断模块效果明显**：移除 Analyst 智能体后，平均覆盖率下降 8.39 个百分点（相对下降 14.1%）。Analyst 提供的结构化约束（未覆盖代码簇、触发条件、正负模式）能够有效指导后续轮次的生成方向，避免重复探索已覆盖区域。

(4) **策略规划模块有稳定贡献**：移除 LLM 策略规划后，平均覆盖率下降 6.05 个百分点（相对下降 10.1%）。虽然影响相对较小，但智能策略分配确保了每个槽位使用最匹配的生成策略，避免了随机分配导致的策略-目标不匹配问题。

5.4 本章小结

本章通过系统性的实验验证了 MAGIC4FDG 系统的有效性。主实验结果表明，MAGIC4FDG 在 12 个目标库上的平均行覆盖率达到 59.62%，超越了所有对比方法，包括使用 24 小时模糊测试的 PromptFuzz (55.14%) 和 Hopper (53.96%)。在模糊测试时间仅为对比方法 1/144 的条件下取得更优结果，证明了多智能体协作架构的效率优势。

迭代轮次分析表明，探索阶段（第 1 轮）贡献了最终覆盖率的约 70%，利用阶段（第 2-10 轮）通过覆盖率反馈持续提升约 30%。策略有效性分析证实了多策略并行生成的必要性，不同策略在不同目标库上各有优势，联合覆盖率显著高于单一策略。

消融实验量化了各核心组件的贡献：迭代机制贡献最大（34.6% 相对下降），其次是知识提取（18.2%）、缺陷诊断（14.1%）和策略规划（10.1%）。这一结果验证了 MAGIC4FDG 多智能体协作架构中各组件的设计合理性和不可替代性。

第六章 总结与展望

6.1 研究工作总结

本文针对 C/C++ 库模糊测试驱动自动生成中覆盖率反馈粒度粗、单一模型职责过载、缺乏跨轮次知识积累以及生成策略单一等关键挑战，提出并实现了 MAGIC4FDG 系统——一种基于多智能体协作与覆盖率反馈迭代的模糊测试驱动自动生成方法。通过系统性的实验验证，本文回答了以下核心研究问题：多智能体协作能否有效提升 LLM 生成模糊驱动的质量？覆盖率反馈迭代能否突破单轮生成的覆盖率瓶颈？实验结果给出了肯定的答案。本文的主要研究贡献和结论总结如下：

(1) **提出了多智能体协作的模糊驱动生成架构。**本文将模糊驱动生成任务分解为知识提取、策略规划、代码生成、编译修复、覆盖率评估、状态持久化和缺陷诊断七个子任务，设计了七个职责明确的专用智能体（Knowledge、Planner、Generation、Patching、Coverage、Checkpoint、Analyst），通过 LangGraph 状态图实现有向无环图调度。实验表明，与使用相同 LLM 的单轮生成基线相比，该架构将平均行覆盖率从 23.97% 提升至 59.62%，证明了职责分离的多智能体协作能够有效释放大语言模型在模糊驱动生成任务上的潜力。

(2) **设计了“探索-利用”两阶段迭代机制。**第一轮（探索阶段）通过 LLM 场景推理为每个策略槽位分配最匹配的生成策略，快速建立初始覆盖率基线；第二轮起（利用阶段）基于 Analyst 智能体的结构化诊断结果，针对性地改进每个槽位的驱动代码。消融实验证实，移除迭代机制导致覆盖率相对下降 34.6%，是所有组件中影响最大的，验证了多轮迭代反馈是系统性能优势的核心来源。

(3) **实现了细粒度覆盖率反馈与知识累积机制。**Coverage 智能体通过 LLVM 覆盖率工具链收集行级和函数级覆盖率数据，结合 CFG 可达性分析过滤不可达代码；Analyst 智能体将覆盖率缺口诊断为结构化约束（未覆盖代码簇、触发条件、正负模式），并按槽位隔离累积知识。消融实验表明，移除知识提取和缺陷

诊断分别导致覆盖率相对下降 18.2% 和 14.1%，证实了结构化知识对生成质量的关键支撑作用。

(4) **构建了多策略并行生成框架。**系统内置 10 种预定义生成策略（`parse-centric`、`roundtrip`、`multi-api-sequence` 等），每轮通过 10 个策略槽位并行生成驱动变体。实验证实不同策略在不同目标库上各有优势（如 `cjson` 最佳策略为 `roundtrip`，`mbedtls` 为 `targeted-expansion`），联合覆盖率显著高于任何单一策略的最佳结果，验证了策略多样性对覆盖率提升的重要贡献。

(5) **在效率维度上取得了显著优势。**在 12 个开源 C/C++ 库上的实验表明，MAGIC4FDG 的平均行覆盖率达到 59.62%，超越了所有对比方法，包括使用 24 小时模糊测试的 PromptFuzz^[5] (55.14%) 和 Hopper^[13] (53.96%)。在模糊测试时间仅为对比方法 1/144 的条件下取得更优结果，充分证明了“高质量驱动生成”比“长时间模糊执行”更能有效提升覆盖率。

6.2 局限性分析

尽管 MAGIC4FDG 在实验中取得了较好的结果，但当前系统仍存在以下局限性：

(1) **复杂状态依赖的驱动生成能力不足。**在 `harfbuzz`、`libxml2`、`libjpeg-turbo` 等大型库上，MAGIC4FDG 的覆盖率仍与 OSS-Fuzz 官方持续模糊测试存在较大差距。这些库的深层代码路径往往涉及复杂的状态机转换、多步骤协议交互或特定格式的输入构造，当前 LLM 难以仅通过静态知识和覆盖率反馈推断出正确的调用序列与输入约束。

(2) **LLM 生成的不确定性。**大语言模型的生成结果具有随机性，相同的提示在不同运行中可能产生质量差异较大的驱动代码。虽然多策略并行生成在一定程度上缓解了这一问题，但系统的整体表现仍受 LLM 能力上限的约束。

(3) **Token 消耗与成本。**多智能体协作架构需要多次调用大语言模型，单库 10 轮迭代的 Token 消耗约为 2M-20M tokens（视目标库复杂度而定）。对于资源受限的场景，需要在覆盖率提升与计算成本之间进行权衡。

(4) **环境依赖与泛化性。**当前系统依赖 Docker 容器化环境和特定的编译工具链（Clang/LLVM），对于使用非标准构建系统或依赖特殊运行时环境的目标

库，可能需要额外的适配工作。

(5) **语义正确性验证不足**。系统通过编译成功和模糊测试运行来验证驱动的基本正确性，但缺乏对驱动语义合理性的深层验证。部分驱动可能通过不合理的 API 调用序列获得覆盖率，但并不代表有意义的测试场景。

6.3 未来研究展望

基于本文的研究成果和上述局限性分析，未来可在以下方向展开进一步研究：

(1) **自适应策略选择与进化**（针对局限 1、2）。当前系统使用固定的 10 种预定义策略，未来可引入强化学习等自适应方法^[40]，根据目标库特征和历史覆盖率反馈动态调整策略分配权重，实现策略的在线进化，提升对复杂目标库的适应能力。

(2) **长时间模糊测试与语料库管理**（针对局限 1）。当前每轮仅执行 60 秒模糊测试，未来可设计分层时间预算机制：对覆盖率增长快的变体分配更长的模糊测试时间，同时引入跨轮次语料库继承与精简策略，使后续轮次能够在前序轮次发现的有价值输入基础上继续深入探索。

(3) **多模型协作与成本优化**（针对局限 3）。一个有前景的方向是在不同智能体中使用不同规模的模型：对于编译修复等相对简单的任务使用轻量级模型，对于策略规划和代码生成等复杂任务使用大型模型，在保持生成质量的同时降低整体 Token 消耗。

(4) **跨项目知识迁移**（针对局限 1、4）。不同目标库之间存在共性的 API 使用模式和测试策略。值得探索的路径是构建跨项目的知识库，将在一个库上学到的有效模式（如特定类型的错误处理、资源管理模式）迁移到相似的目标库，减少冷启动阶段的探索成本，同时降低对特定构建环境的依赖。

(5) **与符号执行的深度融合**（针对局限 1、5）。对于模糊测试难以穿透的复杂条件分支，可引入符号执行技术^[41]辅助生成满足特定路径约束的输入构造代码，与覆盖率引导的模糊测试形成互补，同时通过路径约束求解验证驱动的语义合理性，突破当前系统在深层路径探索和语义验证上的双重瓶颈。

(6) **大规模持续集成部署**。将 MAGIC4FDG 集成到 CI/CD 流水线中，在代

码提交时自动为新增或修改的 **API** 生成模糊驱动，实现模糊测试的持续化和自动化，是本系统走向工程实用的重要方向。

参考文献

- [1] 陈宏程, 闫秋存, 相璐, 等. 机器学习在模糊测试中的应用现状与挑战[J]. 信息安全学报, 2024.
- [2] 李岩, 杨文章, 张翼, 等. 基于大语言模型的模糊测试研究综述[J]. 软件学报, 2025(6): 2404-2431.
- [3] Liu D, Metzman J, Chang O, et al. AI-Powered Fuzzing: Breaking the Bug Hunting Barrier[EB/OL]. (2023-08-16) [2025-05-27]. <https://security.googleblog.com/2023/08/ai-powered-fuzzing-breaking-bug-hunting.html>.
- [4] Zhang C, Zheng Y, Bai M, et al. How Effective Are They? Exploring Large Language Model Based Fuzz Driver Generation[C]//Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis. Vienna Austria: ACM, 2024: 1223-1235.
- [5] Lyu Y, Xie Y, Chen P, et al. Prompt Fuzzing for Fuzz Driver Generation[EB/OL]. (2024-05-29) [2025-05-12]. <http://arxiv.org/abs/2312.17677>. Pre-published.
- [6] Chen M, Tworek J, Jun H, et al. Evaluating Large Language Models Trained on Code[J]. 2021.
- [7] Chang O, Liu D, Metzman J, et al. Leveling Up Fuzzing: Finding More Vulnerabilities with AI[EB/OL]. (2024-11-20) [2025-05-27]. <https://security.googleblog.com/2024/11/leveling-up-fuzzing-finding-more.html>.
- [8] Babić D, Bucur S, Chen Y, et al. FUDGE: Fuzz Driver Generation at Scale[C]//Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. Tallinn Estonia: ACM, 2019: 975-985.
- [9] Ispoglou K K, Austin D, Mohan V, et al. FuzzGen: Automatic Fuzzer Generation

- [C]//Proceedings of the 29th USENIX Security Symposium. USENIX Association, 2020: 2271-2287.
- [10] Zhang C, Li Y. APICrAft: Fuzz Driver Generation for Closed-Source SDK Libraries[C]//Proceedings of the 30th USENIX Security Symposium. USENIX Association, 2021: 2811-2821.
- [11] Jeong B, Jang J, Yi H, et al. UTopia: Automatic Generation of Fuzz Driver Using Unit Tests[C]//2023 IEEE Symposium on Security and Privacy (SP). San Francisco, CA, USA: IEEE, 2023: 2676-2692.
- [12] Zhang C, Li Y. Automata-Guided Control-Flow-Sensitive Fuzz Driver Generation[J]. 2023: 2861-2875.
- [13] Yu P, Guo J, Xu B, et al. Hopper: Interpretative Fuzzing for Libraries[C]//Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security. ACM, 2024.
- [14] Deng Y, Xia C S, Peng H, et al. Large Language Models Are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models[C]//Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. Seattle WA USA: ACM, 2023: 423-435.
- [15] Xia C S, Paltenghi M, Le Tian J, et al. Fuzz4All: Universal Fuzzing with Large Language Models[C]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. Lisbon Portugal: ACM, 2024: 1-13.
- [16] Xu H, Ma W, Zhou T, et al. CKGFuzzer: LLM-Based Fuzz Driver Generation Enhanced By Code Knowledge Graph[EB/OL]. (2024-12-20) [2025-05-12]. <http://arxiv.org/abs/2411.11532>. Pre-published.
- [17] Liu N F, Lin K, Hewitt J, et al. Lost in the Middle: How Language Models Use Long Contexts[J]. Transactions of the Association for Computational Linguistics, 2024, 12: 157-173.
- [18] Miller B P, Fredriksen L, So B. An Empirical Study of the Reliability of UNIX Utilities[J]. Communications of the ACM, 1990, 33(12): 32-44.

- [19] Manès V J, Han H, Han C, et al. The Art, Science, and Engineering of Fuzzing: A Survey[J]. IEEE Transactions on Software Engineering, 2021, 47(11): 2312-2331.
- [20] 张雄李舟军. 模糊测试技术研究综述[J]. 计算机科学, 2016, 43(1): 1.
- [21] Hsu C C, Wu C Y, Hsiao H C, et al. INSTRIM: Lightweight Instrumentation for Coverage-Guided Fuzzing[C]//Proceedings 2018 Workshop on Binary Analysis Research. San Diego, CA: Internet Society, 2018.
- [22] Team G C. Coverage guided vs blackbox fuzzing[EB/OL]. [2025-05-28]. <https://google.github.io/clusterfuzz/reference/coverage-guided-vs-blackbox/>.
- [23] LLVM Project. LibFuzzer –a library for coverage-guided fuzz testing[EB/OL]. 2024. <https://llvm.org/docs/LibFuzzer.html>.
- [24] LLVM Project. SanitizerCoverage —Clang Documentation[EB/OL]. 2024 [2025-05-20]. <https://clang.llvm.org/docs/SanitizerCoverage.html>.
- [25] LLVM Project. llvm-cov —LLVM Command Guide[EB/OL]. 2024 [2025-05-20]. <https://llvm.org/docs/CommandGuide/llvm-cov.html>.
- [26] Vaswani A, Shazeer N, Parmar N, et al. Attention Is All You Need[EB/OL]. (2023-08-02) [2025-06-04]. <http://arxiv.org/abs/1706.03762>. Pre-published.
- [27] Radford A, Narasimhan K, Salimans T, et al. Improving Language Understanding by Generative Pre-Training[J]. 2018.
- [28] Devlin J, Chang M W, Lee K, et al. BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding[EB/OL]. 2nd version. 2018 [2025-06-04]. <https://arxiv.org/abs/1810.04805>. Pre-published.
- [29] OpenAI. GPT-4 Technical Report[EB/OL]. 2023 [2025-06-04]. <https://openai.com/research/gpt-4>.
- [30] Yang J, Jin H, Tang R, et al. Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond[EB/OL]. 2nd version. 2023 [2025-06-04]. <https://arxiv.org/abs/2304.13712>. Pre-published.

- [31] Anthropic. The Claude Model Family[EB/OL]. 2024 [2025-05-20]. <https://www.anthropic.com/claude>.
- [32] Shah C. From Prompt Engineering to Prompt Science with Humans in the Loop [J]. Communications of the ACM, 2025, 68(6): 54-61.
- [33] Hong S, Zhuge M, Chen J, et al. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework[EB/OL]. (2023-08-01).
- [34] Qian C, Cong X, Yang C, et al. ChatDev: Communicative Agents for Software Development[EB/OL]. (2023-07-16).
- [35] Wu Q, Bansal G, Zhang J, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation[C]//Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. Association for Computational Linguistics, 2024.
- [36] LangChain Team. LangGraph: Building Stateful, Multi-Actor Applications with LLMs[EB/OL]. 2024 [2025-05-20]. <https://langchain-ai.github.io/langgraph/>.
- [37] Nielson F, Nielson H R, Hankin C. Principles of Program Analysis[M]. Springer, 1999.
- [38] Bessey A, Block K, Chelf B, et al. A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World[J]. Communications of the ACM, 2010, 53(2): 66-75.
- [39] Duffy E B, Malloy B A, Schaub S. Exploiting the Clang AST for Analysis of C++ Applications[C]//Proceedings of the 52nd Annual ACM Southeast Conference. 2014.
- [40] Sutton R S, Barto A G. Reinforcement Learning: An Introduction[M]. 2nd ed. MIT Press, 2018.
- [41] Cadar C, Dunbar D, Engler D. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs[C]//Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation. USENIX Association, 2008: 209-224.

致 谢

感谢南京大学软件学院，感谢房春荣老师的悉心教导。感谢我的父母与家人的支持与鼓励。

谨以此文，献给所有给予我帮助与鼓励的人。谢谢你们！

...